

Hybrid AC/DC Module Technical Notebook

Current C++ Architecture, Information Models, Mathematical Models, and Solver Workflows

HACDCPF C++ Module

May 23, 2026

Contents

1 Scope and Documentation Contract

This notebook is the implementation-facing handbook for the current `hacdcpf` C++ module. It connects mathematical models to the public data structures, preprocessing steps, solver paths, optional build features, and result objects that are present in the repository now. It deliberately separates implemented runtime code from declared stubs and absent/planned APIs.

1.1 Implemented Runtime Families

The current notebook covers these implemented module families:

- the public model layer under `include/hacdcpf/model`;
- canonical projection and solver-data assembly in `src/model` and `src/power_flow`;
- hybrid AC/DC power flow through `solve_power_flow`;
- DC-only, FDPF, linearized AC/DC, adaptive/islanded, distributed-slack, HELM, and continuation PF entry points exposed in the power-flow headers;
- AC OPF through `solve_ac_opf`, including native reduced-space and parity full-space IPM routes;
- DC OPF through `solve_dc_opf`;
- reactive-power optimization through `solve_rpo`;
- JSON, MATPOWER/JPC, case-builder, PF result JSON, OPF result JSON, and carbon result JSON support;
- optional Excel workbook I/O when `HACDCPF_ENABLE_EXCEL=ON`;
- optional OpenDSS bridge source inclusion when `HACDCPF_ENABLE_OPENDSS=ON`.

1.2 Current Stub and Absent Boundaries

Area	Current code status
<code>analysis::solve_distribution_power_flow</code>	Declared and compiled, but <code>src/analysis/distribution_power_flow.cpp</code> returns a non-converged result with <code>status="not_implemented"</code> .
<code>run_distribution_resilience_assessment</code>	Public facade exists, but <code>src/analysis/resilience_assessment.cpp</code> returns <code>completed=false</code> and <code>status="not_implemented"</code> .
<code>analysis::run_carbon_analysis</code>	Result structs and JSON/Excel serializers exist; the analysis source currently returns <code>tracing_verified=false</code> , <code>matrix_solved=false</code> , and empty result arrays.
Time-series PF	<code>time_series_pf.hpp</code> defines data types; <code>time_series_pf.cpp</code> is a stub translation unit and does not implement a simulation routine.
Reliability assessment	Headers define option/result types; no implemented public assessment solver is exposed by <code>hacdcpf.hpp</code> .
Short-circuit analysis	No public <code>compute_short_circuit</code> , <code>SCOptions</code> , or <code>BusFaultResult</code> header exists in the current include tree.
Optimal network reconfiguration	No public <code>solve_optimal_reconfiguration</code> header/API exists in the current include tree.
Annual production and lifecycle simulation	No public annual-production or lifecycle-simulation solver APIs exist in the current include tree.

1.3 Documentation Principle

This notebook follows four rules.

1. Equations are written in the same sign convention as the current C++ implementation.
2. Field explanations are tied to concrete structs, not generic textbook entities.
3. Each component is classified by how it enters a module: direct state, fixed injection, projected equivalent, or not yet consumed.
4. Extension points declared in headers are explicitly separated from functionality that is already wired in source files.

1.4 Canonical Modeling Pipeline

The steady-state solvers do not consume every rich component table directly. Instead, the dominant runtime path is:

1. start from a public `HybridPowerSystem`,
2. call `project_to_canonical_models`,
3. build `SolverData` through `make_solver_data`,
4. assemble residuals, Jacobians, or optimization models for PF and OPF.

The projection stage currently performs the following transformations:

- `FlexibleLoad` → equivalent `Load`,
- `AsymmetricLoad` → equivalent aggregated `Load`,
- `Transformer2W` and `Transformer3W` → one or more equivalent `ACBranch` objects,
- `Switch` → equivalent `ACBranch` whose `in_service` flag follows the switch state,
- `CircuitBreaker` → equivalent `ACBranch`, followed by zero-impedance bus merging when switches or circuit breakers are present,
- `Charger` → accumulation into `ChargingStation` demand,
- `VirtualPowerPlant` and `Microgrid` → equivalent `AC StaticGenerator` rows,
- `MobileStorage` → equivalent `AC Storage` at its current bus,
- `EnergyRouter` → internal DC buses plus VSC/DCDC equivalents,
- optional `ThreePhaseACSystem` → canonical single-phase-equivalent AC models when the explicit AC system is absent.

1.5 Important Boundary Between “Implemented” and “Declared”

The parity OPF formulation now instantiates **all** enhanced component decision blocks as runtime variables with full Jacobian and Hessian wiring:

- generator dispatch (P_g, Q_g) ,
- VSC converter power $(P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}})$,
- DCDC converter input power P^{dc} with analytical I^2R loss derivatives,
- DC bus voltages v^{dc} ,
- per-bus load shedding $(\Delta P_d, \Delta Q_d)$ with VOLL penalty,
- curtailable renewable dispatch $(P^{\text{ren}}, Q^{\text{ren}})$ for `RenewableGen` with `curtailable=true` and `PVSystem` with `controllable=true`,
- storage dispatch $(P^{\text{stor}}, Q^{\text{stor}})$ for AC storage and $P^{\text{stor,dc}}$ for DC storage,
- flexible load adjustment P^{flex} .

These are no longer extension hooks: they are wired decision variable blocks with explicit mapping arrays in `parity_formulation.cpp`. The OPF JSON serializer preserves the enhanced dispatch arrays as both legacy bare arrays and structured dispatch objects with identity maps.

2 Nomenclature

2.1 Notation Policy

This notebook uses a strict notation policy consistent with the C++ module:

- scalar variables use italic math symbols, for example P_i, Q_i, V_i ,
- sets use calligraphic symbols, for example $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,

- all semantic qualifiers use upright superscripts via $\cdot$, for example $P_i^{\text{spec}}, P_i^{\text{calc}}, V_k^{\text{dc}}$,
- for variable symbols, subscripts are reserved for ordered indices (bus/component/time) and not for semantic class labels,
- set symbols may use compact domain tags in subscript, e.g., $\mathcal{N}_{ac}, \mathcal{E}_{dc}$,
- model-family and semantic words must stay in superscripts: write $P^{\text{dcdc}}, P^{\text{flex}}, P_r^{\text{rated}}, c_r^{\text{curt}}$, and $\mathcal{R}^{\text{curt}}$.

2.2 Index and Qualifier Conventions

Token	Meaning
i, j	AC bus indices
k, m	DC bus indices
c	converter index (VSC / DCDc)
b	branch index
g	generator index
spec	specified (assembled) quantity from component injections/demands
calc	network-calculated quantity from current state
ac, dc	AC-side / DC-side quantity qualifier
conv	converter-side quantity qualifier
dcdc, flex	DCDC-dispatch / flexible-load variable class qualifier
rated, curt	rated-capacity / curtailment-related qualifier
min, max	lower/upper operating bounds

2.3 Canonical Naming Grammar

Use the canonical symbol template

$$X_{\text{indices}}^{\text{qualifiers}}, \quad (1)$$

with the following rules:

- index block ($\cdot$): ordered indices only ($i, j, k, c, r, t, \dots$),
- qualifier block ($\cdot$): semantic tags only (spec, calc, conv, dcdc, flex, rated, curt, min, max, ...),
- when multiple qualifiers are needed, order them as: model/class $\rightarrow$ side or direction $\rightarrow$ status/bound/cost.

Examples:

$$P_i^{\text{conv,ac}}, P_k^{\text{dcdc,in}}, P_r^{\text{rated}}, c_r^{\text{curt}}. \quad (2)$$

2.4 Superscript/Subscript Rule

Rule	Usage
Subscript ($\cdot$)	for variable symbols: ordered indices only (i, j, k, c, r, t); set symbols may use domain tags ($\mathcal{N}_{ac}, \mathcal{E}_{dc}$).
Superscript ($\cdot$)	semantic labels: domain, role, operating mode, variable class, bounds, and status qualifiers.
Forbidden mixed style	avoid semantic words in subscript, such as $P_{dcdc}, P_{flex}, \mathcal{R}_{curt}, P_{rated}, c_{curt}$.
Preferred style	$P^{dcdc}, P^{flex}, \mathcal{R}^{curt}, P_r^{rated}, c_r^{curt}$.

2.5 Integration Status Codes

The integration matrix uses these codes:

- D: direct model consumption in the module,
- F: fixed-only consumption (no explicit variable),
- P: consumed after projection to canonical equivalents,
- N: not consumed by the module.

3 Current C++ Architecture

This section describes the repository as it exists in the current C++ module. The library target is `hacdcpf`, configured from the top-level `CMakeLists.txt` as a C++20 static library. The build links against `mipsolvers::mipsolvers`; that dependency supplies the shared engine types, solver adapters, and numerical backends used by the OPF and RPO code.

3.1 Source Layout

Directory	Current role
<code>include/hacdcpf/model</code>	Public data structures for AC, DC, converter, switching, three-phase, and result models.
<code>include/hacdcpf/api</code>	High-level public API facade in <code>hacdcpf.hpp</code> .
<code>include/hacdcpf/power_flow</code> and <code>src/power_flow</code>	Hybrid Newton PF, DC PF, FDPF, adaptive/islanded/distributed-slack variants, HELM, continuation, Jacobian assembly, and PF utilities.
<code>include/hacdcpf/optimal_power_flow</code> and <code>src/optimal_power_flow</code>	AC OPF routing, parity formulation, parity IPM, DC OPF, and reactive-power optimization.
<code>include/hacdcpf/power_models</code> and <code>src/power_models</code>	Builder-style models for AC OPF, AC/DC OPF, DC OPF, LinDistFlow, and SCUC experiments.
<code>include/hacdcpf/io</code> and <code>src/io</code>	MATPOWER/JSON/JPC case I/O, optional Excel workbook I/O, optional OpenDSS bridge, and case builders.

Directory	Current role
<code>include/hacdcpf/analysis</code> and <code>src/analysis</code>	Declared analysis families; several source files are currently stubs and are called out in Section ??.
<code>include/hacdcpf/engine</code>	Compatibility/forwarding headers into <code>mipsolvers::engine</code> , plus the <code>hacdcpf</code> Newton PF class declaration. There is no <code>src/engine</code> directory in this repository.

3.2 Top-Level Public API

The facade header `include/hacdcpf/api/hacdcpf.hpp` currently exposes:

- PF: `solve_power_flow`, `solve_dc_power_flow`, `solve_power_flow_adaptive`, `solve_power_flow_islanded`, `solve_power_flow_distributed_slack`, `solve_power_flow_distributed_slack_full`, `solve_power_flow_fdpf`, and `solve_ac_dc_power_flow`;
- persistent PF handles: `create_solver_handle`, `reset_solver_handle`, `solve_handle`, `solve_dc_handle`, and `destroy_solver_handle`;
- OPF: `solve_ac_opf`, `solve_dc_opf`, and `solve_rpo`;
- analysis facade: `run_distribution_resilience_assessment`, which is currently backed by a stub implementation.

Lower-level namespaces expose additional solver objects and helper entry points. The power-flow namespace includes `solve_hybrid`, `solve_hybrid_adaptive`, `solve_ac`, `solve_dc`, `FDPFSolver`, `AdaptiveSolver`, `DistributedSlackSolver`, `HelmSolver`, `CpfSolver`, `HomotopyContinuationSolver`, and the header-only `solve_three_phase` wrapper.

3.3 Build-Time Optional Features

CMake option	Default	Current effect
<code>HACDCPF_BUILD_TESTS</code>	ON	Enables the Catch2 test targets under <code>tests/</code> .
<code>HACDCPF_ENABLE_EXCEL</code>	OFF	Adds <code>src/io/excel_io.cpp</code> ; otherwise <code>HACDCPF_NO_EXCEL</code> is defined and the public Excel functions throw.
<code>HACDCPF_ENABLE_OPENDSS</code>	OFF	Adds <code>src/io/opendss_bridge.cpp</code> ; otherwise OpenDSS bridge functionality is absent from the build.

3.4 Algorithm Index

The notebook uses the following code-aligned algorithm labels.

Label	Main implementation	Purpose
A1	<code>src/model/network_utils.cpp</code>	Rich-model projection to canonical solver inputs.

Label	Main implementation	Purpose
A2	<code>src/power_flow/solver_data.cpp</code>	SolverData construction, injection aggregation, and matrix build.
A3	<code>src/power_flow/newton_solver.cpp</code>	Hybrid AC/DC Newton power flow.
A4	<code>src/power_flow/fdpf_solver.cpp</code>	AC fast-decoupled power flow.
A5	<code>src/power_flow/dc_solver.cpp</code>	DC-only power flow.
A6	<code>src/power_flow/adaptive_solver.cpp</code>	Island-aware PF orchestration.
A7	<code>src/power_flow/distributed_slack_solver.cpp</code>	Simplified and full distributed-slack PF.
A8	<code>src/optimal_power_flow/acopf.cpp</code>	AC OPF top-level routing and fallback selection.
A9	<code>src/optimal_power_flow/parity_formulation.cpp</code>	Full-space parity NLP construction and derivatives.
A10	<code>src/optimal_power_flow/parity_ipm.cpp</code>	Mehrotra/Gondzio-style parity IPM solve loop.
A11	<code>src/optimal_power_flow/dcopf.cpp</code>	DC OPF QP/LP model build and backend dispatch.
A12	<code>src/optimal_power_flow/reactive_power_opt.cpp</code>	Reactive-power optimization by OA/coordinate search.
A13	<code>src/power_flow/homotopy_continuation.cpp</code>	Continuation Power Flow (CPF) predictor–corrector (§??).
A14	<code>src/power_flow/homotopy_continuation.cpp</code>	Homotopy continuation fallback solver (§??).
A15	<code>src/power_flow/newton_krylov.cpp</code>	Newton–GMRES Krylov fallback with Schur preconditioner (§??).
A16	<code>src/power_models/lindistflow_builder.cpp</code>	Linearised DistFlow LP builder (§??).
A17	<code>src/power_models/acopf_builder.cpp</code>	AML AC OPF model builder (§??).
A18	<code>src/power_models/acdcopf_builder.cpp</code>	AML hybrid AC/DC OPF model builder (§??).
A19	<code>src/power_models/scuc_builder.cpp</code>	MILP Security-Constrained Unit Commitment builder (§??).

4 Engine Solver System

The `hacdcpf::engine` namespace in this repository is mainly a compatibility facade over `mipsolvers::engine`. The local headers keep historical include paths stable while importing engine symbols with `using` declarations. Current implementation files for LCQP, dual simplex, branch and cut, and external adapters live in the `mipsolvers` dependency, not under `src/engine` in this repository.

4.1 Forwarded Engine Surface

Local forwarding header	Imported symbols / role
<code>engine/problem_types.h</code> <code>pp</code>	Problem-class enums, symbolic model structs, LP/QP/MIP/NLP/MINLP model types, <code>SolveStats</code> , and <code>SolveResult</code> .
<code>engine/kernel/ipm/lcqp_solver.hpp</code>	<code>LCQPOptions</code> and <code>NativeLCQPAdapter</code> .
<code>engine/kernel/lp_kernel/dual_simplex.hpp</code>	Simplex options, basis/result types, <code>StandardFormLP</code> , and <code>solve_lp_with_basis</code> .
<code>engine/kernel/linear_algebra/linear_solver.hpp</code>	Sparse solver abstraction and Eigen-based sparse solver factories.
<code>engine/kernel/globalization/globalization.hpp</code>	Dogleg, trust-region update, and pseudo-transient continuation helpers.
<code>engine/solver/external/adapters.hpp</code>	<code>HiGHSAdapter</code> , <code>IpoptAdapter</code> , <code>ScipAdapter</code> , and <code>GurobiAdapter</code> .
<code>engine/branch_and_cut.hpp</code>	Branching and node-selection enums plus B&C statistics/result/option types.

There are no local PDLP or LP-IPM backends documented by the current headers. The backends visible through this compatibility layer are LCQP, dual simplex, HiGHS, Ipopt, SCIP, and Gurobi, subject to the configuration of `mipsolvers` and the optional external packages available at build time.

4.2 Local Newton Solver Class

The one local engine-style solver declaration is `include/hacdcpf/engine/solver/native/nle/newton_solver.hpp`. Its implementation is `src/power_flow/newton_solver.cpp`; it is the hybrid PF Newton class used by the power-flow module. The file is not an engine implementation directory and does not define the optimization-engine LCQP or LP backends.

4.3 OPF Use of Engine Facilities

Consumer	Engine dependency	Current behavior
DC OPF	LCQP, Gurobi, HiGHS, dual simplex	Auto tries <code>NativeQP</code> , then Gurobi, then HiGHS, then native dual simplex. LMP arrays are currently filled with zeros because dual extraction is marked TODO in <code>dc_opf.cpp</code> .
RPO	<code>BCStats</code> only for result reporting	The current source runs an OA/coordinate-search workflow and reuses B&C statistics fields for counters; it does not call a generic branch-and-bound solve loop.

Consumer	Engine dependency	Current behavior
Parity AC OPF	local parity IPM	Uses the local KKT assembly and Eigen dense/sparse factorizations in <code>parity_ipm.cpp</code> .
Native AC OPF	local reduced-space KKT loop	Uses custom KKT assembly and factorization in <code>ac_opf.cpp</code> .

5 Native NLP and Interior-Point Details

The nonlinear OPF solvers are local to `src/optimal_power_flow`. They do not dispatch through an external NLP engine by default. The AC OPF entry point selects either a native reduced-space primal-dual path or the parity full-space IPM path described in Section ??.

5.1 Native AC OPF Loop

The native path in `ac_opf.cpp` uses variables for non-slack AC angles, PQ voltage magnitudes, generator dispatch, DC voltage magnitudes, and VSC converter powers. Equality rows cover AC balances, DC balances, and VSC energy balance. Bound constraints are handled with barrier terms, while nonlinear branch and converter limits enter through exterior penalty terms. If this path fails and `ACOPFOptions::allow_fallback` is true, the code falls back to the fast dispatch plus AC PF projection route.

5.2 Parity IPM Loop

The parity path builds explicit equality and inequality rows in `parity_formulation.cpp` and solves the resulting full-space NLP in `parity_ipm.cpp`. The solve loop uses a Mehrotra predictor-corrector step with second-order complementarity correction, iterative refinement, and fraction-to-boundary primal/dual step lengths. Current centering is

$$\sigma = \left(\frac{\mu_{aff}}{\mu_{cur} + 10^{-16}} \right)^3 \quad (3)$$

after clamping the ratio to $[0, 1]$. The implementation also applies up to two extra Gondzio corrector solves when enabled by the current loop logic.

The KKT system is solved with Eigen `SparseLU` on sparse systems or `PartialPivLU` on dense systems. There is no separate local NLP/IPM engine source tree beneath `src/engine`.

6 Native IPM Versus Parity IPM

The two AC OPF nonlinear routes differ mainly in variable coverage and in how constraints are represented.

Feature	Native reduced-space path	Parity full-space path
Primary files	<code>ac_opf.cpp</code>	<code>parity_formulation.cpp</code> , <code>parity_ipm.cpp</code>
AC bus variables	non-slack angles and PQ magnitudes	full AC angle and magnitude blocks with bounds
Generator dispatch	P_g, Q_g variables	P_g, Q_g variables

Feature	Native reduced-space path	Parity full-space path
VSC converters	explicit P^{ac}, Q^{ac}, P^{dc} variables and balance rows	explicit P^{ac}, Q^{ac}, P^{dc} variables and balance rows
DC bus voltages	variables, with tight bounds for DC_V buses	variables with DC balance rows and DC branch limits
Load shedding	not modeled	$\Delta P_d, \Delta Q_d$ variables with VOLL penalty
Curtaillable renewable / PV	fixed injections	packed decision vectors pre _{n_mw} , qren _{mvar} with identity maps
Storage	fixed injection	AC and DC storage active dispatch packed into pstor _{mw} ; AC storage also has qstor _{mvar}
DCDC converters	not modeled as decision variables	input-side pdc _{dc} _{mw} variables folded into DC bus balance physics
Flexible loads	projected/fixed demand	pf _{flex} _{mw} variables with identity maps
Failure fallback	fast dispatch plus AC PF if allowed	native path if allowed, then native fallback behavior

The parity result stores enhanced component outputs in packed arrays. The maps **ren**_{map}, **stor**_{map}, **dc**_{dc}_{map}, and **flex**_{map} must be used to join those arrays back to the original component collections. For storage, **source_type**=0 denotes AC storage and **source_type**=1 denotes DC storage.

One implementation quirk is important for code review: **ConstraintIndex** still allocates **n**_{dc}_{dc}_{bal} and **i**_{dc}_{dc}_{bal}, and **build_problem** includes those rows in **n**_{eq}_{total}. The current equality evaluator leaves the dedicated DCDC rows unpopulated; DCDC physics is represented instead through the DC bus balance rows.

7 Symbols, Indexing, and Sign Convention

7.1 Electrical Bases

$$S_{base} = \text{system apparent-power base (MVA)}, \quad (4)$$

$$V_{base} = \text{nominal line-line voltage base (kV)}, \quad (5)$$

$$Z_{base} = \frac{V_{base}^2}{S_{base}}, \quad (6)$$

$$I_{base} = \frac{S_{base}}{\sqrt{3}V_{base}}. \quad (7)$$

7.2 Sets and Indexing

$$\mathcal{N}_{ac} : \text{AC buses}, \quad \mathcal{E}_{ac} : \text{AC branches}, \quad (8)$$

$$\mathcal{N}_{dc} : \text{DC buses}, \quad \mathcal{E}_{dc} : \text{DC branches}, \quad (9)$$

$$\mathcal{N}_{sl} : \text{AC slack buses}, \quad \mathcal{N}_{PQ} : \text{AC PQ buses}, \quad (10)$$

$$\mathcal{N}_{PV} : \text{AC PV buses}, \quad \mathcal{N}_{sl}^{dc} : \text{DC voltage-controlled buses}. \quad (11)$$

The public model stores external bus identifiers in each component (`bus`, `from_bus`, `to_bus`). Most solver kernels then remap those identifiers into contiguous zero-based local indices.

7.3 Core AC and DC State Variables

$$V_i = v_i e^{j\theta_i}, \quad (12)$$

$$Y_{ij} = G_{ij} + jB_{ij}, \quad (13)$$

$$V_k^{dc} = v_k^{dc}, \quad (14)$$

$$P_k^{dc,calc} = v_k^{dc} (G_{dc} v^{dc})_k. \quad (15)$$

For the main Newton power-flow path, the state vector is

$$x_{PF} = \begin{bmatrix} \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} & v_{\mathcal{N}_{PQ}} & v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}^{dc}}^{dc} \end{bmatrix}^\top. \quad (16)$$

For the native primal-dual AC OPF path, the decision vector is extended to include generator dispatch and converter operating points:

$$x_{OPF} = \begin{bmatrix} \theta & v & P_g & Q_g & v^{dc} & P^{conv,ac} & Q^{conv,ac} & P^{conv,dc} \end{bmatrix}^\top, \quad (17)$$

with additional load-shedding variables in the parity formulation.

7.4 Unified Sign Convention: Bus Injection Positive

The current C++ module uses one consistent notation in the PF kernels and the notebook adopts the same rule:

Positive active or reactive power at a bus means injection into the network from the component attached to that bus.

Loads therefore store positive demand values in their data model, but they enter power-balance equations with a minus sign.

Quantity / field	Positive value means	Consequence in equations
<code>Generator::pg_mw</code> , <code>qg_mvar</code>	generator injects active/reactive power into its AC bus	added to P_i^{spec}, Q_i^{spec} .
<code>Load::p_mw</code> , <code>q_mvar</code>	consumption requested from the network	subtracted from P_i^{spec}, Q_i^{spec} ; ZIP dependence scales the subtraction.

Quantity / field	Positive value means	Consequence in equations
<code>Storage::p_mw</code>	discharge to the grid; charging is negative P	treated as generation when positive, load when negative.
<code>Microgrid::p_exchange_mw</code>	export from the microgrid to the host AC grid	added as AC-bus injection in PF; ignored when the microgrid is islanded.
<code>VSCConverter</code> AC-side $P^{\text{conv,ac}}$	converter injects active power into the AC bus	positive AC injection must be balanced by negative DC-bus injection plus losses.
<code>VSCConverter</code> DC-side $P^{\text{conv,dc}}$	converter injects active power into the DC bus	the current code enforces $P^{\text{conv,ac}} + P^{\text{conv,dc}} + P^{\text{loss}} = 0$.
<code>DCDCConverter::p_ref_mw</code>	positive transfer from <code>bus_in</code> toward <code>bus_out</code>	implemented as negative injection at <code>bus_in</code> and positive injection at <code>bus_out</code> .
<code>BranchFlow::pf_mw</code>	active power sent from <code>from_bus</code> to <code>to_bus</code>	reporting only; no sign reversal in stored result.

7.5 Power-Balance Template

With bus-injection-positive notation, the balance equations are written as

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad (18)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad (19)$$

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}. \quad (20)$$

The specified term is assembled by adding all component injections and then subtracting all demand terms. This is the convention followed by the PF residual evaluator, the native AC OPF assembly, and the parity formulation.

8 Information Models and Field Dictionaries

8.1 Modeling Layers Used by the Code

The code base uses three distinct model layers.

1. **Public information model:** the rich structs under `include/hacdcpf/model/` used by I/O, case builders, and external callers.
2. **Canonical steady-state model:** the projected subset generated by `project_to_canonical_models`, where rich components are converted into equivalent loads, branches, or aggregated injections.
3. **Solver assembly model:** `powerflow::SolverData`, which stores contiguous vectors, aggregated per-bus injections, ZIP coefficients, and sparse matrices needed by the PF and OPF kernels.

8.2 System Containers and Solver-Level Metadata

System containers

Struct / group	Fields and meaning	Primary consumers
<code>ACSystem</code>	buses, branches, generators, static_generators, loads, flexible_loads, asymmetric_loads, shunts, storage, renewable_gens, pv_systems, external_grids, transformers_2w, transformers_3w, switches, circuit_breakers, charging_stations, chargers, motors; plus base_mva, freq_hz, name.	PF, OPF, I/O, projection
<code>DCSystem</code>	buses, branches, loads, storage, static_generators, dc_static_generators, pv_arrays, dc_circuit_breakers, plus base_mva and name.	PF, OPF, I/O
<code>HybridPowerSystem</code>	Holds ac, dc, vsc_converters, dcdc_converters, energy_routers, mobile_storage, vpps, microgrids, global base_mva, name, optional three_phase_ac, and optional provenance maps bus_merge_map and branch_expand_map.	all top-level APIs
<code>ThreePhaseACSystem</code>	Optional unbalanced AC subsystem with three-phase buses, lines, transformers, loads, generators, and external grids. It is projected to canonical AC objects when the explicit <code>ACSystem</code> is empty.	projection, PF, I/O

Power-flow options and results

Struct / group	Fields and meaning	Primary consumers
<code>InitialState</code>	vm, va, vdc provide optional warm starts for AC magnitude, AC angle, and DC voltage states.	Newton PF, DC PF
<code>ReactiveLimit</code>	qmin, qmax define reactive limits used by adaptive/PV-PQ logic.	adaptive PF extensions
<code>DistributedSlack</code>	participating_buses, participation_factors, reference_bus, and optional max_participation_p describe how slack mismatch is redistributed.	distributed-slack PF

Struct / group	Fields and meaning	Primary consumers
PowerFlowOptions	Iteration and tolerance controls: <code>max_iter</code> , <code>tol</code> ; robustness controls: line search, regularization, and step clipping; PV/PQ and converter hysteresis parameters; ZIP weights; solver profiling flag; <code>loss_model</code> ; FDPF iteration cap.	PF family
SolverProfiling	Sparse-factorization counters, line-search and regularization statistics, PV/PQ switches, converter-mode switches, timing vectors, and cumulative timing totals.	PF result diagnostics
PowerFlowResult	Solved states <code>vm</code> , <code>va</code> , <code>vdc</code> ; status fields <code>converged</code> , <code>iterations</code> , <code>residual</code> ; and derived reports <code>branch_flows</code> , <code>vsc_transfers</code> , <code>dc_dc_transfers</code> , <code>er_port_transfers</code> .	PF family outputs
AdaptiveSolverResult , IslandedSolveResult , DistributedSlackResult	Variants of PF results that additionally expose detected <code>islands</code> , distributed-slack allocations, and buses hitting slack-sharing limits.	adaptive / islanded / distributed-slack PF

Analysis and OPF options/results

Struct / group	Fields and meaning	Primary consumers
ACOPFOptions	Inner/outer iteration caps, feasibility/stationarity tolerances, barrier schedule, merit-penalty and regularization terms, line-search settings, parallel AC evaluation threads, and path-selection flags <code>enable_primal_dual</code> , <code>use_parity_ipm</code> , <code>allow_fallback</code> .	AC OPF
ACOPFResult	Voltage solution, generator dispatch, optional bus-level load shedding, converter operating points, convergence metrics, status text, and profiling summary.	AC OPF output
DCOPFOptions , DCOPFResult	DC approximation controls, backend selector, branch-limit flag, load-shedding flag, voltage angles, generator dispatch, branch flows, load shedding, zero-filled LMP/vector dual placeholders, and solver status.	DC OPF
RPOOptions , RPOResult	Reactive-power objective settings, search limits, voltage/generator before/after vectors, tap and shunt changes, loss metrics, status, and B&C-style statistics fields reused as counters.	RPO

Struct / group	Fields and meaning	Primary consumers
Analysis result types	Distribution PF, resilience, reliability, time-series, and carbon headers define option/result structs, but the corresponding analysis runtime implementations are currently stubs or type-only surfaces; see Section ??.	analysis declarations

powerflow::SolverData

Group	Fields and meaning	Primary consumers
Canonical component tables	Copies of projected AC/DC buses, branches, generators, converters, DCDC converters, energy routers, loads, DERs, shunts, switches, charging stations, external grids, VPPs, microgrids, and mobile storage.	PF, native OPF
Aggregated steady-state injections	pg , qg hold per-bus generation after aggregation of generators, static generators, renewables, PV, storage, VPPs, microgrids, and mobile storage.	PF, native OPF
Demand model data	has_component_loads , pd_pu , qd_pu , and per-bus ZIP coefficient vectors describe the demand model after load-table aggregation.	PF, native OPF
Network matrices	ybus and gdc store the canonical AC admittance and DC conductance matrices; build_id is used to validate Jacobian-pattern caching.	PF, native OPF
Bookkeeping	base_mva , loss_model , and global ZIP weights copied from PowerFlowOptions .	PF, native OPF

8.3 Component Inventory

Table ?? enumerates the rich public models currently present in the repository.

Table 15: Public component-model inventory

Struct	Mathematical role	Current steady-state integration mode
ACBus	AC state host, voltage limits, embedded bus demand and shunt	direct
ACBranch	AC series/shunt admittance, tap and phase-shift branch	direct
Transformer2W	named two-winding transformer with short-circuit/tap data	projected to ACBranch

Struct	Mathematical role	Current steady-state integration mode
Transformer3W	explicit three-winding transformer with pairwise short-circuit data	projected to three ACBranch equivalents
ExternalGrid	slack source and source-equivalent metadata	direct PF source metadata; richer fault fields are stored but no public SC solver is present
AsynchronousMotor	motor fault-contribution metadata	stored by the model and I/O; no public SC solver currently consumes it
Generator	synchronous generator dispatch and limits	direct
StaticGenerator	fixed/scaled distributed generation	direct fixed injection
RenewableGen	renewable injection with curtailment metadata	fixed injection in PF/native OPF; parity OPF decision variable when curtailable
PVSystem	PV inverter and array metadata	fixed injection in PF/native OPF; parity OPF decision variable when controllable
Load	AC demand with ZIP and motor-fraction metadata	direct
FlexibleLoad	demand-response model	projected to Load for PF; parity OPF has packed flexible-load variables
AsymmetricLoad	per-phase aggregate demand	projected to Load
Shunt	fixed or stepped shunt element	direct fixed shunt in PF; switchable steps are RPO discrete controls
Switch	switching device with electrical/protection data	projected to ACBranch for PF/OPF
CircuitBreaker	high-voltage breaker on an AC bus pair	projected to ACBranch ; can trigger zero-impedance bus merging
ChargingStation	aggregate EV charging demand	direct fixed load
Charger	individual EV charger	aggregated into ChargingStation on projection
Storage	stationary storage power and energy state	fixed injection in PF/native OPF; parity OPF dispatch variable for AC and DC storage
MobileStorage	relocatable storage resource	projected to AC Storage at current bus
VirtualPowerPlant	aggregated DER portfolio	projected to AC StaticGenerator at aggregation bus

Struct	Mathematical role	Current steady-state integration mode
Microgrid	PCC exchange model	projected to AC Static Generator at aggregation bus/PCC when grid-connected
DCBus	DC voltage host and DC demand host	direct
DCBranch	DC resistive branch	direct
DCLoad	DC active demand	direct
StaticGeneratorDC	typed DC active-power source with profile/control metadata	stored in <code>dc.dc_static_generators</code> ; serialized and available for DC injection assembly where supported
PVArrayDC	DC-side photovoltaic array metadata and setpoint	stored in <code>dc.pv_arrays</code> ; serialized and available for DC injection assembly where supported
DCCircuitBreaker	breaker on a DC bus pair	stored and serialized; no projection equivalent currently runs in <code>network_utils.cpp</code>
VSCConverter	AC/DC coupling with loss model and control mode	direct
DCDCConverter	DC/DC transfer device with efficiency and I^2R conduction loss	direct in PF and parity OPF (decision variable with Jacobian and Hessian)
EnergyRouterPort	AC or DC port injection for a multi-port router	direct fixed injection
EnergyRouter	multi-port converter container	direct through its ports
ThreePhaseACBus	optional unbalanced bus model	projected to ACBus
ThreePhaseACLine	optional unbalanced line model	projected to ACBranch using positive sequence
ThreePhaseTransformer	optional unbalanced transformer model	projected to Transformer2W equivalent
ThreePhaseLoad	optional unbalanced load model	projected to Load
ThreePhaseGenerator	optional unbalanced generator model	projected to Generator
ThreePhaseExternalGrid	optional unbalanced slack/grid equivalent	projected to ExternalGrid

8.4 AC Network Elements

ACBus

Field group	Detailed explanation	Used by
Identity and topology	index is the external bus identifier referenced by other components; name is purely descriptive; area and zone are retained for partitioning and reporting.	shared model / solver inputs
Steady-state electrical state	bus_type , vm_pu , va_deg , base_kv , vmax_pu , vmin_pu , in_service . These fields initialize the PF state, define slack/PV/PQ behavior, and bound voltages.	PF, OPF
Embedded demand and shunt	pd_mw , qd_mvar store fallback bus demand when no load table is present; gs_mw , bs_mvar store bus shunt terms.	PF, OPF
Resilience / geography	n_customers , importance , latitude , longitude . These are not yet decision variables, but they are useful for planning logic, GIS, or load-priority extensions.	planning / future use

ACBranch

Field group	Detailed explanation	Used by
Connectivity	index , from_bus , to_bus , in_service , name . Bus references use external bus IDs.	PF, OPF
Positive-sequence electrical data	r_pu , x_pu , b_pu , tap , shift_deg . These define the AC π -model and branch-flow reporting.	PF, OPF
Ratings and physical metadata	rate_a_mva , rate_b_mva , rate_c_mva , length_km . rate_a_mva is used directly by OPF thermal limits.	OPF, reporting
Reliability and sequence data	failure_rate , mttr_hr , r0_pu , x0_pu , b0_pu , and transformer-nameplate fields vn_hv_kv , vn_lv_kv , sn_mva . These are stored for planning/fault-model extensions, but no public SC solver is present.	model/I/O meta-data

Transformer2W, Transformer3W, Switch, and CircuitBreaker

Struct	Field-level explanation	Current treatment
Transformer2W	Identity/connectivity fields <code>index</code> , <code>name</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>in_service</code> ; nameplate fields <code>sn_mva</code> , <code>vn_hv_kv</code> , <code>vn_lv_kv</code> ; short-circuit fields <code>vk_percent</code> , <code>vkr_percent</code> , <code>pk_kw</code> ; no-load fields <code>pf_e_kw</code> , <code>i0_percent</code> ; tap-changer fields <code>tap_side</code> , <code>tap_pos</code> , <code>tap_min</code> , <code>tap_max</code> , <code>tap_neutral</code> , <code>tap_step_percent</code> ; phase shift <code>shift_deg</code> ; zero-sequence fields <code>z0_percent</code> , <code>x0_r0</code> ; reliability fields <code>mtbf_hours</code> , <code>mttr_hours</code> .	projected to one equivalent ACBranch
Transformer3W	Connectivity <code>hv_bus</code> , <code>mv_bus</code> , <code>lv_bus</code> ; per-winding ratings <code>sn_hv_mva</code> , <code>sn_mv_mva</code> , <code>sn_lv_mva</code> ; pairwise short-circuit data <code>vk_hv_mv_percent</code> , <code>vk_hv_lv_percent</code> , <code>vk_mv_lv_percent</code> and the matching <code>vkr</code> fields; tap data and phase shifts <code>shift_mv_deg</code> , <code>shift_lv_deg</code> .	projected to three equivalent ACBranch objects
Switch	Topology fields <code>bus_from</code> , <code>bus_to</code> , <code>closed</code> ; electrical fields <code>r_contact_ohm</code> , <code>z_ohm</code> , <code>i Rated_ka</code> , <code>i_breaking_ka</code> ; protection fields <code>element_type</code> , <code>element_id</code> ; automation fields <code>is_remote</code> , <code>is_automated</code> , <code>t_operation_s</code> ; reliability fields <code>p_sw_fail</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> , <code>t_tp_hr</code> .	projected to equivalent branch with <code>in_service=closed</code>
CircuitBreaker	AC breaker fields <code>index</code> , <code>name</code> , <code>bus_from</code> , <code>bus_to</code> , <code>in_service</code> , <code>breaker_type</code> , <code>closed</code> , <code>z_ohm</code> , <code>rated_voltage_kv</code> , <code>i Rated_ka</code> , <code>i_breaking_ka</code> , <code>element_type</code> , <code>element_id</code> .	projected to equivalent branch; if switches or breakers exist, zero-impedance bus merging may run after projection

ExternalGrid and AsynchronousMotor

Struct	Field-level explanation	Current treatment
ExternalGrid	<code>bus</code> , <code>vm_pu</code> , <code>va_deg</code> establish slack voltage targets; <code>s_sc_max_mva</code> , <code>s_sc_min_mva</code> , <code>rx_max</code> , <code>rx_min</code> , <code>r_pu</code> , <code>x_pu</code> , <code>r0_pu</code> , <code>x0_pu</code> , <code>vn_kv</code> , <code>ikq_ka</code> , <code>x_r</code> store source-equivalent/fault meta-data; <code>controllable</code> can support dispatch logic.	PF source meta-data; additional fields are stored for future analysis
AsynchronousMotor	<code>bus</code> , <code>vn_kv</code> , <code>sn_mva</code> , <code>r_pu</code> , <code>x_pu</code> , <code>x_r</code> , <code>lrc</code> , <code>poles</code> , <code>cos_phi</code> , <code>efficiency</code> , <code>r0_pu</code> , <code>x0_pu</code> . These fields describe possible motor fault contribution, not a steady-state PF state.	model/I/O meta-data

8.5 Load Models and Demand-Side Components

Struct	Field-level explanation	Current treatment
Load	Identity fields <code>index</code> , <code>bus</code> , <code>name</code> , <code>in_service</code> ; steady-state demand <code>p_mw</code> , <code>q_mvar</code> ; model selector <code>model</code> ; ZIP coefficients <code>z_percent_p</code> , <code>i_percent_p</code> , <code>p_percent_p</code> , and the matching <code>q</code> fields; demand-response metadata <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> ; load-priority metadata <code>priority</code> , <code>n_customers</code> , <code>profile_id</code> ; motor-fraction metadata <code>sn_mva</code> , <code>motor_percent</code> , <code>x_sub_pu</code> , <code>r_sc_pu</code> .	direct in PF/OPF
FlexibleLoad	Baseline demand <code>p_mw</code> , <code>q_mvar</code> ; flexibility envelope <code>flex_up_mw</code> , <code>flex_down_mw</code> , <code>flex_duration_h</code> ; response timing <code>response_time_s</code> , <code>ramp_rate_mw_min</code> , <code>availability_pct</code> ; <code>control_area</code> .	projected to equivalent Load for PF; parity OPF has <code>pflex_mw</code> variables
AsymmetricLoad	Per-phase rated values <code>pa_rated_mw</code> , ..., <code>qc_rated_mvar</code> ; per-phase actual values <code>pa_mw</code> , ..., <code>qc_mvar</code> ; <code>connection</code> , <code>grounded</code> , <code>scaling</code> ; ZIP shares <code>const_z_percent</code> , <code>const_i_percent</code> , <code>const_p_percent</code> .	projected to aggregated Load
DCLoad	<code>p_mw</code> is the direct active demand at a DC bus; <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> , and <code>profile_id</code> are control metadata for future extensions.	direct in DC PF and hybrid OPF balances
ChargingStation	<code>bus</code> , <code>n_fast</code> , <code>n_slow</code> , <code>num_chargers</code> ; capacity fields <code>p_fast_max_kw</code> , <code>p_slow_max_kw</code> , <code>max_power_kw</code> ; utilization fields <code>simultaneity_factor</code> , <code>power_factor</code> , <code>utilization_rate</code> ; aggregated operating demand <code>p_total_kw</code> , <code>q_total_kvar</code> .	direct fixed load
Charger	<code>station_id</code> links the charger to its parent station; <code>charger_type</code> , <code>p_rated_kw</code> , <code>p_ch_max_kw</code> , <code>p_ch_min_kw</code> , <code>eta</code> , <code>v2g_capable</code> , <code>p_dis_max_kw</code> .	aggregated into ChargingStation during projection

8.6 Generation, Storage, and Aggregation Components

Struct	Field-level explanation	Current treatment
Generator	Dispatch/state fields <code>pg_mw</code> , <code>qg_mvar</code> , <code>vg_pu</code> , <code>is_slack</code> ; limits <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; quadratic cost coefficients <code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code> ; unit-commitment metadata <code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code> , <code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code> ; machine/stability data <code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code> , <code>xd_pu</code> , <code>xdp_pu</code> , <code>xdpp_pu</code> , <code>td0p_s</code> , <code>td0pp_s</code> ; SC and environmental metadata <code>ra_pu</code> , <code>vn_kv</code> , <code>cos_phi</code> , <code>xq_pu</code> , <code>xd_xq</code> , <code>x0_pu</code> , <code>r0_pu</code> , emission factors, outage rate, and repair time.	direct
StaticGenerator	Fixed or scaled DER injection <code>p_mw</code> , <code>q_mvar</code> , <code>scaling</code> ; device rating/limits <code>p_rated_mw</code> , <code>sn_mva</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; control metadata <code>controllable</code> , <code>v_ref_pu</code> , <code>k_p</code> , <code>k_q</code> ; SC metadata <code>k</code> , <code>rx</code> .	direct fixed injection
RenewableGen	type, actual output <code>p_mw</code> , <code>q_mvar</code> , rated power <code>p_rated_mw</code> , reactive limits, curtailability flag, curtailment cost, capacity factor, profile pointer, and avoided-emission factor.	fixed injection in PF/native OPF; parity OPF uses <code>pren_mw/qren_mvar</code> for curtailable rows
PVSystem	Actual output <code>p_mw</code> , <code>q_mvar</code> ; inverter limits and control <code>sn_mva</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> , <code>control_mode</code> , <code>controllable</code> , <code>v_ac_set_pu</code> , <code>v_dc_set_pu</code> ; inverter efficiency <code>inverter_eff</code> , <code>loss_percent</code> ; array data <code>num_series</code> , <code>num_parallel</code> , <code>vmpp</code> , <code>impp</code> , <code>voc</code> , <code>isc</code> , <code>alpha_isc</code> , <code>beta_voc</code> ; environmental state <code>irradiance</code> , <code>temperature</code> .	fixed injection in PF/native OPF; parity OPF uses renewable dispatch variables for controllable rows
Storage	Snapshot dispatch <code>p_mw</code> , <code>q_mvar</code> ; power limits <code>p_rated_mw</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code> ; energy state <code>e_rated_mwh</code> , <code>soc_init</code> , <code>soc_min</code> , <code>soc_max</code> ; efficiency <code>eta_charge</code> , <code>eta_discharge</code> , <code>self_discharge_pct</code> ; lifecycle <code>max_cycles</code> , <code>current_cycles</code> , <code>soh</code> , <code>replacement_cost</code> .	fixed injection in PF/native OPF; parity OPF has AC and DC storage dispatch variables
MobileStorage	Reuses storage-style power and energy fields and adds mobility fields <code>is_mobile</code> , <code>status</code> , <code>current_location</code> , <code>target_bus</code> , <code>e_consumption_mwh_km</code> , <code>max_travel_distance_km</code> .	projected to AC Storage at its current bus
VirtualPowerPlant	Fleet counts, aggregated capacities, output fields <code>p_output_mw</code> , <code>q_output_mvar</code> , limits, and ramp rates.	projected to AC StaticGenerator at aggregation bus

Struct	Field-level explanation	Current treatment
Microgrid	PCC/aggregation bus, internal bus list, operating-mode fields <code>operating_mode</code> , <code>islanding_capability</code> , <code>auto_reconnection</code> ; exchange limits and current <code>p_exchange_mw</code> ; aggregate capability metrics and droop/voltage setpoints.	projected to AC <code>StaticGenerator</code> when grid-connected

8.7 DC and Power-Electronic Components

Struct	Field-level explanation	Current treatment
DCBus	<code>index</code> , <code>bus_type</code> , <code>vm_pu</code> , <code>vmax_pu</code> , <code>vmin_pu</code> , <code>pd_mw</code> , <code>in_service</code> , <code>name</code> , <code>base_kv</code> , <code>area</code> , <code>zone</code> , <code>n_customers</code> , <code>importance</code> , <code>is_load</code> , <code>latitude</code> , <code>longitude</code> . <code>pd_mw</code> is used only when the dedicated DC load table is empty.	direct
DCBranch	<code>from_bus</code> , <code>to_bus</code> , <code>r_pu</code> , <code>rate_a_mva</code> , <code>length_km</code> , <code>base_kv</code> , <code>s_max_mva</code> , <code>n_parallel</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> , <code>in_service</code> .	direct
StaticGeneratorDC	<code>index</code> , <code>bus</code> , <code>in_service</code> , <code>name</code> , <code>type</code> , <code>p_set_mw</code> , <code>scaling</code> , <code>profile_id</code> , <code>pmax_mw</code> , <code>pmin_mw</code> , <code>controllable</code> , <code>mtbf_hours</code> , <code>mttr_hours</code> , <code>t_scheduled_hr</code> .	stored in <code>dc.dc_static_generators</code> ; serialized as its own DC source family
PVArrayDC	<code>index</code> , <code>bus</code> , <code>in_service</code> , <code>name</code> , <code>p_set_mw</code> , <code>profile_id</code> , array electrical parameters, environmental state, and reliability fields.	stored in <code>dc.pv_arrays</code> ; serialized as its own DC PV family
DCCircuitBreaker	<code>index</code> , <code>name</code> , <code>bus_from</code> , <code>bus_to</code> , <code>in_service</code> , <code>breaker_type</code> , <code>closed</code> , <code>r_ohm</code> , <code>rated_voltage_kv</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code> , <code>element_type</code> , <code>element_id</code> .	stored and serialized; no current projection equivalent
VSCConverter	Connectivity <code>bus_ac</code> , <code>bus_dc</code> ; control mode <code>control_mode</code> ; topology type; setpoints <code>p_set_mw</code> , <code>q_set_mvar</code> , <code>v_dc_set_pu</code> , <code>v_ac_set_pu</code> ; loss parameters <code>eta</code> , <code>loss_percent</code> , <code>loss_mw</code> ; control gains <code>k_vdc</code> , <code>k_p</code> , <code>k_q</code> ; limits and ratings including <code>vn_ac_kv</code> , <code>vn_dc_kv</code> ; controllability, grid-forming and SC-current metadata, and reliability fields.	direct in PF and OPF; some C++ fields are not serialized by the current JSON writer

Struct	Field-level explanation	Current treatment
DCDCConverter	Connectivity bus_in , bus_out ; operating mode control_mode ; references p_ref_mw (output-side referenced), v_ref_pu ; rating fields sn_mva , vn_in_kv , vn_out_kv ; efficiency eta ; conduction-loss resistance r_eq_pu ; droop gain k_droop ; limits pmax_mw , pmin_mw ; controllable , f_switching_hz , and reliability fields. Sign convention: positive direction is bus_in → bus_out ; PF uses output-referenced p_ref_mw , OPF uses input-referenced P^{in} .	direct in PF and parity OPF; some C++ fields are not serialized by the current JSON writer
EnergyRouterPort	bus , port_type , side , voltage_level_kv , operating injection p_mw , q_mvar , v_pu , efficiency eta ; limits and control metadata.	expanded by projection into internal converter equivalents
EnergyRouter	Container fields router_type , num_ports , ports ; rating/loss fields; control_mode , power_dispatch_strategy , t_scheduled_hr ; device-level limits and reliability.	expanded by projection into internal DC buses, VSCs, and DCDC converters

8.8 Optional Three-Phase Models

Struct	Field-level explanation	Current treatment
ThreePhaseACBus	Per-phase voltages vm_a_pu , va_a_deg , vm_b_pu , ...; per-phase demand and shunts; common voltage limits, area, and zone.	projected to ACBus by phase summation / averaging
ThreePhaseACLine	Positive- and zero-sequence per-km parameters, optional precomputed per-unit values, parallel count, thermal rating, and reliability data.	projected to ACBranch using positive-sequence values
ThreePhaseTransformer	Two-winding transformer with vector group, positive- and zero-sequence short-circuit data, tap changer, and phase shift.	projected to Transformer2W equivalent before branch projection
ThreePhaseLoad	Connection type, grounding flag, per-phase powers, ZIP shares, and motor fraction.	projected to aggregated Load
ThreePhaseGenerator	Per-device dispatch and limits plus sequence impedances xd_pu , xdpp_pu , x2_pu , x0_pu , r0_pu .	projected to Generator
ThreePhaseExternalGrid	Per-sequence grid-equivalent impedances and short-circuit capacities.	projected to ExternalGrid

Struct	Field-level explanation	Current treatment
<code>ThreePhaseBusResult</code>	Per-phase solved voltage and injection report. The current one-line notebook path documents the model, but the main steady-state solvers still operate on projected single-phase equivalents.	reporting / future unbalanced workflows

9 I/O Contracts, Sheet Dictionaries, and JSON Keys

The model dictionaries in the previous section explain the physical meaning of each field. This section documents the external I/O contract actually exposed by the current C++ implementation: JSON path names, Excel sheet names, column names, defaulting rules, aggregated views, and the current serialization gaps.

9.1 Serialization Families

API entry point	Artifact	Direction	Practical behavior
<code>to_json / from_json</code>	one JSON document for <code>HybridPowerSystem</code>	bidirectional	covers the major public model families attached to <code>HybridPowerSystem</code> , including circuit breakers, DC static generators, DC PV arrays, DC circuit breakers, <code>ac.motors</code> , and optional <code>three_phase_ac</code> ; several C++ struct fields are still not serialized.
<code>load_excel / save_excel</code>	public Excel API declared in <code>excel_io.hpp</code>	bidirectional when enabled	without <code>HACDCPF_ENABLED_EXCEL</code> , both inline functions throw; with Excel enabled, the header declares these names, while the current source defines the workbook helpers below under <code>load_xlsx/save_xlsx</code> .
source-only <code>save_xlsx / load_xlsx</code>	engineering workbook with one table per supported component class	bidirectional when compiled	implemented in <code>src/io/excel_io.cpp</code> ; input-oriented Excel template plus derived aggregated views such as <code>ACLoad</code> and <code>DCLoad</code> .
source-only PF workbook helpers	dedicated/compact PF result workbooks	export only	<code>src/io/excel_io.cpp</code> implements result workbook writers, but these helper names are not declared in <code>include/hacdcpf/io/excel_io.hpp</code> .

API entry point	Artifact	Direction	Practical behavior
source-only OPF workbook helpers	dedicated/compact OPF result workbooks	export only	write <code>OPF_*</code> sheets; compact OPF export does not append <code>result_*</code> columns into input tables.
<code>power_flow_result_to_json / save_power_flow_result_json</code>	PF result JSON	export only	serializes convergence summary plus AC/DC bus states, AC/DC branch flows, VSC transfers, and DCDC transfers.
<code>opf_result_to_json / opf_result_from_json</code>	AC OPF result JSON	bidirectional	serializes solver status, solver path, primal solution vectors, enhanced dispatch arrays with identity maps, legacy dispatch arrays, and profiling summary.
<code>save_carbon_results_xlsx / carbon_result_to_json</code>	carbon-analysis workbook / JSON	export only for Excel, bidirectional for JSON	uses dedicated report-oriented structures, not the main system model.

9.2 Global Parsing and Workbook Rules

Rule	Current implementation behavior
JSON missing keys	Most JSON importers use <code>jget(..., default)</code> ; missing optional keys are silently defaulted. Whole arrays are optional as well.
JSON required keys	Identity and connectivity fields are usually mandatory: for example <code>index</code> , <code>bus</code> , <code>from_bus</code> , <code>to_bus</code> , <code>hv_bus</code> , <code>lv_bus</code> , <code>bus_ac</code> , and <code>bus_dc</code> .
JSON enums	Enum values are string-coded: for example <code>PQ/PV/SLACK</code> , <code>DC_V/DC_P</code> , <code>VDC_Q</code> , <code>Wind</code> , <code>FuelCell</code> , <code>Recloser</code> , <code>Islanded</code> .
Excel header row	All Excel tables use row 1 as the header row. Data rows start at row 2.
Excel row termination	Readers stop at the first row with an empty <code>index</code> cell. This means blank rows inside a table terminate import early.
Excel missing sheets	<code>load_xlsx</code> silently skips missing sheets; the corresponding component vector remains empty.
Excel boolean parser	<code>TRUE</code> , <code>T</code> , <code>YES</code> , <code>Y</code> , and <code>1</code> map to <code>true</code> ; <code>FALSE</code> , <code>F</code> , <code>NO</code> , <code>N</code> , and <code>0</code> map to <code>false</code> .
Sheet precedence	For AC loads, <code>Load</code> is preferred; <code>ACLoad</code> is used only as a fallback. For DC loads, <code>DCLoadRaw</code> is preferred; <code>DCLoad</code> is only a fallback.
Bus-demand synchronization	After Excel import, <code>sync_bus_demands_from_load_tables(sys)</code> is called so the bus-level embedded demand reflects the explicit load tables.

Rule	Current implementation behavior
Aggregated Excel views	ACBus and DCBus written by the Excel exporters are aggregated bus views, not always the raw struct contents; ACLoad and DCLoad are explicitly derived tables.
Empty result sheets in templates	save_xlsx currently pre-creates the Results_* sheets even though it does not fill them. A freshly exported template therefore contains empty result tabs.

9.3 Hybrid System JSON Contract

Top-level JSON object

JSON path	Value type	Current meaning
name	scalar string	top-level HybridPowerSystem::name .
base_mva	scalar number	top-level system base.
ac	object	AC subsystem object with its own name, base, frequency, and AC component arrays.
dc	object	DC subsystem object with its own name, base, and DC component arrays.
vsc_converters	array	top-level VSC array.
dc.dcdc_converters	array	exported DC/DC converter array nested inside the DC subsystem; import also accepts legacy top-level dcdc_converters .
energy_routers	array	top-level multi-port router array; ports are nested inside each router object.
mobile_storage	array	top-level mobile-storage array.
vpps	array	top-level virtual power plant array.
microgrids	array	top-level microgrid array.
three_phase_ac	object	optional unbalanced AC subsystem with its own base, frequency, and three-phase component arrays.

Current JSON caveats

Public model area	Current JSON status
Standalone energy-router port array	not used; ports appear only as nested objects under each energy_routers[i].ports .
Load short-circuit sub-fields	the embedded SC motor-fraction fields sn_mva , motor_percent , x_sub_pu , and r_sc_pu are still not part of the main ac.loads[] JSON contract.

Public model area	Current JSON status
Excel API names	the public header exposes <code>load_excel/save_excel</code> , but the optional source file currently defines <code>load_xlsx/save_xlsx</code> and additional workbook result helpers that are not public-header declarations.
DC bus/branch struct fields	<code>DCBus</code> and <code>DCBranch</code> have more C++ struct fields than the current JSON writer exports; see Section 02b_io_field_dictionaries.

AC-network and demand JSON objects

Model	JSON path	Required keys on import	Other exported keys
ACBus	<code>ac.buses[]</code>	index	<code>bus_type</code> , <code>pd_mw</code> , <code>qd_mvar</code> , <code>vm_pu</code> , <code>va_deg</code> , <code>area</code> , <code>base_kv</code> , <code>vmax_pu</code> , <code>vmin_pu</code> , <code>gs_mw</code> , <code>bs_mvar</code> , <code>zone</code> , <code>in_service</code> , <code>name</code> , <code>n_customers</code> , <code>importance</code> , <code>latitude</code> , <code>longitude</code>
ACBranch	<code>ac.branches[]</code>	index, <code>from_bus</code> , <code>to_bus</code>	<code>r_pu</code> , <code>x_pu</code> , <code>b_pu</code> , <code>tap</code> , <code>shift_deg</code> , <code>rate_a_mva</code> , <code>rate_b_mva</code> , <code>rate_c_mva</code> , <code>in_service</code> , <code>name</code> , <code>length_km</code> , <code>failure_rate</code> , <code>mttr_hr</code>
Load	<code>ac.loads[]</code>	index, bus	<code>in_service</code> , <code>name</code> , <code>p_mw</code> , <code>q_mvar</code> , <code>model</code> , <code>z_percent_p</code> , <code>i_percent_p</code> , <code>p_percent_p</code> , <code>z_percent_q</code> , <code>i_percent_q</code> , <code>p_percent_q</code> , <code>controllable</code> , <code>p_min_mw</code> , <code>cost_mw</code> , <code>priority</code> , <code>n_customers</code> , <code>profile_id</code>
FlexibleLoad	<code>ac.flexible_loads[]</code>	index, bus	<code>in_service</code> , <code>name</code> , <code>p_mw</code> , <code>q_mvar</code> , <code>flex_up_mw</code> , <code>flex_down_mw</code> , <code>flex_duration_h</code> , <code>response_time_s</code> , <code>ramp_rate_mw_min</code> , <code>availability_pct</code> , <code>controllable</code> , <code>priority</code> , <code>control_area</code>
AsymmetricLoad	<code>ac.asymmetric_loads[]</code>	index, bus	<code>in_service</code> , <code>name</code> , <code>connection</code> , <code>grounded</code> , <code>paRated_mw</code> , <code>qaRated_mvar</code> , <code>pbRated_mw</code> , <code>qbRated_mvar</code> , <code>pcRated_mw</code> , <code>qcRated_mvar</code> , <code>pa_mw</code> , <code>qa_mvar</code> , <code>pb_mw</code> , <code>qb_mvar</code> , <code>pc_mw</code> , <code>qc_mvar</code> , <code>scaling</code> , <code>const_z_percent</code> , <code>const_i_percent</code> , <code>const_p_percent</code> , <code>controllable</code> , <code>priority</code>
Shunt	<code>ac.shunts[]</code>	index, bus	<code>in_service</code> , <code>name</code> , <code>gs_mw</code> , <code>bs_mvar</code> , <code>switchable</code> , <code>n_steps</code> , <code>current_step</code> , <code>bs_per_step</code>
Switch	<code>ac.switches[]</code>	index, <code>bus_from</code> , <code>bus_to</code>	<code>name</code> , <code>in_service</code> , <code>switch_type</code> , <code>closed</code> , <code>r_contact_ohm</code> , <code>z_ohm</code> , <code>iRated_ka</code> , <code>iBreaking_ka</code> , <code>element_type</code> , <code>element_id</code> , <code>is_remote</code> , <code>is_automated</code> , <code>t_operation_s</code> , <code>p_sw_fail</code> , <code>mtbf_hours</code> , <code>mttr_hours</code>
CircuitBreaker	<code>ac.circuit_breakers[]</code>	index, <code>bus_from</code> , <code>bus_to</code>	<code>name</code> , <code>in_service</code> , <code>breaker_type</code> , <code>closed</code> , <code>z_ohm</code> , <code>rated_voltage_kv</code> , <code>iRated_ka</code> , <code>iBreaking_ka</code> , <code>element_type</code> , <code>element_id</code>

Model	JSON path	Required keys on import	Other exported keys
ChargingStation	ac.charging_stations[]	index, bus	name, location, in_service, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, utilization_rate, p_total_kw, q_total_kvar, mtbf_hours, mttr_hours
Charger	ac.chargers[]	index	name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mttr_hours
Asynchronous Motor	ac.motors[]	index, bus	name, in_service, machine ratings, direct / zero-sequence parameters, locked-rotor data, and efficiency metadata
ExternalGrid	ac.external_grids[]	index, bus	name, in_service, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, r_pu, x_pu, r0_pu, x0_pu, vn_kv, controllable
Transformer2W	ac.transformers_2w[]	index, hv_bus, lv_bus	name, std_type, in_service, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vkr_percent, pk_kw, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, z0_percent, x0_r0, mtbf_hours, mttr_hours
Transformer3W	ac.transformers_3w[]	index, hv_bus, mv_bus, lv_bus	name, std_type, in_service, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mttr_hours

Generation, storage, and hybrid JSON objects

Model	JSON path	Required keys on import	Other exported keys
Generator	ac.generators[]	index, bus	in_service, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, cost_c2, cost_c1, cost_c0, name, fuel_type, startup_cost, shutdown_cost, min_up_time_hr, min_dn_time_hr, ramp_up_mw_min, ramp_dn_mw_min, mbase_mva, inertia_h, droop_r, xd_pu, xdp_pu, xdpp_pu, td0p_s, td0pp_s, emission_factor_tco2_mwh, nox_factor_kg_mwh, so2_factor_kg_mwh, forced_outage_rate, mtrr_hr
StaticGenerator	ac.static_generators[] or dc.static_generators[]	index, bus	in_service, name, sgen_type, p_mw, q_mvar, p_rated_mw, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, scaling, controllable, v_ref_pu, k_p, k_q, k, rx, co2_emission_rate, mtbf_hours, mtrr_hours
RenewableGen	ac.renewable_generators[]	index, bus	in_service, name, type, p_mw, q_mvar, p_rated_mw, qmax_mvar, qmin_mvar, curtailable, cost_curtail_mwh, capacity_factor, profile_id, emission_offset_tco2_mwh
PVSystem	ac.pv_systems[]	index, bus	in_service, name, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, controllable, v_ac_set_pu, v_dc_set_pu, inverter_eff, loss_percent, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, irradiance, temperature, mtbf_panel_hours, mtrr_panel_hours, mtbf_inverter_hours, mtrr_inverter_hours
Storage	ac.storage[] or dc.storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, soc_carbon_intensity_tco2_mwh, eta_charge, eta_discharge, self_discharge_pct, max_cycles, current_cycles, soh, replacement_cost, forced_outage_rate, mtrr_hr
DCBus	dc.buses[]	index	bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service, name, latitude, longitude
DCBranch	dc.branches[]	index, from_bus, to_bus	r_pu, rate_a_mva, in_service, name, length_km
VSCConverter	vsc_converters[]	index, bus_ac, bus_dc	in_service, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, loss_mw, k_vdc, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, name, forced_outage_rate, mtrr_hr
StaticGeneratorDC	dc.dc_static_generators[]	index, bus	in_service, name, type, p_set_mw, scaling, profile_id, pmax_mw, pmin_mw, controllable, mtbf_hours, mtrr_hours, t_scheduled_hr

Model	JSON path	Required keys on import	Other exported keys
PVArrayDC	dc.pv_arrays[]	index, bus	in_service, name, p_set_mw, profile_id, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, temperature, irradiance, mtbf_hours, mttr_hours, t_scheduled_hr
DCCircuitBreaker	dc.dc_circuit_breakers[]	index, bus_from, bus_to	name, in_service, breaker_type, closed, r_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id
DCDCConverter	dc.dcdc_converters[]	index, bus_in, bus_out	in_service, name, control_mode, p_ref_mw, v_ref_pu, sn_mva, vn_in_kv, vn_out_kv, eta, r_eq_pu, pmax_mw, pmin_mw, k_droop, mtbf_hours, mttr_hours
EnergyRouterPort	energy_routers[].ports[]	index, bus	name, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service
EnergyRouter	energy_routers[]	index	name, in_service, router_type, num_ports, ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mttr_hours
MobileStorage	mobile_storage[]	index, bus	in_service, name, p_mw, q_mvar, p_rated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, e_rated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, max_cycles, current_cycles, soh, is_mobile, status, current_location, target_bus, e_consumption_mwh_km, max_travel_distance_km, mtbf_hours, mttr_hours
VirtualPowerPlant	vpps[]	index	name, description, pcc_bus, resource-count fields, aggregate capability / output terms, ramp limits, reliability metadata, and in_service; import also accepts legacy aggregation_bus
Microgrid	microgrids[]	index	name, description, in_service, pcc_bus, internal_buses, operating_mode, islanding_capability, auto_reconnection, p_exchange_max_mw, p_exchange_min_mw, p_import_max_mw, p_export_max_mw, p_exchange_mw, total_generation_mw, total_storage_mwh, total_load_mw, capacity_mw, peak_load_mw, f_set_hz, v_set_pu, k_droop, area, mtbf_hours, mttr_hours
ThreePhaseACSystem	three_phase_ac	none	name, base_mva, base_freq_hz, and component arrays for buses, lines, transformers, loads, generators, and external grids

Model	JSON path	Required keys on import	Other exported keys
ThreePhaseAC Bus	three_phase_ac.buses[]	index	bus_type, name, base_kv, in_service, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone
ThreePhaseAC Line	three_phase_ac.lines[]	index, from_bus, to_bus	name, in_service, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mttr_hr
ThreePhase Transformer	three_phase_ac.transformers[]	index, hv_bus, lv_bus	name, in_service, nameplate data, leakage and magnetizing terms, zero-sequence data, tap settings, vector group, and reliability metadata
ThreePhaseLoad	three_phase_ac.loads[]	index, bus	name, in_service, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio
ThreePhase Generator	three_phase_ac.generators[]	index, bus	name, in_service, is_slack, dispatch setpoints / limits, voltage target, machine base, and sequence reactance terms
ThreePhase ExternalGrid	three_phase_ac.external_grids[]	index, bus	name, in_service, voltage target, short-circuit level bounds, R/X bounds, and sequence Thevenin terms

Result JSON objects

JSON artifact	Top-level keys	Current note
PowerFlowResult JSON	converged, iterations, residual, ac_buses, dc_buses, ac_branch_flows, dc_branch_flows, vsc_transfers, dcdc_transfers	export-only helper writes solved bus states, AC/DC branch flow and loading fields, and converter transfer reports.

JSON artifact	Top-level keys	Current note
ACOPFResult JSON	converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status, solver_path, vm, va, vdc, pg_mw, qg_mvar, dpd_mw, dqd_mvar, pac_mw, qac_mvar, enhanced dispatch objects, legacy dispatch arrays, profiling	enhanced dispatch objects are renewable_dispatch, storage_dispatch, dcdc_dispatch, and flex_dispatch; each contains values plus a map of original_index/source_type.
CarbonAnalysisResult JSON	tracing_verified, matrix_solved, matrix_residual, load_carbon, dc_load_carbon, branch_carbon, dc_branch_carbon, bus_carbon, dc_bus_carbon, vsc_carbon, dcdc_carbon, storage_carbon, energy_router_carbon, generator_loss_allocation, tracing_summary, matrix_summary	AC/DC load arrays carry the same scalar fields plus nested generator_supply_mw; branch / converter / router arrays expose loss and direction fields; storage_carbon adds charge/discharge-side storage semantics including traced charging intensity.

9.4 Excel Input Workbook Contract

Input-sheet catalog

Sheet	Primary model / load target	Columns	Current note
Info	global metadata	Key, Value	loader recognizes name, base_mva, ac_freq_hz, three_phase_name, three_phase_base_mva, and three_phase_base_freq_hz.
ACBus	ACBus	index, name, bus_type, pd_mw, qd_mvar, vm_pu, va_deg, base_kv, vmin_pu, vmax_pu, gs_mw, bs_mvar, area, zone, in_service, n_customers, importance, latitude, longitude	write path uses aggregated AC demand, not necessarily the raw bus fields.
ACBranch	ACBranch	index, name, from_bus, to_bus, r_pu, x_pu, b_pu, tap, shift_deg, rate_a_mva, rate_b_mva, rate_c_mva, length_km, in_service, failure_rate, mttr_hr	near-1:1 with JSON coverage.
Generator	Generator	index, name, bus, pg_mw, qg_mvar, vg_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, is_slack, fuel_type, mbase_mva, cost_c2, cost_c1, cost_c0, in_service, inertia_h, droop_r, ramp_up_mw_min, ramp_dn_mw_min	Excel omits the deeper machine, emissions, and commitment fields that JSON keeps.
Load	Load	index, name, bus, p_mw, q_mvar, model, z_percent_p, i_percent_p, p_percent_p, z_percent_q, i_percent_q, p_percent_q, controllable, priority, cost_mw, in_service	raw explicit AC load table.

Sheet	Primary model / load target	Columns	Current note
ACLoad	fallback Load import; aggregated export view	same columns as Load	written as a derived table that aggregates Load, charging stations, charging storage, VPP imports, microgrid imports, and AC router ports.
FlexibleLoad	FlexibleLoad	index, name, bus, p_mw, q_mvar, flex_up_mw, flex_down_mw, flex_duration_h, response_time_s, ramp_rate_mw_min, availability_pct, controllable, priority, control_area, in_service	dedicated Excel input sheet now exists; these rows are converted to canonical Load equivalents before PF and also have parity OPF variables.
AsymmetricLoad	AsymmetricLoad	index, name, bus, connection, grounded, paRated_mw, qaRated_mvar, pbRated_mw, qbRated_mvar, pcRated_mw, qcRated_mvar, pa_mw, qa_mvar, pb_mw, qb_mvar, pc_mw, qc_mvar, scaling, const_z_percent, const_i_percent, const_p_percent, controllable, priority, in_service	dedicated Excel input sheet now exists; solver-side use still relies on projection to a single Load equivalent.
StaticGenerator	AC StaticGenerator	index, name, bus, sgen_type, p_mw, q_mvar, pRated_mw, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, scaling, controllable, in_service	Excel omits control, SC, emissions, and reliability metadata.
Storage	AC Storage	index, name, bus, p_mw, q_mvar, pRated_mw, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, eRated_mwh, soc_init, soc_min, soc_max, eta_charge, eta_discharge, in_service	Excel omits life-cycle and outage fields.
RenewableGen	RenewableGen	index, name, bus, type, p_mw, q_mvar, pRated_mw, qmax_mvar, qmin_mvar, curtailable, capacity_factor, in_service	Excel omits cost_curtail_mwh, profile_id, and emission-offset metadata.
PVSystem	PVSystem	index, name, bus, p_mw, q_mvar, sn_mva, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, inverter_eff, in_service	Excel keeps only a narrow operational subset; JSON is much richer.
Shunt	Shunt	index, name, bus, gs_mw, bs_mvar, switchable, n_steps, current_step, bs_per_step, in_service	near-1:1 with JSON coverage.
ExternalGrid	ExternalGrid	index, name, bus, vm_pu, va_deg, s_sc_max_mva, s_sc_min_mva, rx_max, rx_min, controllable, in_service	Excel omits explicit Thevenin impedance fields present in JSON.
Transformer2W	Transformer2W	index, name, hv_bus, lv_bus, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, vkr_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_min, tap_max, tap_neutral, tap_step_percent, shift_deg, vector_group, in_service	Excel omits std_type, pk_kw, zero-sequence, and reliability fields.

Sheet	Primary model / load target	Columns	Current note
Transformer3W	Transformer3W	index, name, std_type, hv_bus, mv_bus, lv_bus, sn_hv_mva, sn_mv_mva, sn_lv_mva, vn_hv_kv, vn_mv_kv, vn_lv_kv, vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent, vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent, pfe_kw, i0_percent, tap_side, tap_pos, tap_step_percent, shift_mv_deg, shift_lv_deg, mtbf_hours, mtr_hours, in_service	dedicated Excel input sheet now exists; solver-side treatment still projects the device into equivalent branches.
ChargingStation	ChargingStation	index, name, bus, n_fast, n_slow, num_chargers, p_fast_max_kw, p_slow_max_kw, max_power_kw, simultaneity_factor, power_factor, p_total_kw, q_total_kvar, in_service	Excel omits location, utilization_rate, and reliability fields.
Charger	Charger	index, name, station_id, charger_type, in_service, p_rated_kw, p_ch_max_kw, p_ch_min_kw, eta, v2g_capable, p_dis_max_kw, mtbf_hours, mtr_hours	dedicated Excel input sheet now exists; charger rows are linked to their parent station by station_id.
Asynchronous Motor	Asynchronous Motor	index, name, bus, machine ratings, direct / zero-sequence parameters, SC motor data, and in_service	dedicated Excel input sheet now exists for short-circuit motor data.
Switch	Switch	index, name, bus_from, bus_to, switch_type, closed, in_service	Excel keeps only the topology/status subset.
CircuitBreaker	CircuitBreaker	index, name, bus_from, bus_to, breaker_type, closed, z_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id, in_service	dedicated AC breaker sheet.
DCBus	DCBus	index, name, bus_type, vm_pu, vmax_pu, vmin_pu, pd_mw, in_service	write path uses aggregated DC demand.
DCLoad	fallback DCLoad import; aggregated export view	index, name, bus, p_mw, controllable, p_min_mw, cost_mw, in_service	written as derived aggregated DC demand.
DCLoadRaw	raw DCLoad table	same columns as DCLoad	preserves the unaggregated DC load objects and is preferred on import.
DCBranch	DCBranch	index, name, from_bus, to_bus, r_pu, rate_a_mva, length_km, in_service	near-1:1 with JSON coverage.
DCStorage	DC Storage	same columns as Storage	same coverage limits as AC storage.
DCStaticGen	DC StaticGenerator	same columns as StaticGenerator	same coverage limits as AC static generators.
DCStaticGenerator	StaticGeneratorDC	index, name, bus, type, p_set_mw, scaling, profile_id, pmax_mw, pmin_mw, controllable, mtbf_hr, mtr_hr, t_scheduled_hr, in_service	preferred explicit typed DC source sheet; importer also accepts legacy DCStaticGeneratorDC.
DCPVArray	PVArrayDC	index, name, bus, p_set_mw, profile_id, num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc, temperature, irradiance, mtbf_hr, mtr_hr, t_scheduled_hr, in_service	typed DC PV array sheet.

Sheet	Primary model / load target	Columns	Current note
DCCircuitBreaker	DCCircuitBreaker	index, name, bus_from, bus_to, breaker_type, closed, r_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id, in_service	dedicated DC breaker sheet.
VSCConverter	VSCConverter	index, name, bus_ac, bus_dc, control_mode, p_set_mw, q_set_mvar, v_dc_set_pu, v_ac_set_pu, eta, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, p_rated_mw, in_service	Excel omits loss_mw, k_vdc, and outage metadata.
DCDCConverter	DCDCConverter	index, name, bus_in, bus_out, control_mode, p_ref_mw, v_ref_pu, sn_mva, eta, pmax_mw, pmin_mw, in_service	Excel omits voltage-rating, droop, resistance, and reliability fields.
EnergyRouter	EnergyRouter	index, name, router_type, num_ports, p_rated_mw, vn_ac_kv, vn_dc_kv, loss_percent, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, mtbf_hours, mtrr_hours, in_service	router-level metadata now has a dedicated Excel sheet.
EnergyRouterPort	EnergyRouterPort	router_index, index, name, bus, port_type, voltage_level_kv, p_mw, q_mvar, v_pu, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, control_mode, p_set_mw, q_set_mvar, v_set_pu, in_service	port rows are matched back to their parent router through router_index.
VPP	VirtualPowerPlant	index, name, pcc_bus, aggregate output / limit snapshot, selected resource counts, and in_service	Excel writes pcc_bus from aggregation_bus; it omits description, EV/load counts, aggregate capability, ramp rates, and reliability.
Microgrid	Microgrid	index, name, pcc_bus, operating_mode, islanding_capability, p_exchange_mw, p_exchange_max_mw, total_generation_mw, total_load_mw, in_service	Excel omits internal_buses and most planning/control metadata.
MobileStorage	MobileStorage	index, name, bus, status, p_mw, q_mvar, p_rated_mw, e_rated_mwh, soc_init, in_service	Excel keeps only a snapshot subset and omits most mobility/limit metadata.
ThreePhaseBus	ThreePhaseACBus	index, name, bus_type, base_kv, vm_a_pu, va_a_deg, vm_b_pu, va_b_deg, vm_c_pu, va_c_deg, pd_a_mw, qd_a_mvar, pd_b_mw, qd_b_mvar, pd_c_mw, qd_c_mvar, gs_a_mw, bs_a_mvar, gs_b_mw, bs_b_mvar, gs_c_mw, bs_c_mvar, vmin_pu, vmax_pu, area, zone, in_service	explicit unbalanced bus table for the optional three_phase_ac subsystem.
ThreePhaseLine	ThreePhaseACLine	index, name, from_bus, to_bus, length_km, parallel, r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km, r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km, r1_pu, x1_pu, b1_pu, r0_pu, x0_pu, b0_pu, max_i_ka, rate_a_mva, failure_rate, mtrr_hr, in_service	sequence-parameter line sheet for explicit unbalanced input.

Sheet	Primary model / load target	Columns	Current note
ThreePhaseTransformer	ThreePhaseTransformer	index, name, buses, nameplate data, leakage / zero-sequence terms, tap settings, reliability metadata, and in_service	explicit three-phase transformer input sheet; solver path still projects it to Transformer2W.
ThreePhaseLoad	ThreePhaseLoad	index, name, bus, connection, grounded, p_a_mw, q_a_mvar, p_b_mw, q_b_mvar, p_c_mw, q_c_mvar, const_z_percent, const_i_percent, const_p_percent, motor_percent, lrc_pu, x_r_ratio, in_service	explicit unbalanced demand table; projected before main PF/OPF assembly.
ThreePhaseGenerator	ThreePhaseGenerator	index, name, bus, slack flag, dispatch setpoints / limits, sequence reactance terms, and in_service	explicit unbalanced generator sheet with sequence impedance metadata.
ThreePhaseExternalGrid	ThreePhaseExternalGrid	index, name, bus, voltage target, SC level bounds, sequence impedance terms, and in_service	explicit three-phase grid-equivalent sheet.

Remaining field-level Excel caveats

Area	Current Excel status
Load short-circuit motor subfields	the embedded SC fields sn_mva, motor_percent, x_sub_pu, and r_sc_pu still do not have dedicated Load/ACLoad workbook columns.
ThreePhaseACSystem metadata container	subsystem metadata is carried in Info keys rather than a dedicated ThreePhaseInfo sheet.

JSON Key / Excel Column Dictionary for the Newly Added Contracts

The next tables are intentionally 1:1 with the current serializer code. The “JSON default” text below means the value used by `from_json` when a key is absent. Excel uses the exact sheet and column names shown here.

Optional three-phase subsystem metadata: three_phase_ac and Info

Field	Excel sheet / column	JSON default / import rule	Meaning
name	Info[three_phase_name]	default "Three-Phase ACSystem"	user-visible subsystem name for the optional explicit unbalanced AC network
base_mva	Info[three_phase_base_mva]	default 100.0	per-subsystem power base; independent from top-level base_mva
extttCircuitBreaker	ac.circuit_breakers[]	index, bus_from, bus_to	name, in_service, breaker_type, closed, z_ohm, rated_voltage_kv, i_rated_ka, i_breaking_ka, element_type, element_id
base_freq_hz	Info[three_phase_base_freq_hz]	default 50.0	nominal subsystem frequency in Hz

Field	Excel sheet / column	JSON default / import rule	Meaning
buses[]	sheet ThreePhaseBus	missing array / sheet keeps vector empty	explicit three-phase bus family
lines[]	sheet ThreePhaseLine	missing array / sheet keeps vector empty	explicit three-phase line family
transformers[]	sheet ThreePhaseTransformer	missing array / sheet keeps vector empty	explicit three-phase transformer family
loads[]	sheet ThreePhaseLoad	missing array / sheet keeps vector empty	explicit unbalanced load family
generators[]	sheet ThreePhaseGenerator	missing array / sheet keeps vector empty	explicit three-phase generator family
external_grids[]	sheet ThreePhaseExternalGrid	missing array / sheet keeps vector empty	explicit sequence-based grid-equivalent family

Asynchronous motor contract: `ac.motors[]` and sheet **AsynchronousMotor**

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable device identifier
bus	bus	required on JSON import	connected AC bus index
name	name	default empty string	display label only
in_service	in_service	default true	availability flag; not a solved PF state
vn_kv	vn_kv	default 0.0	rated terminal voltage in kV
sn_mva	sn_mva	default 0.0	rated apparent-power metadata for motor fault-contribution models
r_pu	r_pu	default 0.0	motor equivalent positive-sequence resistance input
x_pu	x_pu	default 0.0	motor equivalent positive-sequence reactance input
x_r	x_r	default 0.0	X/R ratio metadata for short-circuit-style motor models
lrc	lrc	default 0.0	locked-rotor current ratio
poles	poles	default 2	machine pole count
cos_phi	cos_phi	default 0.85	rated power factor
efficiency	efficiency	default 0.95	rated efficiency factor
r0_pu	r0_pu	default 0.0	zero-sequence resistance input for unsymmetrical faults
x0_pu	x0_pu	default 0.0	zero-sequence reactance input for unsymmetrical faults

Three-phase bus contract: `three_phase_ac.buses[]` and sheet **ThreePhaseBus**

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable bus identifier
name	name	default empty string	bus label
bus_type	bus_type	default PQ	bus category using the same string tokens as the AC bus model
base_kv	base_kv	default 0.0	nominal line-to-line bus voltage base
in_service	in_service	default true	availability flag
vm_a_pu, vm_b_pu, vm_c_pu	same columns	default 1.0 each	per-phase initial or specified voltage magnitudes
va_a_deg, va_b_deg, va_c_deg	same columns	defaults 0.0, -120.0, 120.0	phase-angle initialization in degrees for phase order a, b, c
pd_a_mw, pd_b_mw, pd_c_mw	same columns	default 0.0	explicit per-phase active demand before projection
qd_a_mvar, qd_b_mvar, qd_c_mvar	same columns	default 0.0	explicit per-phase reactive demand before projection
gs_a_mw, gs_b_mw, gs_c_mw	same columns	default 0.0	per-phase conductance shunt terms
bs_a_mvar, bs_b_mvar, bs_c_mvar	same columns	default 0.0	per-phase susceptance shunt terms
vmin_pu	vmin_pu	default 0.9	lower per-unit voltage bound
vmax_pu	vmax_pu	default 1.1	upper per-unit voltage bound
area	area	default 1	planning / reporting area tag
zone	zone	default 1	planning / reporting zone tag

Three-phase line contract: `three_phase_ac.lines[]` and sheet `ThreePhaseLine`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel index stops table import	stable line identifier
from_bus, to_bus	same columns	required on JSON import	line terminal bus indices
name	name	default empty string	line label
in_service	in_service	default true	availability flag
length_km	length_km	default 0.0	physical route length
parallel	parallel	default 1	number of parallel circuits represented by the row
r1_ohm_per_km, x1_ohm_per_km, c1_nf_per_km	same columns	default 0.0	positive-sequence per-km impedance / capacitance data
r0_ohm_per_km, x0_ohm_per_km, c0_nf_per_km	same columns	default 0.0	zero-sequence per-km impedance / capacitance data
r1_pu, x1_pu, b1_pu	same columns	default 0.0	positive-sequence per-unit branch parameters

Field	Excel column	JSON default / import rule	Meaning
r0_pu, x0_pu, b0_pu	same columns	default 0.0	zero-sequence per-unit branch parameters
max_i_ka	max_i_ka	default 0.0	thermal current limit in kA
rate_a_mva	rate_a_mva	default 0.0	apparent-power rating used by planning / screening logic
failure_rate	failure_rate	default 0.0	reliability failure-rate input
mttr_hr	mttr_hr	default 0.0	repair-time input in hours

Three-phase transformer contract: `three_phase_ac.transformers[]` and sheet `ThreePhaseTransformer`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable transformer identifier
hv_bus, lv_bus	same columns	required on JSON import	high-voltage and low-voltage terminal bus indices
name	name	default empty string	transformer label
in_service	in_service	default <code>true</code>	availability flag
sn_mva	sn_mva	default 0.0	rated apparent power
vn_hv_kv, vn_lv_kv	same columns	default 0.0	rated winding voltages
vk_percent, vkr_percent	same columns	default 0.0	short-circuit magnitude and resistive share in percent
pfe_kw, i0_percent	same columns	default 0.0	no-load loss and excitation-current inputs
vector_group	vector_group	default empty string	winding connection / clock notation for zero-sequence path interpretation
vk0_percent, vkr0_percent, mag0_percent, mag0_rx, si0_hv_partial	same columns	defaults 0.0 except <code>si0_hv_partial=0.5</code>	zero-sequence transformer data preserved for SC and future unbalanced workflows
tap_side, tap_pos, tap_min, tap_max, tap_neutral	same columns	default 0	tap-changer placement and discrete tap position meta-data
tap_step_percent	tap_step_percent	default 0.0	tap step size in percent
shift_deg	shift_deg	default 0.0	phase shift angle in degrees
mtbf_hours, mttr_hours	same columns	default 0.0	reliability inputs retained in the explicit contract

Three-phase load contract: `three_phase_ac.loads[]` and sheet `ThreePhaseLoad`

Field	Excel column	JSON default / import rule	Meaning
index	index	required on JSON import; blank Excel <code>index</code> stops table import	stable load identifier
bus	bus	required on JSON import	connected three-phase bus index
name	name	default empty string	load label
in_service	in_service	default <code>true</code>	availability flag

Field	Excel column	JSON default / import rule	Meaning
<code>connection</code>	<code>connection</code>	default "wye"	wiring model; current code expects free-form connection text such as wye or delta
<code>grounded</code>	<code>grounded</code>	default true	grounding-path assumption for the connection
<code>p_a_mw</code> , <code>p_b_mw</code> , <code>p_c_mw</code>	same columns	default 0.0	per-phase active demand
<code>q_a_mvar</code> , <code>q_b_mvar</code> , <code>q_c_mvar</code>	same columns	default 0.0	per-phase reactive demand
<code>const_z_percent</code> , <code>const_i_percent</code>	same columns	default 0.0	ZIP-model constant-impedance and constant-current shares
<code>const_p_percent</code>	<code>const_p_percent</code>	default 100.0	ZIP-model constant-power share
<code>motor_percent</code>	<code>motor_percent</code>	default 0.0	embedded motor-fraction share inside the unbalanced load object
<code>lrc_pu</code>	<code>lrc_pu</code>	default 0.0	locked-rotor current ratio for the embedded motor fraction
<code>x_r_ratio</code>	<code>x_r_ratio</code>	default 0.0	X/R ratio for the embedded motor fraction

Three-phase generator contract: `three_phase_ac.generators[]` and sheet `ThreePhaseGenerator`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel index stops table import	stable generator identifier
<code>bus</code>	<code>bus</code>	required on JSON import	connected three-phase bus index
<code>name</code>	<code>name</code>	default empty string	generator label
<code>in_service</code>	<code>in_service</code>	default true	availability flag
<code>is_slack</code>	<code>is_slack</code>	default false	slack-machine indicator for explicit three-phase studies
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	current active / reactive injection setpoints
<code>vm_pu</code>	<code>vm_pu</code>	default 1.0	terminal voltage target or initialization
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	capability limits
<code>mbase_mva</code>	<code>mbase_mva</code>	default 0.0	machine MVA base for the sequence reactances
<code>xd_pu</code> , <code>xdpp_pu</code> , <code>x2_pu</code> , <code>x0_pu</code> , <code>r0_pu</code>	same columns	default 0.0	positive / subtransient / negative / zero-sequence machine impedance set

Three-phase external-grid contract: `three_phase_ac.external_grids[]` and sheet `ThreePhaseExternalGrid`

Field	Excel column	JSON default / import rule	Meaning
<code>index</code>	<code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable external-grid identifier
<code>bus</code>	<code>bus</code>	required on JSON import	connected three-phase bus index
<code>name</code>	<code>name</code>	default empty string	source label
<code>in_service</code>	<code>in_service</code>	default <code>true</code>	availability flag
<code>vm_pu</code>	<code>vm_pu</code>	default 1.0	Thevenin voltage magnitude target
<code>va_deg</code>	<code>va_deg</code>	default 0.0	Thevenin voltage angle in degrees
<code>s_sc_max_mva</code> , <code>s_sc_min_mva</code>	same columns	default 0.0	short-circuit strength limits
<code>rx_max</code> , <code>rx_min</code>	same columns	default 0.0	R/X ratio bounds used in equivalent-source parametrization
<code>r1_pu</code> , <code>x1_pu</code>	same columns	default 0.0	positive-sequence Thevenin impedance
<code>r2_pu</code> , <code>x2_pu</code>	same columns	default 0.0	negative-sequence Thevenin impedance
<code>r0_pu</code> , <code>x0_pu</code>	same columns	default 0.0	zero-sequence Thevenin impedance

9.5 Compact Power-Flow Workbook Additions

Sheet updated by <code>save_results_compact.xlsx</code>	Appended columns	Current note
ACBus	<code>result_vm_pu</code> , <code>result_va_deg</code>	solved AC bus state.
DCBus	<code>result_vdc_pu</code>	solved DC voltage.
ACBranch	<code>result_pf_mw</code> , <code>result_qf_mvar</code> , <code>result_pt_mw</code> , <code>result_qt_mvar</code>	branch terminal powers.
DCBranch	<code>result_v_from_pu</code> , <code>result_v_to_pu</code> , <code>result_i_pu</code> , <code>result_p_from_mw</code> , <code>result_p_to_mw</code> , <code>result_loss_mw</code>	derived from solved DC bus voltages and branch resistance.
Storage	<code>result_vm_pu</code> , <code>result_va_deg</code>	bus-state annotation only; storage dispatch is not recomputed.
DCStorage	<code>result_vdc_pu</code>	bus-state annotation only.
Load	<code>result_vm_pu</code> , <code>result_va_deg</code>	bus-state annotation on the raw AC load table.
ACLoad	<code>result_vm_pu</code> , <code>result_va_deg</code>	bus-state annotation on the aggregated AC load view.

Sheet updated by save_results_c compact_xlsx	Appended columns	Current note
DCLoad	result_vdc_pu	bus-state annotation on the aggregated DC load view.
ChargingStation	result_vm_pu, result_va_deg	station bus state.
VSCConverter	result_p_ac_mw, result_q_ac_mvar, result_p_dc_mw, result_loss_mw, result_v_ac_pu, result_v_dc_pu	solved converter transfer and terminal voltages.
DCDCConverter	result_p_in_mw, result_p_out_mw, result_loss_mw, result_v_in_pu, result_v_out_pu	solved converter transfer and terminal voltages.
VPP	result_vm_pu, result_va_deg	aggregation-bus annotation only.
Microgrid	result_vm_pu, result_va_deg	PCC-bus annotation only.
MobileStorage	result_vm_pu, result_va_deg	current bus annotation only.
Results_Summary	Key, Value	key-value sheet with converged, iterations, and residual.

9.6 Dedicated Result Workbooks

Dedicated power-flow result sheets

Sheet	Columns	Current note
Results_ACBus	index, name, vm_pu, va_deg	solved AC bus state.
Results_DCBus	index, name, vm_pu	this sheet uses the AC-style column name vm_pu even though the value is the DC solved voltage.
Results_Branch Flow	index, from_bus, to_bus, pf_mw, qf_mvar, pt_mw, qt_mvar	branch flows reported in branch order.
Results_DCBranch	index, name, from_bus, to_bus, in_service, v_from_pu, v_to_pu, i_pu, p_from_mw, p_to_mw, loss_mw	computed from solved voltages and branch resistance.
Results_VSCConverter	index, name, bus_ac, bus_dc, in_service, control_mode, p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw, p_set_mw, q_set_mvar, v_ac_pu, v_dc_pu, eta, loss_percent	includes both solved transfer and original setpoint metadata.

Sheet	Columns	Current note
Results_DCDCConverter	index, name, bus_in, bus_out, in_service, control_mode, p_ref_mw, v_ref_pu, v_in_pu, v_out_pu, eta, p_in_mw, p_out_mw, loss_mw	includes both solved transfer and original reference metadata.
Results_Storage_AC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vm_pu, va_deg	uses <code>soc_init</code> as the reported <code>soc</code> .
Results_Storage_DC	index, name, bus, in_service, p_mw, q_mvar, soc, soh, vdc_pu	uses <code>soc_init</code> as the reported <code>soc</code> .
Results_ChargingStation	index, name, bus, in_service, num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate, vm_pu, va_deg	station bus state plus current aggregate demand fields.
Results_Charger	index, name, station_id, station_bus, in_service, charger_type, p_rated_kw, p_ch_max_kw, p_dis_max_kw, eta, v2g_capable, vm_pu, va_deg	<code>charger_type</code> now uses the same string token family as JSON and workbook input sheets.
Results_EnergyRouter	index, name, in_service, router_type, num_ports, p_rated_mw, loss_percent, p_port_sum_mw, q_port_sum_mvar	device-level summary only; solved per-port values live in the port sheet.
Results_EnergyRouterPort	router_index, port_index, port_name, bus, port_type, control_mode, in_service, p_mw, q_mvar, v_solved_pu	flattens nested router ports into one table.
Results_VPP	index, name, aggregation_bus, in_service, p_output_mw, q_output_mvar, pmax_mw, pmin_mw, vm_pu, va_deg	aggregation-bus annotation only.
Results_Microgrid	index, name, pcc_bus, operating_mode, in_service, p_exchange_mw, total_generation_mw, total_load_mw, vm_pu, va_deg, internal_buses	<code>internal_buses</code> is written as one comma-separated string.
Results_MobileStorage	index, name, bus, status, in_service, p_mw, q_mvar, soc, target_bus, vm_pu, va_deg	uses <code>soc_init</code> as the reported <code>soc</code> .
Results_Summary	Key, Value	keys are <code>converged</code> , <code>iterations</code> , and <code>residual</code> .

Result-Sheet Enum and Flag Conventions

Artifact	Field	Output representation	Field note
Results_VSCConverter	<code>control_mode</code>	string token: PQ, VDC_Q, VDC_VAC	copied from the input converter mode; not a solver-selected mode switch

Artifact	Field	Output representation	Field note
Results_DCDCConverter	control_mode	string token: Voltage, Power, Droop	copied from the input converter mode
Results_Charger	charger_type	string token: AC_L1, AC_L2, DC_Fast	now aligned with JSON and workbook input instead of numeric enum codes
Results_EnergyRouterPort	port_type	string token: AC or DC	physical port side classification
Results_EnergyRouterPort	control_mode	string token: PQ, VF, Droop	copied from the input port control mode
Results_Microgrid	operating_mode	string token: Grid Connected, Islanded, Transition	reported operating mode metadata, not a PF state variable
Results_MobileStorage	status	string token: Stationary, InTransit, Deployed	reported mobility state metadata
Results_Charger	v2g_capable	boolean token: TRUE / FALSE	static capability flag, not current dispatch direction
Results_* device sheets	in_service	boolean token: TRUE / FALSE	copied device availability flag across result workbooks
Results_Summary, OPF_Summary	converged	boolean token: TRUE / FALSE	solver convergence outcome

Dedicated OPF and carbon result sheets

Sheet	Columns	Current note
OPF_Bus	index, name, base_kv, vm_pu, va_deg, dpd_mw, dqd_mvar	bus voltages plus bus-level load shedding vectors.
OPF_Generator	index, name, bus, pg_mw, qg_mvar, pmax_mw, pmin_mw, qmax_mvar, qmin_mvar, cost_c2, cost_c1, cost_c0, fuel_type	generator dispatch report.
OPF_Converter	index, name, bus_ac, bus_dc, pac_mw, qac_mvar	only VSC converters are reported here.
OPF_Summary	Key, Value	keys are converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, and status.
LoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 1.

Sheet	Columns	Current note
DCLoadCarbon	load_index, bus, demand_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 2.
BusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 3.
DCBusCarbon	bus_index, carbon_intensity_tco2_mwh	dedicated carbon workbook, sheet 4.
BranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 5.
DCBranchCarbon	branch_index, from_bus, to_bus, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 6.
VSCCarbon	converter_index, bus_ac, bus_dc, ac_to_dc, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 7.
DCDCCarbon	converter_index, bus_in, bus_out, input_to_output, input_power_mw, output_power_mw, loss_mw, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook, sheet 8.
StorageCarbon	storage_index, bus, is_dc, p_mw, soc, stored_energy_mwh, soc_carbon_intensity_tco2_mwh, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook sheet for explicit storage charge/discharge carbon semantics.
EnergyRouterCarbon	router_index, input_power_mw, output_power_mw, loss_mw, active_input_ports, active_output_ports, carbon_intensity_tco2_mwh, total_emissions_tco2	dedicated carbon workbook sheet for router-internal loss allocation reconstructed from port transfers.
Summary	key-value blocks for tracing and matrix summaries	dedicated carbon workbook summary sheet; contains generation/load/loss totals, balance errors, and flags such as tracing_verified and matrix_solved .

9.7 Current I/O Coverage Gaps That Matter to Engineering Work

Public model area	JSON system file	Excel input workbook	Engineering consequence
ACBus, ACBranch, DCBus, DCBranch	yes	yes	core network objects are available in both I/O families.
Generator, Load, Shunt, ExternalGrid	yes	yes	Excel coverage is narrower than JSON for the richer machine / source metadata.
StaticGenerator, RenewableGen, PVSystem, Storage	yes	yes	Excel exposes only the operational subset; JSON is the better notebook-to-code contract for detailed studies.
Transformer2W	yes	yes	Excel omits zero-sequence and reliability fields.
Transformer3W	yes	yes	JSON and Excel now both carry the explicit 3-winding transformer data, although solver use still goes through projection.
FlexibleLoad, AsymmetricLoad	yes	yes	both I/O families now preserve the rich pre-projection load models.
ChargingStation	yes	yes	input support exists in both families.
Charger	yes	yes	both I/O families now preserve per-charger metadata; the parent station link remains <code>station_id</code> .
DCLoad	yes	yes	the main system JSON and Excel workbook are now aligned for the explicit DC load table.
VSCConverter, DCDConverter	yes	yes	Excel coverage is narrower than JSON.
EnergyRouter, EnergyRouterPort	yes	yes	router studies can now use either JSON or Excel input, with ports flattened into a dedicated Excel table.
VPP, Microgrid, MobileStorage	yes	yes	Excel again keeps only a reduced operational subset.
AsynchronousMotor	yes	yes	the SC motor model now has a stable external contract in both families.
ThreePhaseACSystem family	yes	yes	the optional unbalanced subsystem can now round-trip through both JSON and Excel, although main PF/OPF/SC still consume it after projection.

9.8 Interpretation

The current I/O layer is already useful, but it is not a single canonical schema yet.

1. JSON is now the richest external representation for the full current `HybridPowerSystem` container, including `ac.motors` and optional `three_phase_ac`.
2. Excel is an engineering workbook interface with deliberate aggregation and reduced field coverage.

3. The JSON and Excel paths are now aligned for `DCLoad`, `Transformer3W`, `FlexibleLoad`, `AsymmetricLoad`, `Charger`, `EnergyRouter`, `AsynchronousMotor`, and the `ThreePhaseACSystem` family.
4. The remaining gaps are now field-level rather than family-level, especially the SC-specific motor-fraction fields embedded inside `Load` and the lack of dedicated result-workbook contracts for explicit three-phase solved states.

10 Detailed I/O Field Dictionaries

This section expands the per-field contract to the remaining public model families. Section 02a_io_contracts already gives detailed field maps for `AsynchronousMotor`, the optional `three_phase_ac` family, and the result-sheet enum conventions. The tables below complete the same JSON-key / Excel-column dictionary pattern for the other current input models.

When the “Excel sheet / column” entry says `none`, the engineering workbook does not preserve that JSON field directly. When two workbook sheet names are listed, the first is the preferred explicit input table and the second is a fallback or derived view with the same column family.

10.1 System Metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>name</code>	<code>Info[name]</code>	default "HybridAC/DC System"	top-level <code>HybridPowerSystem</code> display name
<code>base_mva</code>	<code>Info[base_mva]</code>	default 100.0	top-level system power base; workbook import also copies it into <code>ac.base_mva</code> and <code>dc.base_mva</code>
<code>ac.name</code>	<code>none</code>	default "ACSystem"	JSON-only AC subsystem label
<code>ac.base_mva</code>	<code>none</code>	default 100.0	JSON-only AC subsystem base; workbook uses the shared top-level <code>base_mva</code> instead
<code>ac.freq_hz</code>	<code>Info[ac_freq_hz]</code>	default 50.0	nominal AC frequency in Hz
<code>dc.name</code>	<code>none</code>	default "DCSystem"	JSON-only DC subsystem label
<code>dc.base_mva</code>	<code>none</code>	default 100.0	JSON-only DC subsystem base; workbook again uses the shared top-level <code>base_mva</code>
<code>three_phase_ac.*</code>	<code>Info[three_phase_*]</code> plus dedicated sheets	see Section 02a_io_contracts	optional explicit unbalanced subsystem documented in the earlier detailed dictionary

10.2 AC Network and Demand Models

`ACBus` : `ac.buses[]` / sheet `ACBus`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>ACBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable bus identifier
<code>name</code>	<code>ACBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>ACBus.bus_type</code>	default PQ	bus category token PQ/PV/SLACK/ISOLATED
<code>pd_mw, qd_mvar</code>	same columns	default 0.0	embedded bus demand snapshot; workbook export may contain synchronized aggregated demand rather than original raw struct values
<code>vm_pu, va_deg</code>	same columns	defaults 1.0, 0.0	initial or specified bus voltage magnitude and angle
<code>base_kv</code>	<code>ACBus.base_kv</code>	default 110.0	nominal bus voltage base
<code>vmin_pu, vmax_pu</code>	same columns	defaults 0.9, 1.1	voltage operating bounds
<code>gs_mw, bs_mvar</code>	same columns	default 0.0	shunt conductance and susceptance embedded at the bus
<code>area, zone</code>	same columns	defaults 1, 1	planning / reporting partitions
<code>in_service</code>	<code>ACBus.in_service</code>	default true	availability flag
<code>n_customers</code>	<code>ACBus.n_customers</code>	default 0	customer-count metadata
<code>importance</code>	<code>ACBus.importance</code>	default 1.0	bus criticality / weighting metadata
<code>latitude, longitude</code>	same columns	default 0.0	geospatial metadata only

ACBranch : `ac.branches[]` / sheet ACBranch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>ACBranch.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable branch identifier
<code>from_bus, to_bus</code>	same columns	required on JSON import	branch terminal bus indices
<code>name</code>	<code>ACBranch.name</code>	default empty string	branch label
<code>r_pu, x_pu, b_pu</code>	same columns	default 0.0	series resistance, series reactance, and total shunt charging in per unit
<code>tap</code>	<code>ACBranch.tap</code>	default 1.0	off-nominal tap ratio used in the branch model
<code>shift_deg</code>	<code>ACBranch.shift_deg</code>	default 0.0	phase-shifting angle in degrees
<code>rate_a_mva, rate_b_mva, rate_c_mva</code>	same columns	default 0.0	thermal / planning ratings
<code>length_km</code>	<code>ACBranch.length_km</code>	default 0.0	physical route length metadata
<code>in_service</code>	<code>ACBranch.in_service</code>	default true	availability flag
<code>failure_rate, mttr_hr</code>	same columns	default 0.0	reliability metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
r0_pu, x0_pu, b0_pu, vn_hv_kv, vn_lv_kv, sn_mva	none	not part of the current JSON / Excel contract	struct-level short-circuit and transformer-nameplate helpers currently stay internal to the in-memory model

Load : `ac.loads[]` / sheet **Load (fallback ACLoad)**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	Load.index or ACLoad.index	required on JSON import; blank Excel index stops table import	stable load identifier
bus	Load.bus or ACLoad.bus	required on JSON import	connected AC bus index
name	same column	default empty string	load label
in_service	same column	default true	availability flag
p_mw, q_mvar	same columns	default 0.0	active and reactive demand
model	same column	default ConstantPower	ZIP / exponential / constant-power load-model token
z_percent_p, i_percent_p, p_percent_p	same columns	defaults 0.0, 0.0, 100.0	active-power ZIP shares
z_percent_q, i_percent_q, p_percent_q	same columns	defaults 0.0, 0.0, 100.0	reactive-power ZIP shares
controllable	same column	default false	whether shedding / dispatch logic may treat the load as controllable
p_min_mw	none	default 0.0	JSON-only minimum retained active demand; workbook currently omits this field for AC loads
cost_mw	same column	default 0.0	marginal shedding or curtailment penalty
priority	same column	default Medium	service-priority token Low/Medium/High/Critical
n_customers	none	default 0	JSON-only customer-count metadata
profile_id	none	default -1	JSON-only profile linkage identifier
sn_mva, motor_percent, x_sub_pu, r_sc_pu	none	not part of the current main JSON contract	short-circuit helper fields remain outside the stable Load I/O contract

FlexibleLoad : `ac.flexible_loads[]` / sheet **FlexibleLoad**

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected AC bus
name	FlexibleLoad.name	default empty string	load label

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>in_service</code>	<code>FlexibleLoad.in_service</code>	default <code>true</code>	availability flag
<code>p_mw, q_mvar</code>	same columns	default 0.0	baseline active and reactive demand
<code>flex_up_mw, flex_down_mw</code>	same columns	default 0.0	upward / downward flexibility envelope in MW
<code>flex_duration_h</code>	same column	default 0.0	duration for which the flex envelope can be sustained
<code>response_time_s</code>	same column	default 0.0	response latency in seconds
<code>ramp_rate_mw_min</code>	same column	default 0.0	adjustment ramp-rate limit
<code>availability_pct</code>	same column	default 100.0	expected availability of the flexibility service
<code>controllable</code>	same column	default <code>true</code>	whether the flexibility service is enabled
<code>priority</code>	same column	default <code>Medium</code>	load-service priority token
<code>control_area</code>	same column	default empty string	operator / aggregation area tag

AsymmetricLoad : `ac.asymmetric_loads[]` / sheet `AsymmetricLoad`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>AsymmetricLoad.name</code>	default empty string	load label
<code>in_service</code>	<code>AsymmetricLoad.in_service</code>	default <code>true</code>	availability flag
<code>connection</code>	<code>AsymmetricLoad.connection</code>	default empty string	wiring model string such as wye or delta
<code>grounded</code>	<code>AsymmetricLoad.grounded</code>	default <code>true</code>	grounding-path assumption
<code>paRated_mw, pbRated_mw, pcRated_mw</code>	same columns	default 0.0	rated per-phase active demand
<code>qaRated_mvar, qbRated_mvar, qcRated_mvar</code>	same columns	default 0.0	rated per-phase reactive demand
<code>pa_mw, pb_mw, pc_mw</code>	same columns	default 0.0	current per-phase active demand
<code>qa_mvar, qb_mvar, qc_mvar</code>	same columns	default 0.0	current per-phase reactive demand
<code>scaling</code>	<code>AsymmetricLoad.scaling</code>	default 1.0	overall scaling factor applied to the phase values
<code>const_z_percent, const_i_percent</code>	same columns	default 0.0	ZIP constant-impedance and constant-current shares
<code>const_p_percent</code>	same column	default 100.0	ZIP constant-power share
<code>controllable</code>	same column	default <code>false</code>	controllability flag
<code>priority</code>	same column	default <code>Medium</code>	load-service priority token

Shunt : `ac.shunts[]` / sheet `Shunt`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>Shunt.name</code>	default empty string	shunt label
<code>in_service</code>	<code>Shunt.in_service</code>	default <code>true</code>	availability flag
<code>gs_mw, bs_mvar</code>	same columns	default 0.0	fixed shunt conductance and susceptance
<code>switchable</code>	<code>Shunt.switchable</code>	default <code>false</code>	whether the shunt is modeled as stepped / switchable
<code>n_steps</code>	<code>Shunt.n_steps</code>	default 1	number of available discrete shunt steps
<code>current_step</code>	<code>Shunt.current_step</code>	default 1	currently selected shunt step
<code>bs_per_step</code>	<code>Shunt.bs_per_step</code>	default 0.0	susceptance increment per step

ExternalGrid : `ac.external_grids[]` / sheet ExternalGrid

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>ExternalGrid.name</code>	default empty string	source label
<code>in_service</code>	<code>ExternalGrid.in_service</code>	default <code>true</code>	availability flag
<code>vm_pu, va_deg</code>	same columns	defaults 1.0, 0.0	voltage magnitude and angle target for the slack / Thevenin source
<code>s_sc_max_mva, s_sc_min_mva</code>	same columns	default 0.0	short-circuit strength range
<code>rx_max, rx_min</code>	same columns	default 0.0	R/X ratio bounds for equivalent-source construction
<code>r_pu, x_pu</code>	<code>none</code>	default 0.0	JSON-only explicit positive-sequence Thevenin impedance
<code>r0_pu, x0_pu</code>	<code>none</code>	default 0.0	JSON-only zero-sequence source impedance
<code>vn_kv</code>	<code>none</code>	default 0.0	JSON-only nominal source voltage rating
<code>ikq_ka, x_r</code>	<code>none</code>	not part of the current JSON / Excel contract	struct-level IEC 60909 helper fields currently stay internal and are not preserved by either import path
<code>controllable</code>	<code>ExternalGrid.controllable</code>	default <code>true</code>	whether optimization logic may vary the source dispatch

Transformer2W : `ac.transformers_2w[]` / sheet Transformer2W

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, hv_bus, lv_bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and terminal bus indices
<code>name</code>	<code>Transformer2W.name</code>	default empty string	transformer label
<code>std_type</code>	<code>none</code>	default empty string	JSON-only library / standard type name
<code>in_service</code>	<code>Transformer2W.in_service</code>	default <code>true</code>	availability flag
<code>sn_mva</code>	<code>Transformer2W.sn_mva</code>	default 0.0	rated apparent power
<code>vn_hv_kv, vn_lv_kv</code>	same columns	default 0.0	winding rated voltages
<code>vk_percent, vkr_percent</code>	same columns	default 0.0	short-circuit magnitude and resistive share in percent
<code>pk_kw</code>	<code>none</code>	default 0.0	JSON-only load-loss field retained in the richer transformer contract
<code>pfe_kw, i0_percent</code>	same columns	default 0.0	no-load loss and excitation current
<code>tap_side, tap_pos, tap_min, tap_max, tap_neutral</code>	same columns	default 0	tap-changer placement and discrete tap metadata
<code>tap_step_percent</code>	<code>Transformer2W.tap_step_percent</code>	default 0.0	tap step size in percent
<code>shift_deg</code>	<code>Transformer2W.shift_deg</code>	default 0.0	phase-shift angle in degrees
<code>vector_group</code>	<code>Transformer2W.vector_group</code>	default empty string	winding connection / clock notation
<code>z0_percent, x0_r0</code>	<code>none</code>	default 0.0	JSON-only zero-sequence impedance parameters
<code>mtbf_hours, mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

Transformer3W : `ac.transformers_3w[]` / sheet `Transformer3W`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, hv_bus, mv_bus, lv_bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and three winding bus indices
<code>name</code>	<code>Transformer3W.name</code>	default empty string	transformer label
<code>std_type</code>	<code>Transformer3W.std_type</code>	default empty string	library / standard type name
<code>in_service</code>	<code>Transformer3W.in_service</code>	default <code>true</code>	availability flag
<code>sn_hv_mva, sn_mv_mva, sn_lv_mva</code>	same columns	default 0.0	winding ratings on the HV / MV / LV sides
<code>vn_hv_kv, vn_mv_kv, vn_lv_kv</code>	same columns	default 0.0	winding rated voltages
<code>vk_hv_mv_percent, vk_hv_lv_percent, vk_mv_lv_percent</code>	same columns	default 0.0	pairwise short-circuit magnitudes
<code>vkr_hv_mv_percent, vkr_hv_lv_percent, vkr_mv_lv_percent</code>	same columns	default 0.0	pairwise resistive shares

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
pfe_kw, i0_percent	same columns	default 0.0	no-load loss and excitation-current data
tap_side, tap_pos	same columns	default 0	tap-changer side and current position
tap_step_percent	Transformer3W.tap_step_percent	default 0.0	tap step size in percent
shift_mv_deg, shift_lv_deg	same columns	default 0.0	phase shifts from HV to MV / LV windings
mtbf_hours, mttr_hours	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

Switch : ac.switches[] / sheet Switch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_from, bus_to	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	Switch.name	default empty string	switch label
in_service	Switch.in_service	default true	availability flag
switch_type	Switch.switch_type	default CircuitBreaker	switching-device token such as Recloser or Fuse
closed	Switch.closed	default true	topology status flag
r_contact_ohm, z_ohm	none	default 0.0	JSON-only detailed contact / arc impedance inputs
i_rated_ka, i_breaking_ka	none	default 0.0	JSON-only current capability ratings
element_type, element_id	none	defaults 0, 0	JSON-only foreign-key reference to the protected element
is_remote, is_automated	none	default false	JSON-only automation metadata
t_operation_s, p_sw_fail	none	default 0.0	JSON-only operation timing and failure-probability metadata
mtbf_hours, mttr_hours	none	default 0.0	JSON-only reliability metadata

CircuitBreaker : ac.circuit_breakers[] / sheet CircuitBreaker

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_from, bus_to	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	CircuitBreaker.name	default empty string	breaker label
in_service	CircuitBreaker.in_service	default true	availability flag
breaker_type	CircuitBreaker.breaker_type	default CB	breaker-type token parsed by breaker_type_from_str

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>closed</code>	<code>CircuitBreaker.closed</code>	default <code>true</code>	topology status flag
<code>z_ohm</code>	<code>CircuitBreaker.z_ohm</code>	default 0.0	AC breaker impedance
<code>rated_voltage_kv</code> , <code>i_rated_ka</code> , <code>i_breaking_ka</code>	same columns	default 0.0	voltage and current ratings
<code>element_type</code> , <code>element_id</code>	same columns	defaults empty string, 0	optional protected-element reference

ChargingStation : `ac.charging_stations[]` / sheet `ChargingStation`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>ChargingStation.name</code>	default empty string	station label
<code>location</code>	<code>none</code>	default empty string	JSON-only free-form location descriptor
<code>in_service</code>	<code>ChargingStation.in_service</code>	default <code>true</code>	availability flag
<code>n_fast</code> , <code>n_slow</code> , <code>num_chargers</code>	same columns	default 0	counts of charger assets by class and in total
<code>p_fast_max_kw</code> , <code>p_slow_max_kw</code>	same columns	default 0.0	per-charger maximum power by service class
<code>max_power_kw</code>	same column	default 0.0	aggregate station power ceiling
<code>simultaneity_factor</code>	same column	default 1.0	diversification factor used in the aggregate station model
<code>power_factor</code>	same column	default 0.95	station-level reactive-power assumption
<code>utilization_rate</code>	<code>none</code>	default 0.0	JSON-only expected utilization metadata
<code>p_total_kw</code> , <code>q_total_kvar</code>	same columns	default 0.0	aggregate present demand of the station
<code>mtbf_hours</code> , <code>mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

Charger : `ac.chargers[]` / sheet `Charger`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>Charger.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable charger identifier
<code>name</code>	<code>Charger.name</code>	default empty string	charger label
<code>station_id</code>	<code>Charger.station_id</code>	default 0	parent charging-station foreign key
<code>charger_type</code>	<code>Charger.charger_type</code>	default <code>AC_L2</code>	charger-class token <code>AC_L1/AC_L2/DC_Fast</code>
<code>in_service</code>	<code>Charger.in_service</code>	default <code>true</code>	availability flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>pRated_kw</code>	<code>Charger.pRated_kw</code>	default 0.0	rated charger power
<code>pCh_max_kw</code> , <code>pCh_min_kw</code>	same columns	default 0.0	charging power limits
<code>eta</code>	<code>Charger.eta</code>	default 0.95	charger conversion efficiency
<code>v2g_capable</code>	<code>Charger.v2g_capable</code>	default false	whether reverse power flow is allowed
<code>pDis_max_kw</code>	<code>Charger.pDis_max_kw</code>	default 0.0	maximum discharging power when V2G is enabled
<code>mtbf_hours</code> , <code>mttr_hours</code>	same columns	default 0.0	reliability metadata preserved in both JSON and workbook

10.3 Generation and Storage Models

Generator : `ac.generators[]` / sheet `Generator`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>Generator.name</code>	default empty string	generator label
<code>in_service</code>	<code>Generator.in_service</code>	default true	availability flag
<code>pg_mw</code> , <code>qg_mvar</code>	same columns	default 0.0	active and reactive generation setpoints
<code>vg_pu</code>	<code>Generator.vg_pu</code>	default 1.0	voltage setpoint on the connected bus
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>is_slack</code>	<code>Generator.is_slack</code>	default false	slack-machine flag
<code>fuel_type</code>	<code>Generator.fuel_type</code>	default Unknown	fuel-type token such as Gas, Wind, or Solar
<code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code>	same columns	default 0.0	quadratic production-cost coefficients
<code>startup_cost</code> , <code>shutdown_cost</code> , <code>min_up_time_hr</code> , <code>min_dn_time_hr</code>	none	default 0.0	JSON-only unit-commitment metadata
<code>ramp_up_mw_min</code> , <code>ramp_dn_mw_min</code>	same columns	default 0.0	ramp-rate limits
<code>mbase_mva</code> , <code>inertia_h</code> , <code>droop_r</code>	same columns	defaults 0.0, 0.0, 0.05	machine base, inertia, and governor droop metadata
<code>xd_pu</code> , <code>xdp_pu</code> , <code>xdpp_pu</code> , <code>td0p_s</code> , <code>td0pp_s</code>	none	default 0.0	JSON-only machine dynamic / short-circuit parameters
<code>ra_pu</code> , <code>vn_kv</code> , <code>cos_phi</code> , <code>xq_pu</code> , <code>xd_xq</code> , <code>x0_pu</code> , <code>r0_pu</code>	none	not part of the current JSON / Excel contract	additional short-circuit parameters exist in the C++ struct but are not yet serialized or workbook-backed
<code>emission_factor_tco2_mwh</code> , <code>nox_factor_kg_mwh</code> , <code>so2_factor_kg_mwh</code>	none	default 0.0	JSON-only emissions metadata

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>forced_outage_rate</code> , <code>mttr_hr</code>	<code>none</code>	default 0.0	JSON-only reliability meta-data

StaticGenerator : `ac.static_generators[]` or `dc.static_generators[]` / sheets StaticGenerator and DCStaticGen

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC or DC bus
<code>name</code>	corresponding <code>name</code> column	default empty string	generator label
<code>in_service</code>	corresponding <code>in_service</code> column	default <code>true</code>	availability flag
<code>sgen_type</code>	corresponding <code>sgen_type</code> column	default <code>Other</code>	generator-type token such as <code>Wind</code> , <code>Battery</code> , or <code>FuelCell</code>
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present active / reactive injection
<code>p_rated_mw</code> , <code>sn_mva</code>	same columns	default 0.0	rated active power and apparent power
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>scaling</code>	same column	default 1.0	scalar multiplier for the static generation profile
<code>controllable</code>	same column	default <code>false</code>	controllability flag
<code>v_ref_pu</code> , <code>k_p</code> , <code>k_q</code> , <code>k_rx</code>	<code>none</code>	defaults 1.0, 0.0, 0.0, 1.0, 0.0	JSON-only control and equivalent-source tuning parameters
<code>co2_emission_rate</code>	<code>none</code>	default 0.0	JSON-only emissions meta-data
<code>mtbf_hours</code> , <code>mttr_hours</code>	<code>none</code>	default 0.0	JSON-only reliability meta-data

Storage : `ac.storage[]` or `dc.storage[]` / sheets Storage and DCStorage

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected bus
<code>name</code>	corresponding <code>name</code> column	default empty string	storage label
<code>in_service</code>	corresponding <code>in_service</code> column	default <code>true</code>	availability flag
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present active and reactive dispatch
<code>p_rated_mw</code>	same column	default 0.0	rated converter power
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	operating limits

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>eRated_mwh</code>	same column	default 0.0	energy capacity
<code>soc_init</code> , <code>soc_min</code> , <code>soc_max</code>	same columns	defaults 0.5, 0.1, 0.9	initial and admissible state-of-charge values
<code>soc_carbon_intensity_tco2_mwh</code>	same column	default 0.0	explicit carbon intensity of the stored energy state used by carbon-flow analysis during discharge
<code>eta_charge</code> , <code>eta_discharge</code>	same columns	default 0.95	charging and discharging efficiency
<code>self_discharge_pct</code>	none	default 0.0	JSON-only storage leakage metadata
<code>max_cycles</code> , <code>current_cycles</code>	none	defaults 5000, 0	JSON-only lifecycle counters
<code>soh</code>	none	default 1.0	JSON-only state-of-health value
<code>replacement_cost</code>	none	default 0.0	JSON-only replacement-cost metadata
<code>forced_outage_rate</code> , <code>mtr_hr</code>	none	default 0.0	JSON-only reliability metadata

RenewableGen : `ac.renewable_gens[]` / sheet RenewableGen

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>RenewableGen.name</code>	default empty string	generator label
<code>in_service</code>	<code>RenewableGen.in_service</code>	default true	availability flag
<code>type</code>	<code>RenewableGen.type</code>	default Wind	renewable-type token such as Wind , SolarPV , or Hydro
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present active and reactive output
<code>p_rated_mw</code>	same column	default 0.0	rated active power
<code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	default 0.0	reactive capability limits
<code>curtailable</code>	same column	default true	whether curtailment is allowed
<code>cost_curtail_mwh</code>	none	default 0.0	JSON-only curtailment penalty
<code>capacity_factor</code>	<code>RenewableGen.capacity_factor</code>	default 0.3	expected normalized availability
<code>profile_id</code>	none	default -1	JSON-only profile linkage identifier
<code>emission_offset_tco2_mwh</code>	none	default 0.0	JSON-only avoided-emissions metadata

PVSystem : `ac.pv_systems[]` / sheet PVSystem

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected AC bus
<code>name</code>	<code>PVSystem.name</code>	default empty string	PV system label
<code>in_service</code>	<code>PVSystem.in_service</code>	default <code>true</code>	availability flag
<code>p_mw, q_mvar</code>	same columns	default 0.0	present active and reactive output
<code>sn_mva</code>	<code>PVSystem.sn_mva</code>	default 0.0	apparent-power rating
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default 0.0	operating capability limits
<code>control_mode</code>	<code>PVSystem.control_mode</code>	default MPPT	inverter control-mode token; one of MPPT, VoltVar, VoltWatt, FixedPF, or Curtailed
<code>controllable</code>	<code>none</code>	default <code>false</code>	JSON-only controllability flag
<code>v_ac_set_pu, v_dc_set_pu</code>	<code>none</code>	default 1.0	JSON-only AC/DC voltage reference targets
<code>inverter_eff</code>	<code>PVSystem.inverter_eff</code>	default 0.97	inverter efficiency
<code>loss_percent</code>	<code>none</code>	default 0.0	JSON-only miscellaneous loss factor
<code>num_series, num_parallel, vmpp, impp, voc, isc, alpha_isc, beta_voc</code>	<code>none</code>	default 0 or 0.0	JSON-only panel-array electrical metadata
<code>irradiance, temperature</code>	<code>none</code>	defaults 1000.0, 25.0	JSON-only environmental inputs
<code>mtbf_panel_hours, mttr_panel_hours, mtbf_inverter_hours, mttr_inverter_hours</code>	<code>none</code>	default 0.0	JSON-only reliability metadata

10.4 DC and Hybrid Coordination Models

DCBus : `dc.buses[]` / sheet DCBus

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	<code>DCBus.index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable DC bus identifier
<code>name</code>	<code>DCBus.name</code>	default empty string	bus label
<code>bus_type</code>	<code>DCBus.bus_type</code>	default DC_P	DC bus-type token DC_P or DC_V
<code>vm_pu</code>	<code>DCBus.vm_pu</code>	default 1.0	DC voltage magnitude in per unit
<code>vmax_pu, vmin_pu</code>	same columns	defaults 1.1, 0.9	DC voltage operating bounds
<code>pd_mw</code>	<code>DCBus.pd_mw</code>	default 0.0	embedded DC demand snapshot; workbook export may reflect synchronized aggregate demand
<code>in_service</code>	<code>DCBus.in_service</code>	default <code>true</code>	availability flag

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
latitude, longitude	none	defaults 0.0; import also accepts lat/lon aliases	JSON-only coordinates
base_kv, area, zone, n_customers, importance, is_load	none	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCBranch : dc.branches[] / sheet DCBranch

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, from_bus, to_bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal bus indices
name	DCBranch.name	default empty string	branch label
r_pu	DCBranch.r_pu	default 0.0	branch resistance in per unit
rate_a_mva	DCBranch.rate_a_mva	default 0.0	branch rating
length_km	DCBranch.length_km	default 0.0	physical route length meta-data
in_service	DCBranch.in_service	default true	availability flag
base_kv, s_max_mva, n_parallel, mtbf_hours, mtr_hours, t_scheduled_hr	none	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCLoad : dc.loads[] / sheet DCLoadRaw (fallback DCLoad)

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and connected DC bus
name	same column	default empty string	load label
in_service	same column	default true	availability flag
p_mw	same column	default 0.0	active demand
controllable	same column	default false	controllability flag
p_min_mw	same column	default 0.0	minimum retained demand under curtailment
cost_mw	same column	default 0.0	marginal curtailment penalty
profile_id	none	default -1	JSON-only profile linkage identifier

StaticGeneratorDC : dc.dc_static_generators[] / sheet DCStaticGenerator

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected DC bus
<code>name</code>	<code>DCStaticGenerator.name</code>	default empty string	source label
<code>in_service</code>	<code>DCStaticGenerator.in_service</code>	default <code>true</code>	availability flag
<code>type</code>	<code>DCStaticGenerator.type</code>	default empty string	free-form DC source type
<code>p_set_mw</code>	<code>DCStaticGenerator.p_set_mw</code>	default 0.0	active-power setpoint
<code>scaling</code>	<code>DCStaticGenerator.scaling</code>	default 1.0	profile scaling factor
<code>profile_id</code>	<code>DCStaticGenerator.profile_id</code>	default -1	profile linkage identifier
<code>pmax_mw, pmin_mw</code>	same columns	default 0.0	operating limits
<code>controllable</code>	<code>DCStaticGenerator.controllable</code>	default <code>false</code>	controllability flag
<code>mtbf_hours, mttr_hours</code>	Excel columns <code>mtbf_hr, mttr_hr</code>	default 0.0	reliability metadata; Excel uses the <code>hr</code> suffix
<code>t_scheduled_hr</code>	same column	default 0.0	scheduled outage duration metadata

PVArrayDC : `dc.pv_arrays[]` / sheet `DCPVArray`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and connected DC bus
<code>name</code>	<code>DCPVArray.name</code>	default empty string	DC PV array label
<code>in_service</code>	<code>DCPVArray.in_service</code>	default <code>true</code>	availability flag
<code>p_set_mw</code>	<code>DCPVArray.p_set_mw</code>	default 0.0	active-power setpoint
<code>profile_id</code>	<code>DCPVArray.profile_id</code>	default -1	profile linkage identifier
<code>num_series, num_parallel</code>	same columns	default 0	array topology
<code>vmpp, impp, voc, isc</code>	same columns	default 0.0	module electrical parameters
<code>alpha_isc, beta_voc</code>	same columns	default 0.0	temperature coefficients
<code>temperature, irradiance</code>	same columns	defaults 25.0, 1000.0	operating condition inputs
<code>mtbf_hours, mttr_hours</code>	Excel columns <code>mtbf_hr, mttr_hr</code>	default 0.0	reliability metadata; Excel uses the <code>hr</code> suffix
<code>t_scheduled_hr</code>	same column	default 0.0	scheduled outage duration metadata

VSCConverter : `vsc_converters[]` / sheet `VSCConverter`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus_ac, bus_dc</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and AC/DC terminal bus indices
<code>name</code>	<code>VSCConverter.name</code>	default empty string	converter label
<code>in_service</code>	<code>VSCConverter.in_service</code>	default <code>true</code>	availability flag
<code>control_mode</code>	<code>VSCConverter.control_mode</code>	default <code>PQ</code>	converter-mode token <code>PQ/VD C_Q/VDC_VAC</code>
<code>p_set_mw, q_set_mvar</code>	same columns	default <code>0.0</code>	AC-side active and reactive setpoints
<code>v_dc_set_pu, v_ac_set_pu</code>	same columns	default <code>1.0</code>	DC and AC voltage references
<code>eta</code>	<code>VSCConverter.eta</code>	default <code>0.99</code>	conversion efficiency
<code>loss_percent</code>	<code>VSCConverter.loss_percent</code>	default <code>0.0</code>	generic loss factor
<code>loss_mw</code>	<code>none</code>	default <code>0.0</code>	JSON-only fixed loss term
<code>k_vdc</code>	<code>none</code>	default <code>0.1</code>	JSON-only DC-voltage droop coefficient
<code>pmax_mw, pmin_mw, qmax_mvar, qmin_mvar</code>	same columns	default <code>0.0</code>	operating capability limits
<code>pRated_mw</code>	<code>VSCConverter.pRated_mw</code>	default <code>0.0</code>	converter rating
<code>forced_outage_rate, mtr_hr</code>	<code>none</code>	default <code>0.0</code>	JSON-only reliability meta-data
<code>type, vn_ac_kv, vn_dc_kv, k_p, k_q, v_ref_pu, f_ref_hz, controllable, grid-forming / SC fields, t_scheduled_hr</code>	<code>none</code>	not part of current JSON / Excel contract	newer C++ struct fields that are not preserved by either serializer

DCDCConverter : `dc.dcdc_converters[]` / sheet `DCDCConverter`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index, bus_in, bus_out</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and input / output DC bus indices
<code>name</code>	<code>DCDCConverter.name</code>	default empty string	converter label
<code>in_service</code>	<code>DCDCConverter.in_service</code>	default <code>true</code>	availability flag
<code>control_mode</code>	<code>DCDCConverter.control_mode</code>	default <code>Voltage</code>	control-mode token <code>Voltage/Power/Droop</code>
<code>p_ref_mw</code>	<code>DCDCConverter.p_ref_mw</code>	default <code>0.0</code>	active-power reference
<code>v_ref_pu</code>	<code>DCDCConverter.v_ref_pu</code>	default <code>1.0</code>	output-voltage reference
<code>sn_mva</code>	<code>DCDCConverter.sn_mva</code>	default <code>0.0</code>	converter rating
<code>vn_in_kv, vn_out_kv</code>	<code>none</code>	default <code>0.0</code>	JSON-only nominal voltage ratings
<code>eta</code>	<code>DCDCConverter.eta</code>	default <code>0.98</code>	conversion efficiency
<code>r_eq_pu</code>	<code>none</code>	default <code>0.0</code>	JSON-only equivalent resistance
<code>pmax_mw, pmin_mw</code>	same columns	default <code>0.0</code>	power limits
<code>k_droop</code>	<code>none</code>	default <code>0.0</code>	JSON-only droop coefficient

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
mtbf_hours, mtrr_hours	none	default 0.0	JSON-only reliability meta-data
controllable, f_switching_hz, t_scheduled_hr	none	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

DCCircuitBreaker : dc.dc_circuit_breakers[] / sheet DCCircuitBreaker

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index, bus_from, bus_to	corresponding columns	required on JSON import; blank Excel index stops table import	stable identifier and terminal DC bus indices
name	DCCircuitBreaker.name	default empty string	breaker label
in_service	DCCircuitBreaker.in_service	default true	availability flag
breaker_type	DCCircuitBreaker.breaker_type	default CB	breaker-type token parsed by breaker_type_from_str
closed	DCCircuitBreaker.closed	default true	topology status flag
r_ohm	DCCircuitBreaker.r_ohm	default 0.0	DC breaker resistance
rated_voltage_kv, i_rated_ka, i_breaking_ka	same columns	default 0.0	voltage and current ratings
element_type, element_id	same columns	defaults empty string, 0	optional protected-element reference

EnergyRouter : energy_routers[] / sheet EnergyRouter

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	EnergyRouter.index	required on JSON import; blank Excel index stops table import	stable router identifier
name	EnergyRouter.name	default empty string	router label
in_service	EnergyRouter.in_service	default true	availability flag
router_type	EnergyRouter.router_type	default empty string	free-form router classification token
num_ports	EnergyRouter.num_ports	default 0	expected number of ports; workbook import may later recompute it from the separate port table
ports[]	sheet EnergyRouterPort	missing JSON array keeps the vector empty	nested port objects are flattened into a dedicated workbook sheet
p_rated_mw	EnergyRouter.p_rated_mw	default 0.0	total router rating
vn_ac_kv, vn_dc_kv	same columns	default 0.0	nominal AC/DC voltage ratings
loss_percent	EnergyRouter.loss_percent	default 0.0	aggregate conversion loss factor
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	aggregate operating limits

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
mtbf_hours, mttr_hours	same columns	default 0.0	reliability metadata preserved in both JSON and workbook
control_mode, power_dispatch_strategy, t_scheduled_hr	none	not part of current JSON / Excel contract	C++ struct fields that are not preserved by either serializer

EnergyRouterPort : nested energy_routers[] .ports[] / sheet EnergyRouterPort

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
router_index	EnergyRouterPort.router_index	workbook-only join key	parent router identifier used because ports are nested in JSON but flattened in Excel
index	EnergyRouterPort.index	required on JSON import; workbook row also needs router_index	stable port identifier within its router
name	EnergyRouterPort.name	default empty string	port label
bus	EnergyRouterPort.bus	required on JSON import	connected AC or DC bus index
port_type	EnergyRouterPort.port_type	default AC	side-type token AC or DC
voltage_level_kv	EnergyRouterPort.voltage_level_kv	default 0.0	nominal port voltage level
p_mw, q_mvar	same columns	default 0.0	present active and reactive transfer assigned to the port
v_pu	EnergyRouterPort.v_pu	default 1.0	local port voltage state or initialization
pmax_mw, pmin_mw, qmax_mvar, qmin_mvar	same columns	default 0.0	operating limits
control_mode	EnergyRouterPort.control_mode	default PQ	control-mode token PQ/VF/Droop
p_set_mw, q_set_mvar, v_set_pu	same columns	defaults 0.0, 0.0, 1.0	reference setpoints for the local port controller
in_service	EnergyRouterPort.in_service	default true	availability flag
side, eta	none	not part of current JSON / Excel contract	C++ port fields that are not preserved by either serializer

VirtualPowerPlant : vpps[] / sheet VPP

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
index	VPP.index	required on JSON import; blank Excel index stops table import	stable VPP identifier
name	VPP.name	default empty string	VPP label
description	none	default empty string	JSON-only free-form description

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>aggregation_bus</code> / JSON <code>pcc_bus</code>	VPP. <code>pcc_bus</code>	exporter writes <code>pcc_bus</code> ; importer accepts <code>pcc_bus</code> or legacy <code>aggregation_bus</code> ; default 0	reporting / control aggregation bus stored in the C++ <code>aggregation_bus</code> member
<code>in_service</code>	VPP. <code>in_service</code>	default <code>true</code>	availability flag
<code>n_pv_systems</code> , <code>n_wind_turbines</code> , <code>n_battery_systems</code>	same columns	default 0	asset counts preserved in both JSON and workbook
<code>n_ev_chargers</code> , <code>n_controlrollable_loads</code>	none	default 0	JSON-only aggregation metadata
<code>p_generation_sum_mw</code> , <code>e_storage_sum_mwh</code> , <code>p_load_controllable_mw</code>	none	default 0.0	JSON-only aggregate capability metadata
<code>p_output_mw</code> , <code>q_output_mvar</code>	same columns	default 0.0	present aggregated power output
<code>pmax_mw</code> , <code>pmin_mw</code>	same columns	default 0.0	aggregate operating limits
<code>ramp_up_max_mw_min</code> , <code>ramp_down_max_mw_min</code>	none	default 0.0	JSON-only ramping metadata
<code>mtbf_hours</code> , <code>mttr_hours</code>	none	default 0.0	JSON-only reliability metadata

Microgrid : `microgrids[]` / sheet Microgrid

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code>	Microgrid. <code>index</code>	required on JSON import; blank Excel <code>index</code> stops table import	stable microgrid identifier
<code>name</code>	Microgrid. <code>name</code>	default empty string	microgrid label
<code>description</code>	none	default empty string	JSON-only free-form description
<code>in_service</code>	Microgrid. <code>in_service</code>	default <code>true</code>	availability flag
<code>pcc_bus</code>	Microgrid. <code>pcc_bus</code>	exporter writes <code>pcc_bus</code> ; importer accepts <code>pcc_bus</code> or legacy <code>aggregation_bus</code> ; default 0	point-of-common-coupling bus index stored in the C++ <code>aggregation_bus</code> member
<code>internal_buses</code>	none	missing JSON array keeps the vector empty	JSON-only explicit internal bus membership list
<code>operating_mode</code>	Microgrid. <code>operating_mode</code>	default <code>GridConnected</code>	operating-mode token; one of <code>GridConnected</code> , <code>Islanded</code> , or <code>Transition</code>
<code>islanding_capability</code>	Microgrid. <code>islanding_capability</code>	default <code>false</code>	whether the microgrid may island
<code>auto_reconnection</code>	none	default <code>false</code>	JSON-only reconnection automation flag
<code>p_exchange_max_mw</code> , <code>p_exchange_min_mw</code>	Excel keeps only <code>p_exchange_max_mw</code>	default 0.0	aggregate exchange limits with the host grid
<code>p_import_max_mw</code> , <code>p_export_max_mw</code>	none	default 0.0	JSON-only directional exchange limits

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>p_exchange_mw</code>	<code>Microgrid.p_exchange_mw</code>	default 0.0	present net power exchange at the PCC
<code>total_generation_mw</code> , <code>total_load_mw</code>	same columns	default 0.0	current aggregate generation and demand
<code>total_storage_mwh</code> , <code>capacity_mw</code> , <code>peak_load_mw</code>	none	default 0.0	JSON-only planning meta-data
<code>f_set_hz</code> , <code>v_set_pu</code> , <code>k_droop</code>	none	defaults 50.0, 1.0, 0.0	JSON-only islanded-control targets
<code>area</code>	none	default 0	JSON-only area tag
<code>mtbf_hours</code> , <code>mttr_hours</code>	none	default 0.0	JSON-only reliability meta-data

MobileStorage : `mobile_storage[]` / sheet `MobileStorage`

Field	Excel sheet / column	JSON default / import rule	Meaning / preservation note
<code>index</code> , <code>bus</code>	corresponding columns	required on JSON import; blank Excel <code>index</code> stops table import	stable identifier and current connected bus
<code>name</code>	<code>MobileStorage.name</code>	default empty string	unit label
<code>in_service</code>	<code>MobileStorage.in_service</code>	default <code>true</code>	availability flag
<code>p_mw</code> , <code>q_mvar</code>	same columns	default 0.0	present dispatch at the current location
<code>p_rated_mw</code>	<code>MobileStorage.p_rated_mw</code>	default 0.0	converter power rating
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	none	default 0.0	JSON-only operating limits
<code>e_rated_mwh</code>	<code>MobileStorage.e_rated_mwh</code>	default 0.0	energy capacity
<code>soc_init</code>	<code>MobileStorage.soc_init</code>	default 0.5	initial state of charge
<code>soc_min</code> , <code>soc_max</code>	none	defaults 0.1, 0.9	JSON-only admissible state-of-charge bounds
<code>eta_charge</code> , <code>eta_discharge</code>	none	default 0.95	JSON-only efficiency meta-data
<code>max_cycles</code> , <code>current_cycles</code> , <code>soh</code>	none	defaults 5000, 0, 1.0	JSON-only lifecycle / health metadata
<code>is_mobile</code>	none	default <code>true</code>	JSON-only flag distinguishing mobile vs. stationary storage representation
<code>status</code>	<code>MobileStorage.status</code>	default <code>Stationary</code>	mobility-state token; one of <code>Stationary</code> , <code>InTransit</code> , or <code>Deployed</code>
<code>current_location</code>	none	default empty string	JSON-only free-form location string
<code>target_bus</code>	none	default 0	JSON-only relocation target
<code>e_consumption_mwh_km</code> , <code>max_travel_distance_km</code>	none	default 0.0	JSON-only transportation-energy metadata
<code>mtbf_hours</code> , <code>mttr_hours</code>	none	default 0.0	JSON-only reliability meta-data

10.5 Cross-Reference

The only remaining detailed dictionaries not repeated in this section are the ones already written in Section 02a_io_contracts:

1. AsynchronousMotor
2. the optional three_phase_ac metadata container
3. ThreePhaseBus, ThreePhaseLine, ThreePhaseTransformer, ThreePhaseLoad, ThreePhaseGenerator, and ThreePhaseExternalGrid
4. result-sheet enum and flag conventions

11 Output and Report Field Dictionaries

This section applies the same field-level dictionary template to the current export artifacts. Section 02a_io_contracts already catalogs which workbooks and JSON serializers exist; the tables below make the output-side contract explicit at workbook-column and JSON-key level.

11.1 Export Families

Artifact	Export path	Direction	Current note
PowerFlowResult dedicated workbook	source-only workbook helper	export only	writes the Results_* sheets only when optional Excel support is compiled.
PowerFlowResult compact workbook	save_results_compact.xlsx	export only	rewrites input sheets and appends result_* columns to selected tables, plus Results_Summary.
PowerFlowResult JSON	power_flow_result_to_json / save_power_flow_result.json	export only	serializes convergence summary plus AC/DC bus states, AC/DC branch flows, VSC transfers, and DC/DC transfers.
ACOPFResult dedicated workbook	save_opf_results.xlsx	export only	writes OPF_* sheets only.
ACOPFResult compact workbook	save_opf_results_compact.xlsx	export only	rewrites input sheets but still emits separate OPF_* sheets; no opf_* columns are appended into input tables.
ACOPFResult JSON	opf_result_to_json / opf_result_from_json	bidirectional	serializes scalar summary, solver_path, bus / generator / converter vectors, enhanced dispatch objects with maps, legacy dispatch arrays, and OPF profiling block.
CarbonAnalysisResult workbook	save_carbon_results.xlsx	export only	writes AC/DC load and bus sheets, AC/DC branch-loss sheets, converter-loss sheets, storage / EnergyRouter sheets, and block-style Summary.

Artifact	Export path	Direction	Current note
CarbonAnalysis Result JSON	<code>carbon_result_to_json / carbon_result_from_json</code>	bidirectional	preserves the richer tracing maps and the hybrid AC/DC carrier results that the workbook flattens.
Short-circuit and ONR artifacts	absent from the current public API	none	the current public headers in this repository do not expose <code>compute_short_circuit</code> , <code>BusFaultResult</code> , or <code>solve_optimal_reconfiguration</code> ; no output field contract is documented for them.

11.2 Result Semantic Conventions

Semantic aspect	Current rule	Main artifacts	Engineering note
Electrical units	Active power uses MW, reactive power uses Mvar, AC / DC voltage magnitudes use pu, AC voltage angle uses degree, and current stays in pu unless the field name explicitly says <code>ka</code> .	PF / OPF workbooks and JSON	Workbook and JSON field names usually encode the unit directly, for example <code>pg_mw</code> , <code>va_deg</code> , <code>loss_mw</code> , and <code>matrix_residual</code> .
Carbon units	Carbon intensity uses <code>tCO2/MWh</code> ; total emissions use <code>tCO2</code> .	Carbon workbook and JSON	The JSON / workbook field names use the suffixes <code>_tco2_mwh</code> and <code>_tco2</code> ; no implicit mass-unit conversion is applied by the exporter.
Vector order	Numerical result vectors follow solver-side component order, not sparse physical ids. <code>vm</code> , <code>va</code> , <code>dpg_mw</code> , and <code>dqd_mvar</code> follow AC-bus order; <code>vdc</code> follows DC-bus order; <code>pg_mw</code> , <code>qg_mvar</code> follow AC-generator order; <code>pac_mw</code> , <code>qac_mvar</code> follow VSC order.	PF / OPF JSON and workbook exports	Consumers should treat workbook rows and JSON vectors as aligned to the source component arrays unless a sheet explicitly says otherwise.

Semantic aspect	Current rule	Main artifacts	Engineering note
Row identity	Workbook result sheets copy source model <code>index</code> values when a source component exists; <code>Results_BranchFlow.index</code> is currently <code>ACBranch::index</code> .	PF workbooks and PF JSON	For branch-flow joins, use the source component index plus terminal buses; do not assume row number equals component id.
Fallback values	When a solver vector entry is absent or out of range, exporters write deterministic defaults: typically 0.0 for powers, angles, counters, and residual-like scalars, or 1.0 for voltage magnitudes.	PF / OPF workbooks	This keeps workbook shapes stable, but it means a consumer must distinguish “solver wrote a fallback” from “the physical result is exactly zero” using convergence and artifact context.
JSON missing keys	JSON importers keep absent arrays or object maps empty and default absent scalar keys to false, zero, or empty string according to the target field type.	OPF / carbon JSON	The JSON contracts are intentionally sparse-friendly; round-trip consumers should not require every optional key to be present.
Echoed vs solved values	Some exported fields are metadata echoes or derived annotations rather than true solver states: storage <code>soc</code> comes from input <code>soc_init</code> ; router port sums come from stored port structs; <code>Results_DCBranch</code> is recomputed from solved voltages and branch resistance.	PF workbooks	Consumers should not infer that every result-sheet field belongs to the nonlinear state vector.
Boolean spelling	PF / OPF workbook booleans use the standard workbook token form produced by <code>bool_str</code> ; carbon <code>Summary</code> uses lowercase <code>true/false</code> .	Result workbooks	Downstream parsers should normalize workbook booleans instead of assuming one literal spelling.
Conditional sheets	Some result sheets are pre-created even when no rows are written. <code>OPF_Converter</code> is the main example.	PF / OPF workbooks	Empty sheets are part of the current export contract and do not by themselves indicate an error.

Semantic aspect	Current rule	Main artifacts	Engineering note
Object-map semantics	Carbon contribution maps are exported as JSON object maps keyed by internal carbon-source id rather than positional arrays.	Carbon JSON	Map-key order is not semantically meaningful; workbook export intentionally flattens these maps away.
Stub / absent analysis outputs	Distribution power flow, resilience, and carbon analysis currently return stub-style results; short-circuit and ONR public output APIs are absent.	analysis namespace and scope boundary	The serializers listed here describe existing export code paths, but they do not imply that every analysis family has an implemented solver producing those artifacts.

11.3 Solver-Specific Result Notes

Solver path	Result fields with special meaning	Where visible	Current note
Hybrid PF Newton / FDPF / DC PF	<code>converged</code> , <code>iterations</code> , <code>residual</code>	<code>Results_Summary</code> and compact PF summary sheet	The public PF artifacts expose only the compact scalar summary; detailed profiling arrays stay in memory and are not serialized.
Hybrid PF converter transfer postprocessing	<code>p_ac_mw</code> , <code>q_ac_mvar</code> , <code>p_dc_mw</code> , <code>loss_mw</code> , plus DC/DC transfer fields	dedicated and compact PF workbooks	Converter rows are matched by converter <code>index</code> ; when a solved transfer is missing, the exporter falls back to setpoints and static metadata.
Native AC OPF IPM	<code>outer_iterations</code> , <code>profiling.final_barrier_mu</code> , profiling counters	<code>OPF_Summary</code> and <code>ACOPFResult</code> JSON	These fields are meaningful primarily for the native IPM path; they may remain zero-like on simpler fallback paths.
Parity OPF formulation	<code>dpd_mw</code> , <code>dqd_mvar</code>	<code>OPF_Bus</code> and <code>ACOPFResult</code> JSON	The parity path exposes explicit active / reactive load-shedding vectors. On runs without shedding, the vectors may still exist but stay numerically zero.
OPF fallback selection	<code>status</code> , convergence flags, and partial vectors	<code>OPF_Summary</code> , <code>ACOPFResult</code> JSON	Consumers should read <code>status</code> together with <code>converged</code> ; some fallback paths can return structurally valid but less detailed result blocks.

Solver path	Result fields with special meaning	Where visible	Current note
Carbon tracing method	<code>tracing_verified</code> and <code>tracing_summary.*</code>	carbon workbook and JSON	These fields describe graph-tracing consistency, not matrix-solver success.
Carbon matrix method	<code>matrix_solved</code> , <code>matrix_residual</code> , and <code>matrix_summary.*</code>	carbon workbook and JSON	<code>matrix_residual</code> is specific to the matrix solve; it is not directly comparable to PF or OPF residual metrics.
Distribution resilience stub	<code>completed=false</code> , <code>status=not_implemented</code>	public API result object	the facade exposes <code>run_distribution_resilience_assessment</code> , but the implementation currently returns a stub result rather than a populated report workbook or JSON artifact.

11.4 Consumer Contracts

Results_* workbook family

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
Results_ACBus , Results_DCBus	<code>index</code>	voltage fields are stable; <code>name</code> is descriptive only	consumers should join by <code>index</code> rather than row number even though row order follows the source bus tables.
Entity sheets with explicit ids	<code>index</code> on converter, storage, charging-station, charger, router, VPP, micro-grid, and mobile-storage sheets	identity and meta-data columns are stable; solved voltages / transfer values may fall back to deterministic defaults	additive columns are expected to be backward compatible; consumers should read by header name instead of fixed workbook position.
Results_EnergyRouterPort	pair (<code>router_index</code> , <code>port_index</code>)	solved transfer fields may fall back to stored port meta-data; <code>port_name</code> is descriptive only	the flattened port contract is stable at the key-field level, not at the row-order level.
Results_BranchFlow	<code>index</code> copied from <code>ACBranch::index</code> , plus <code>from_bus/to_bus</code>	<code>flow</code> , <code>loss_mw</code> , and <code>loading_pct</code> fields are stable	consumers should still read by header name and tolerate non-contiguous source branch ids.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
Results_Summary	Key	current scalar keys are stable; extra keys may appear additively	consumers should parse it as a name/value sheet and must not depend on fixed row numbers.

OPF_* workbook family and ACOPFResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
OPF_Bus	index	vm_pu, va_deg, dpd_mw, and dqd_mvar are the stable solved fields; name and base_kv are echoes	workbook rows align to <code>sys.ac.buses</code> , but consumers should still join by <code>index</code> .
OPF_Generator	index	dispatch fields are stable; cost and fuel_type columns are echoed metadata	unknown future columns should be ignored, and consumers should not assume the sheet contains only solved variables.
OPF_Converter	index	pac_mw and qac_mvar are stable when the sheet has rows	the sheet may be present but empty when no VSC converters exist, so emptiness is not itself an error condition.
OPF_Summary and JSON scalar block	workbook Key; JSON top-level scalar names	convergence, iteration, objective, residual-like, solver_path, and status fields are stable	summary consumers should tolerate additive keys while continuing to treat existing keys as stable.
ACOPFResult vector and dispatch blocks	AC-bus order for vm, va, vdc, dpd_mw, dqd_mvar; generator order for pg_mw, qg_mvar; VSC order for pac_mw, qac_mvar; map arrays for enhanced dispatch	the profiling object is optional on import; absent arrays stay empty	JSON serializes renewable_dispatch, storage_dispatch, dcdc_dispatch, flex_dispatch, plus legacy arrays pren_mw, qren_mvar, pstor_mw, qstor_mvar, pdcdc_mw, and pflex_mw.

CarbonAnalysisResult JSON

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
top-level scalar block	top-level key names	<code>tracing_verified</code> , <code>matrix_solved</code> , and <code>matrix_residual</code> are stable; absent keys default to zero-like values on import	additive scalar keys are backward compatible because the current importer only reads known fields.
<code>load_carbon[]</code> and <code>dc_load_carbon[]</code>	<code>load_index</code>	the explicit scalar fields are stable; <code>generator_supply_mw</code> may be absent and then defaults to an empty map	array order follows producer-side load order, but consumers should join by <code>load_index</code> when combining with other artifacts.
<code>branch_carbon[]</code> and <code>dc_branch_carbon[]</code>	<code>branch_index</code>	the explicit scalar fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	object-map key order is not stable and must not be used as a semantic ordering.
<code>bus_carbon[]</code> and <code>dc_bus_carbon[]</code>	<code>bus_index</code>	the explicit scalar fields are stable; absent array keeps the vector empty	row order follows producer-side bus order, but <code>bus_index</code> is the durable join key.
<code>vsc_carbon[]</code> and <code>dc_dc_carbon[]</code>	<code>converter_index</code>	scalar direction and power fields are stable; <code>generator_loss_mw</code> may be absent and then defaults to an empty map	consumers should join on converter index and not rely on array order once projected-canonical filtering is enabled.
<code>storage_carbon[]</code>	pair (<code>storage_index</code> , <code>is_dc</code>)	scalar storage-state fields are stable; <code>source_supply_mw</code> may be absent and then defaults to an empty map	<code>p_mw</code> uses the bus-injection sign convention and determines whether the row should be interpreted as discharge or charging sink.
<code>energy_router_carbon[]</code>	<code>router_index</code>	scalar router loss fields are stable; <code>source_loss_mw</code> may be absent and then defaults to an empty map	consumers should join by router index and not infer semantics from workbook row order.

Consumer slice	Join key	Stable / optional fields	Compatibility constraint
generator-allocation maps and summary blocks	source-id string for maps; top-level object name for summaries	<code>generator_loss_allocation</code> , <code>tracing_summary</code> , and <code>matrix_summary</code> are stable object names	map keys are JSON strings of internal source ids, and consumers must normalize them back to integer identifiers if a typed downstream schema requires that.

11.5 Power-Flow Result JSON

JSON key / row object	Fields	Population rule	Meaning / note
top-level scalar block	<code>converged</code> , <code>iterations</code> , <code>residual</code>	copied from <code>PowerFlowResult</code>	solver summary for the exported result
<code>ac_buses[]</code>	<code>index</code> , <code>name</code> , <code>vm_pu</code> , <code>va_deg</code>	component identity from <code>sys.ac.buses</code> ; solved values from <code>result.vm/result.va</code> , falling back to stored bus <code>vm_pu/va_deg</code>	solved AC bus states
<code>dc_buses[]</code>	<code>index</code> , <code>name</code> , <code>vm_pu</code>	component identity from <code>sys.dc.buses</code> ; solved value from <code>result.vdc</code> , falling back to stored bus <code>vm_pu</code>	solved DC bus states
<code>ac_branch_flows[]</code>	<code>index</code> , <code>name</code> , <code>from_bus</code> , <code>to_bus</code> , <code>pf_mw</code> , <code>qf_mvar</code> , <code>pt_mw</code> , <code>qt_mvar</code> , <code>loss_mw</code> , <code>loading_pct</code>	<code>index</code> and terminals copied from <code>ACBranch</code> ; flow values copied from <code>result.branch_flows</code> ; loading computed against rating fallback logic	AC branch-flow rows use original branch indices, not row ordinals
<code>dc_branch_flows[]</code>	<code>index</code> , <code>name</code> , <code>from_bus</code> , <code>to_bus</code> , <code>pf_mw</code> , <code>pt_mw</code> , <code>loss_mw</code> , <code>loading_pct</code>	component identity from <code>DCBranch</code> ; flow values from <code>result.dc_branch_flows</code> when present	DC branch-flow rows
<code>vsc_transfers[]</code>	<code>index</code> , <code>bus_ac</code> , <code>bus_dc</code> , <code>p_ac_mw</code> , <code>q_ac_mvar</code> , <code>p_dc_mw</code> , <code>loss_mw</code>	copied from <code>result.vsc_transfers</code> aligned with VSC order	AC/DC converter transfer rows

JSON key / row object	Fields	Population rule	Meaning / note
<code>dcdc_transfers</code> []	<code>index</code> , <code>bus_in</code> , <code>bus_out</code> , <code>p_in_mw</code> , <code>p_out_mw</code> , <code>loss_mw</code>	copied from <code>result.dcdc_transfers</code> aligned with DC/DC converter order	DC/DC converter transfer rows

11.6 Dedicated Power-Flow Result Sheets

Results_ACBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_ACBus.index</code>	copied from <code>sys.ac.buses[i].index</code>	stable AC bus identifier
<code>name</code>	<code>Results_ACBus.name</code>	copied from <code>sys.ac.buses[i].name</code>	descriptive bus label
<code>vm_pu</code> , <code>va_deg</code>	same columns	copied from <code>result.v</code> <code>m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	solved AC voltage magnitude and angle in bus-vector order

Results_DCBus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_DCBus.index</code>	copied from <code>sys.dc.buses[i].index</code>	stable DC bus identifier
<code>name</code>	<code>Results_DCBus.name</code>	copied from <code>sys.dc.buses[i].name</code>	descriptive bus label
<code>vm_pu</code>	<code>Results_DCBus.vm_pu</code>	copied from <code>result.vdc[i]</code> ; out-of-range fallback 0.0	solved DC voltage; the sheet keeps the AC-style column name <code>vm_pu</code>

Results_BranchFlow

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code>	<code>Results_BranchFlow.index</code>	copied from <code>sys.ac.branches[i].index</code>	source AC branch identifier
<code>from_bus</code> , <code>to_bus</code>	same columns	copied from <code>sys.ac.branches[i]</code> when present; fallback 0	terminal buses of the reported AC branch row
<code>pf_mw</code> , <code>qf_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	sending-end active and reactive power
<code>pt_mw</code> , <code>qt_mvar</code>	same columns	copied from <code>result.branch_flows[i]</code>	receiving-end active and reactive power
<code>loss_mw</code>	<code>Results_BranchFlow.loss_mw</code>	computed as <code>max(pf_mw+pt_mw, 0.0)</code>	exported AC branch loss

Field	Workbook sheet / column	Population rule	Meaning / note
loading_pct	Results_BranchFlow.loading_pct	computed from the larger terminal apparent power over <code>rate_a_mva</code> , system base MVA, or AC base MVA fallback	branch loading percentage

Results_DCBranch

Field	Workbook sheet / column	Population rule	Meaning / note
index, name	corresponding columns	copied from <code>sys.dc.branches[i]</code>	stable identifier and label
from_bus, to_bus	same columns	copied from <code>sys.dc.branches[i]</code>	terminal DC bus indices
in_service	same column	copied from <code>sys.dc.branches[i].in_service</code> via <code>bool_str</code>	availability flag in workbook boolean-token form
v_from_pu, v_to_pu	same columns	looked up from solved <code>result.vdc</code> by branch terminal bus; fallback 1.0	solved DC terminal voltages
i_pu	Results_DCBranch.i_pu	computed as $(v_{\text{from}} - v_{\text{to}})/r_{\text{pu}}$ when in service and <code>r_pu</code> >0; else 0.0	derived branch current on system base
p_from_mw, p_to_mw	same columns	computed from solved terminal voltages, derived current, and system base MVA	terminal powers implied by the solved DC branch state
loss_mw	Results_DCBranch.loss_mw	computed as <code>p_from_mw+p_to_mw</code>	exported branch-loss figure
loading_pct	Results_DCBranch.loading_pct	computed from terminal active powers over <code>s_max_mva</code> , <code>rate_a_mva</code> , or base MVA fallback	DC branch loading percentage

Results_VSCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus_ac, bus_dc	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable converter identity and terminal buses
in_service, control_mode	same columns	copied from the input converter object	static converter metadata, not a solver-selected mode switch
p_ac_mw, q_ac_mvar, p_dc_mw, loss_mw	same columns	taken from matching <code>result.vsc_transfers[index]</code> when available; fallback uses <code>p_set_mw</code> , <code>q_set_mvar</code> , and stored <code>loss_mw</code>	solved transfer report with deterministic metadata fallback
p_set_mw, q_set_mvar	same columns	copied from <code>sys.vsc_converters[i]</code>	original setpoint metadata retained beside solved transfer values
v_ac_pu, v_dc_pu	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal AC and DC voltage annotations

Field	Workbook sheet / column	Population rule	Meaning / note
<code>eta, loss_percent</code>	same columns	copied from the input converter object	static efficiency / loss metadata

Results_DCDCConverter

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name, bus_in, bus_out</code>	corresponding columns	copied from <code>sys.dcdc_converters[i]</code>	stable converter identity and terminal buses
<code>in_service, control_mode</code>	same columns	copied from the input converter object	static converter metadata
<code>p_ref_mw, v_ref_pu, eta</code>	same columns	copied from <code>sys.dcdc_converters[i]</code>	original reference and efficiency metadata
<code>v_in_pu, v_out_pu</code>	same columns	solved voltages looked up by terminal buses; fallback 1.0	terminal DC voltage annotations
<code>p_in_mw, p_out_mw, loss_mw</code>	same columns	taken from matching <code>result.dcdc_transfers[index]</code> when available; fallback uses <code>p_ref_mw</code> and derived zero-loss default	solved DC/DC transfer report with metadata fallback

Results_Storage_AC

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name, bus, in_service</code>	corresponding columns	copied from <code>sys.ac.storage[i]</code>	stable identity, connected bus, and availability
<code>p_mw, q_mvar</code>	same columns	copied from <code>sys.ac.storage[i]</code>	current storage dispatch snapshot; not solved from a storage state estimator
<code>soc</code>	Results_Storage_AC.soc	copied from <code>sys.ac.storage[i].soc_init</code>	reported state of charge is the input initial SOC
<code>soh</code>	Results_Storage_AC.soh	copied from <code>sys.ac.storage[i].soh</code>	stored state-of-health metadata
<code>vm_pu, va_deg</code>	same columns	solved AC bus voltage looked up by <code>bus</code> ; fallback 1.0/0.0	bus-state annotation for the storage location

Results_Storage_DC

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index, name, bus, in_service</code>	corresponding columns	copied from <code>sys.dc.storage[i]</code>	stable identity, connected bus, and availability
<code>p_mw, q_mvar</code>	same columns	copied from <code>sys.dc.storage[i]</code>	current storage dispatch snapshot
<code>soc</code>	Results_Storage_DC.soc	copied from <code>sys.dc.storage[i].soc_init</code>	reported state of charge is the input initial SOC
<code>soh</code>	Results_Storage_DC.soh	copied from <code>sys.dc.storage[i].soh</code>	stored state-of-health metadata

Field	Workbook sheet / column	Population rule	Meaning / note
vdc_pu	Results_Storage_DC.vdc_pu	solved DC voltage looked up by bus ; fallback 1.0	bus-state annotation for the storage location

Results_ChargingStation

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, in_service	corresponding columns	copied from sys.ac.charging_stations[i]	stable station identity, connected bus, and availability
num_chargers, p_total_kw, q_total_kvar, max_power_kw, utilization_rate	same columns	copied from sys.ac.charging_stations[i]	current aggregate station metadata; no per-solver charging dispatch reconstruction is performed here
vm_pu, va_deg	same columns	solved AC bus voltage looked up by bus ; fallback 1.0/0.0	bus-state annotation for the station location

Results_Charger

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, station_id	corresponding columns	copied from sys.ac.chargers[i]	stable charger identity and parent-station link
station_bus	Results_Charger.station_bus	joined from the parent charging-station map by station_id ; fallback 0	resolved AC bus of the parent charging station
in_service, charger_type, p_rate_d_kw, p_ch_max_kw, p_dis_max_kw, eta, v2g_capable	same columns	copied from sys.ac.chargers[i]	static charger metadata repeated in the result workbook
vm_pu, va_deg	same columns	solved AC bus voltage looked up by the resolved station_bus ; fallback 1.0/0.0	station-bus voltage annotation, not per-charger internal electrical state

Results_EnergyRouter

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, in_service, router_type, num_ports, p_rated_mw, loss_percent	corresponding columns	copied from sys.energy_routers[i]	router metadata preserved in the result workbook
p_port_sum_mw, q_port_sum_mvar	same columns	computed by summing the stored router-port p_mw / q_mvar values	router-level summary over the current port structs; it does not re-sum the solved transfer table

Results_EnergyRouterPort

Field	Workbook sheet / column	Population rule	Meaning / note
router_index, port_index, port_name, bus	corresponding columns	copied from each nested port row under <code>sys.energy_routers[*].ports[*]</code>	flattened router / port identity and connected bus
port_type, control_mode, in_service	same columns	copied from the nested port object	static port metadata
p_mw, q_mvar, v_solved_pu	same columns	taken from the matching solved router-port transfer when available; fallback uses the port's stored <code>p_mw</code> , <code>q_mvar</code> , and solved bus voltage	solved per-port transfer report with deterministic metadata fallback

Results_VPP

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, aggregation_bus, in_service	corresponding columns	copied from <code>sys.vpps[i]</code>	stable VPP identity and aggregation location
p_output_mw, q_output_mvar, pmax_mw, pmin_mw	same columns	copied from <code>sys.vpps[i]</code>	current VPP aggregate output / limits metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>aggregation_bus</code> ; fallback 1.0/0.0	aggregation-bus state annotation

Results_Microgrid

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, pcc_bus, operating_mode, in_service	corresponding columns	copied from <code>sys.microgrids[i]</code>	microgrid identity and current mode metadata
p_exchange_mw, total_generation_mw, total_load_mw	same columns	copied from <code>sys.microgrids[i]</code>	current aggregate exchange / generation / demand snapshot
vm_pu, va_deg	same columns	solved AC bus voltage looked up by <code>pcc_bus</code> ; fallback 1.0/0.0	PCC bus-state annotation
internal_buses	Results_Microgrid.internal_buses	comma-joined string formed from the stored microgrid membership vector	flattened membership list; workbook does not preserve the vector structure directly

Results_MobileStorage

Field	Workbook sheet / column	Population rule	Meaning / note
index, name, bus, status, in_service	corresponding columns	copied from <code>sys.mobile_storage[i]</code>	mobile-storage identity, current location, and mobility state metadata
p_mw, q_mvar	same columns	copied from <code>sys.mobile_storage[i]</code>	current dispatch snapshot
soc	<code>Results_MobileStorage.soc</code>	copied from <code>sys.mobile_storage[i].soc_init</code>	reported state of charge is the input initial SOC
target_bus	<code>Results_MobileStorage.target_bus</code>	copied from <code>sys.mobile_storage[i].target_buses</code>	relocation target metadata
vm_pu, va_deg	same columns	solved AC bus voltage looked up by the current bus; fallback 1.0/0.0	current-bus state annotation

Results_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	<code>Results_Summary</code> row with Key=converged	written from <code>result.converged</code> via <code>bool_string</code>	PF convergence flag using workbook boolean-token form
iterations	<code>Results_Summary</code> row with Key=iterations	written from <code>result.iterations</code>	total PF iteration count
residual	<code>Results_Summary</code> row with Key=residual	written from <code>result.residual</code>	final PF residual metric

11.7 Compact Power-Flow Result Columns

Field	Workbook sheet / column	Population rule	Meaning / note
result_vm_pu, result_va_deg	ACBus appended columns	copied from <code>result.v_m[i]</code> and <code>result.va[i]</code> in AC bus order	compact PF solved AC bus state
result_vdc_pu	DCBus appended column	copied from <code>result.vdc[i]</code> in DC bus order	compact PF solved DC bus voltage
result_pf_mw, result_qf_mvar, result_pt_mw, result_qt_mvar	ACBranch appended columns	copied from <code>result.branch_flows[i]</code>	compact PF AC branch-flow report
result_v_from_pu, result_v_to_pu, result_i_pu, result_p_from_mw, result_p_to_mw, result_loss_mw	DCBranch appended columns	recomputed from solved DC voltages and branch resistance, same as <code>Results_DCBranch</code>	compact PF DC branch report
result_vm_pu, result_va_deg	Storage, Load, ACLoad, ChargingStation, VPP, Microgrid, MobileStorage appended columns	solved AC voltage looked up by each row's connected bus or aggregation bus	compact PF AC-side bus-state annotation on input tables
result_vdc_pu	DCStorage, DCLoad appended column	solved DC voltage looked up by each row's connected bus	compact PF DC-side bus-state annotation on input tables

Field	Workbook sheet / column	Population rule	Meaning / note
<code>result_p_ac_mw</code> , <code>result_q_ac_mvar</code> , <code>result_p_dc_mw</code> , <code>result_loss_mw</code> , <code>result_v_ac_pu</code> , <code>result_v_dc_pu</code>	VSCConverter appended columns	populated from matching <code>result.vsc_transfers[index]</code> plus solved terminal voltages	compact PF converter transfer report
<code>result_p_in_mw</code> , <code>result_p_out_mw</code> , <code>result_loss_mw</code> , <code>result_v_in_pu</code> , <code>result_v_out_pu</code>	DCDCConverter appended columns	populated from matching <code>result.dcdc_transfers[index]</code> plus solved terminal voltages	compact PF DC/DC converter report
<code>Results_Summary</code>	separate sheet still written in compact PF export	same key-value rows as the dedicated PF result workbook	compact PF export still carries a standalone summary sheet

11.8 Dedicated OPF Sheets

OPF_Bus

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>base_kv</code>	corresponding columns	copied from <code>sys.ac.buses[i]</code>	stable bus identity and base-voltage metadata
<code>vm_pu</code> , <code>va_deg</code>	same columns	copied from <code>result.v_m[i]</code> and <code>result.va[i]</code> ; out-of-range fallback 0.0	optimal AC bus voltage solution
<code>dpd_mw</code> , <code>dqd_mvar</code>	same columns	copied from <code>result.dpd_mw[i]</code> and <code>result.dqd_mvar[i]</code> ; out-of-range fallback 0.0	parity-formulation bus-level active / reactive load shedding

OPF_Generator

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>bus</code>	corresponding columns	copied from <code>sys.ac.generators[i]</code>	stable generator identity and connected bus
<code>pg_mw</code> , <code>qg_mvar</code>	same columns	copied from <code>result.pg_mw[i]</code> and <code>result.qg_mvar[i]</code> ; fallback to input <code>pg_mw/qg_mvar</code>	optimal generator dispatch
<code>pmax_mw</code> , <code>pmin_mw</code> , <code>qmax_mvar</code> , <code>qmin_mvar</code>	same columns	copied from <code>sys.ac.generators[i]</code>	generator capability metadata retained beside the optimal point
<code>cost_c2</code> , <code>cost_c1</code> , <code>cost_c0</code> , <code>fuel_type</code>	same columns	copied from <code>sys.ac.generators[i]</code>	cost-model and fuel metadata used for interpretation

OPF_Converter

Field	Workbook sheet / column	Population rule	Meaning / note
<code>index</code> , <code>name</code> , <code>bus_ac</code> , <code>bus_dc</code>	corresponding columns	copied from <code>sys.vsc_converters[i]</code>	stable VSC identity and terminal buses

Field	Workbook sheet / column	Population rule	Meaning / note
pac_mw, qac_mvar	same columns	copied from <code>result.pac_mw[i]</code> and <code>result.qac_mvar[i]</code> ; out-of-range fallback 0.0	optimal AC-side converter operating point

OPF_Summary

Field	Workbook sheet / column	Population rule	Meaning / note
converged	OPF_Summary row with Key=converged	written from <code>result.converged</code> via <code>bool_str</code>	OPF convergence flag
iterations	OPF_Summary row with Key=iterations	written from <code>result.iterations</code>	total inner-iteration count
outer_iterations	OPF_Summary row with Key=outer_iterations	written from <code>result.outer_iterations</code>	barrier / outer-loop iteration count
objective	OPF_Summary row with Key=objective	written from <code>result.objective</code>	final objective value
max_constraint_violation	OPF_Summary row with Key=max_constraint_violation	written from <code>result.max_constraint_violation</code>	worst final feasibility residual
max_stationarity	OPF_Summary row with Key=max_stationarity	written from <code>result.max_stationarity</code>	worst final first-order optimality metric
status	OPF_Summary row with Key=status	written from <code>result.status</code>	textual solver termination status

ACOPFResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
converged, iterations, outer_iterations, objective, max_constraint_violation, max_stationarity, status	top-level scalar keys	bidirectional; missing keys default to false, zero, or empty string	OPF scalar summary block
vm, va	top-level arrays	bidirectional; absent array keeps the vector empty	bus-voltage vectors in bus order
pg_mw, qg_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	generator dispatch vectors in generator order
dpd_mw, dqd_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	bus-level parity load-shedding vectors
pac_mw, qac_mvar	top-level arrays	bidirectional; absent array keeps the vector empty	VSC AC-side operating-point vectors
profiling.linear_solver_backend	profiling[linear_solver_backend]	bidirectional; default empty string	linear solver backend name recorded by the OPF run
profiling.analyze_calls, profiling.linear_solve_calls	same nested keys	bidirectional; default 0	OPF linear-solver call counters
factorization_calls	same nested key	bidirectional; default 0	OPF sparse factorization counter

Field	JSON key / path	Export / import rule	Meaning / note
profiling.total_ iterations	profiling[total_iterations]	bidirectional; default 0	total OPF inner iterations reported by the solver
profiling. accepted_steps	same nested key	bidirectional; default 0	accepted line-search steps
profiling. rejected_steps	same nested key	bidirectional; default 0	rejected line-search steps
profiling.final_ barrier_mu	profiling[final_barrier_mu]	bidirectional; default 0.0	final barrier parameter reported by the IPM path
pren_mw, qren_mv r, pstor_mw, qsto r_mvar, pdcdc_mw, pflex_mw	none	not part of the current JSON or workbook contract	enhanced component-dispatch vectors exist in ACOPFResult but are not serialized today

11.9 Carbon Result JSON and Workbook Dictionaries

CarbonAnalysisResult JSON

Field	JSON key / path	Export / import rule	Meaning / note
tracing_verified, matrix_solved	top-level scalar booleans	bidirectional; default false	method-level status flags for tracing and matrix solutions
matrix_residual	top-level scalar number	bidirectional; default 0.0	residual metric from the matrix method
load_carbon[] and dc_load_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	per-load tracing results for AC loads and DC loads respectively
*.load_index, bus, demand_mw, carbon_intensity _tco2_mwh, total_ emissions_tco2	nested keys on each AC/DC load row	bidirectional; defaults 0 or 0.0	flat load-level carbon report fields
per-load generato r_supply_mw map	nested object map keyed by source id	bidirectional; absent map keeps the per-load map empty	source-contribution map preserved only in JSON form
branch_carbon[] and dc_branch_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	per-branch loss-allocation results for AC branches and DC branches respectively
*.branch_index, from_bus, to_bus, loss_mw, carbon_i ntensity_tco2_mw h, total_emission s_tco2	nested keys on each AC/DC branch row	bidirectional; defaults 0 or 0.0	flat branch-loss carbon report fields
per-branch genera tor_loss_mw map	nested object map keyed by source id	bidirectional; absent map keeps the per-branch map empty	source loss-allocation map preserved only in JSON form
bus_carbon[] and dc_bus_carbon[]	top-level arrays	bidirectional; absent array keeps the vector empty	matrix-based AC-bus and DC-bus carbon intensities
*.bus_index, carbon_intensity _tco2_mwh	nested keys on each AC/DC bus row	bidirectional; defaults 0 or 0.0	explicit bus-level carbon report fields
vsc_carbon[]	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for VSC links in projected-converter order

Field	JSON key / path	Export / import rule	Meaning / note
<code>vsc_carbon[]</code> . <code>converter_index</code> , <code>bus_ac</code> , <code>bus_dc</code> , <code>ac_to_dc</code> , <code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	nested keys on each VSC row	bidirectional; defaults 0, false , or 0.0	reports actual hybrid transfer direction plus converter loss carbon intensity
<code>dc_dc_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	converter-loss report for DCDC links in projected-converter order
<code>dc_dc_carbon[]</code> . <code>converter_index</code> , <code>bus_in</code> , <code>bus_out</code> , <code>input_to_output</code> , <code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	nested keys on each DCDC row	bidirectional; defaults 0, false , or 0.0	reports actual DC/DC transfer direction plus converter loss carbon intensity
<code>storage_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	per-storage carbon report covering both explicit discharge-side sources and traced charging-side sinks
<code>storage_carbon[]</code> . <code>storage_index</code> , <code>bus</code> , <code>is_dc</code> , <code>p_mw</code> , <code>soc</code> , <code>stored_energy_mwh</code> , <code>soc_carbon_intensity_tco2_mwh</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	nested keys on each storage row	bidirectional; defaults 0, false , or 0.0	p_mw keeps the bus-injection sign convention; charging rows additionally preserve a source-attribution map in JSON
<code>energy_router_carbon[]</code>	top-level array	bidirectional; absent array keeps the vector empty	per-router internal-loss report reconstructed from port transfer signs and magnitudes
<code>energy_router_carbon[]</code> . <code>router_index</code> , <code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code> , <code>active_input_ports</code> , <code>active_output_ports</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	nested keys on each router row	bidirectional; defaults 0 or 0.0	router loss is derived from actual port mismatch; JSON preserves the source-loss map
<code>generator_loss_allocation</code> entries	top-level object map keyed by source id	bidirectional; absent map keeps the allocation map empty	total MW of branch and converter losses allocated to each internal source id
<code>tracing_summary.*</code> , <code>matrix_summary.*</code>	nested summary objects	bidirectional; absent object keeps defaults	system-level emissions summary blocks for the tracing and matrix methods

Field	JSON key / path	Export / import rule	Meaning / note
tracing_summary. total_generation _emissions_tco2, total_load_e missions_tco2, total_loss_emiss ions_tco2, balanc e_error_tco2, ba lance_error_pct	same nested keys under both summary objects	bidirectional; default 0.0	emissions-balance totals and consistency metrics

LoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
load_index, bus, demand_mw, carbon _intensity_tco2_ mwh, total_emissi ons_tco2	corresponding columns	copied from each resu lt.load_carbon[i] row	flat workbook view of the per- load carbon result without the generator-contribution map

DCLoadCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
load_index, bus, demand_mw, carbon _intensity_tco2_ mwh, total_emissi ons_tco2	corresponding columns	copied from each resu lt.dc_load_carbon[i] row	flat workbook view of the per- DC-load carbon result without the source-contribution map

BusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
bus_index, carbon _intensity_tco2_ mwh	corresponding columns	copied from each resu lt.bus_carbon[i] row	flat workbook view of the per- bus carbon intensity

DCBusCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
bus_index, carbon _intensity_tco2_ mwh	corresponding columns	copied from each resu lt.dc_bus_carbon[i] row	flat workbook view of the per- DC-bus carbon intensity

BranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>branch_index</code> , <code>from_bus</code> , <code>to_bus</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.branch_carbon[i]</code> row	flat workbook view of the per-branch carbon-loss result without the generator-loss map

DCBranchCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>branch_index</code> , <code>from_bus</code> , <code>to_bus</code> , <code>loss_mw</code> , <code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dc_branch_carbon[i]</code> row	flat workbook view of the per-DC-branch carbon-loss result without the source-loss map

VSCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>converter_index</code> , <code>bus_ac</code> , <code>bus_dc</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	stable converter identifier and terminal buses
<code>ac_to_dc</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	actual solved transfer direction, not merely the input setpoint sign
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	carrier input, carrier output, and converter loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.vsc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

DCDCCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>converter_index</code> , <code>bus_in</code> , <code>bus_out</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	stable converter identifier and declared input/output buses
<code>input_to_output</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	actual solved transfer direction relative to the declared <code>bus_in/bus_out</code> pair
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	converter input, converter output, and DCDC loss in MW
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from each <code>result.dcdc_carbon[i]</code> row	loss-side carbon intensity and the corresponding converter-loss emissions

StorageCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>storage_index</code> , <code>bus</code>	corresponding columns	copied from each <code>result.storage_carbon[i]</code> row	stable storage identifier and connected bus
<code>is_dc</code>	corresponding column	exported as lowercase workbook string <code>true/false</code>	distinguishes AC storage from DC storage when indices overlap across subsystems
<code>p_mw</code>	corresponding column	copied from <code>result.storage_carbon[i].p_mw</code>	signed storage dispatch in bus-injection convention; positive means discharge, negative means charging sink
<code>soc</code> , <code>stored_energy_mwh</code>	corresponding columns	copied from the storage carbon result row	reported storage state snapshot used by the carbon model
<code>soc_carbon_intensity_tco2_mwh</code>	corresponding column	copied from the storage carbon result row	explicit carbon intensity of stored energy used when the unit discharges
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from the storage carbon result row	current-mode carbon intensity and emissions; charging uses traced incoming intensity, discharging uses SOC intensity

EnergyRouterCarbon

Field	Workbook sheet / column	Population rule	Meaning / note
<code>router_index</code>	corresponding column	copied from each <code>result.energy_router_carbon[i]</code> row	stable energy-router identifier
<code>input_power_mw</code> , <code>output_power_mw</code> , <code>loss_mw</code>	corresponding columns	copied from the router carbon result row	router-wide input, output, and internally allocated loss reconstructed from solved port transfers
<code>active_input_ports</code> , <code>active_output_ports</code>	corresponding columns	copied from the router carbon result row	number of non-zero sending and receiving ports participating in the router carbon graph
<code>carbon_intensity_tco2_mwh</code> , <code>total_emissions_tco2</code>	corresponding columns	copied from the router carbon result row	loss-side carbon intensity and the corresponding router-loss emissions

Summary in the carbon workbook

Field	Workbook sheet / cell block	Population rule	Meaning / note
<code>TracingMethod</code>	Summary[A1]	literal label written at row 1	heading for the tracing summary block
<code>tracing_summary.total_generation_emissions_tco2</code>	Summary[A2:B2]	key literal plus <code>result.tracing_summary.total_generation_emissions_tco2</code>	tracing-method generation emissions total
<code>tracing_summary.total_load_emissions_tco2</code>	Summary[A3:B3]	key literal plus <code>result.tracing_summary.total_load_emissions_tco2</code>	tracing-method load emissions total
<code>tracing_summary.total_loss_emissions_tco2</code>	Summary[A4:B4]	key literal plus <code>result.tracing_summary.total_loss_emissions_tco2</code>	tracing-method loss emissions total

Field	Workbook sheet / cell block	Population rule	Meaning / note
<code>tracing_summary.balance_error_tco2</code>	Summary [A5:B5]	key literal plus result <code>.tracing_summary.balance_error_tco2</code>	tracing-method absolute emissions-balance error
<code>tracing_summary.balance_error_pct</code>	Summary [A6:B6]	key literal plus result <code>.tracing_summary.balance_error_pct</code>	tracing-method relative emissions-balance error
<code>tracing_verified</code>	Summary [A7:B7]	key literal plus lower-case string <code>true/false</code>	tracing verification flag; note the lowercase boolean spelling differs from PF / OPF workbooks
<code>MatrixMethod</code>	Summary [A9]	literal label written at row 9	heading for the matrix-summary block
<code>matrix_summary.total_generation_emissions_tco2</code>	Summary [A10:B10]	key literal plus result <code>.matrix_summary.total_generation_emissions_tco2</code>	matrix-method generation emissions total
<code>matrix_summary.total_load_emissions_tco2</code>	Summary [A11:B11]	key literal plus result <code>.matrix_summary.total_load_emissions_tco2</code>	matrix-method load emissions total
<code>matrix_summary.total_loss_emissions_tco2</code>	Summary [A12:B12]	key literal plus result <code>.matrix_summary.total_loss_emissions_tco2</code>	matrix-method loss emissions total
<code>matrix_summary.balance_error_tco2</code>	Summary [A13:B13]	key literal plus result <code>.matrix_summary.balance_error_tco2</code>	matrix-method absolute emissions-balance error
<code>matrix_summary.balance_error_pct</code>	Summary [A14:B14]	key literal plus result <code>.matrix_summary.balance_error_pct</code>	matrix-method relative emissions-balance error
<code>matrix_solved</code>	Summary [A15:B15]	key literal plus lower-case string <code>true/false</code>	matrix-solver success flag
<code>matrix_residual</code>	Summary [A16:B16]	key literal plus result <code>.matrix_residual</code>	matrix-method residual metric

11.10 Current Output-Side Gaps

Area	Artifact	Status	Engineering consequence
<code>PowerFlowResult</code> JSON import	PF JSON	export only	PF result JSON can be written, but there is no matching public result-from-JSON reader.
PF profiling block	PF workbooks	not exported	the in-memory <code>PowerFlowResult.profiling</code> structure is not written to workbook output.
<code>ACOPFResult</code> enhanced dispatch vectors	OPF JSON / workbook	exported with packed semantics	Renewable, storage, DC/DC, and flexible-load dispatch vectors are exported as packed variable arrays with companion <code>ComponentRef</code> identity maps. JSON uses structured <code>renewable_dispatch</code> / <code>storage_dispatch</code> / <code>dc_dc_dispatch</code> / <code>flex_dispatch</code> objects; workbook sheets use <code>var_idx</code> , <code>source_type</code> , and <code>component_index</code> columns.
carbon contribution maps	carbon workbook	not exported	<code>generator_supply_mw</code> , <code>generator_loss_mw</code> , <code>source_supply_mw</code> , <code>source_loss_mw</code> , and <code>generator_loss_allocation</code> are available only in JSON.

Area	Artifact	Status	Engineering consequence
OPF_Converter	OPF workbook	conditional population	the sheet is only populated when VSC converters exist; otherwise the sheet may stay empty even though the workbook pre-creates it.
short-circuit outputs	public API / files	absent	this repository does not currently expose a short-circuit result API or any workbook / JSON serializer for it.
ONR outputs	public API / files	absent	this repository does not currently expose <code>solve_optimal_reconfiguration</code> or a workbook / JSON output contract for network reconfiguration.

12 Power-Flow Models

12.1 Scope and Architecture Overview

This chapter documents the steady-state hybrid AC/DC power-flow (PF) module in `src/power_flow/`. The legacy engine include path `include/hacdcpf/engine/solver/native/nle/newton_solver.hpp` is a declaration/compatibility entry point for the same Newton class; there is no corresponding `src/engine` implementation directory in this repository.

The facade in `include/hacdcpf/api/hacdcpf.hpp` exposes these public PF entry points:

Public API	Primary implementation / declaration	Scope
<code>solve_power_flow</code>	<code>src/power_flow/hybrid.cpp</code>	Hybrid AC/DC Newton through <code>powerflow::solve_hybrid</code> .
<code>solve_dc_power_flow</code>	<code>src/power_flow/dc.cpp</code> / <code>dc_solver.cpp</code>	DC-only Newton result wrapper.
<code>solve_power_flow_adaptive</code>	<code>src/power_flow/adaptive_solver.cpp</code>	Adaptive island-aware solve through <code>powerflow::solve_hybrid_adaptive</code> .
<code>solve_power_flow_islanded</code>	<code>src/power_flow/adaptive_solver.cpp</code>	Explicit islanded solve wrapper.
<code>solve_power_flow_distributed_slack</code>	<code>src/power_flow/distributed_slack_solver.cpp</code>	Simplified distributed-slack solve.
<code>solve_power_flow_distributed_slack_full</code>	<code>src/power_flow/distributed_slack_solver.cpp</code>	Full-Jacobian distributed-slack solve.
<code>create_solver_handle</code> , <code>reset_solver_handle</code> , <code>solve_handle</code> , <code>solve_dc_handle</code> , <code>destroy_solver_handle</code>	API handle facade	Reusable solver-handle lifecycle and solve calls.
<code>solve_power_flow_fast_dp</code>	<code>src/power_flow/fast_dp_solver.cpp</code>	Fast-decoupled AC-only PF.

Public API	Primary implementation / declaration	Scope
<code>solve_ac_dc_power_flow</code>	<code>src/power_flow/acc_linearized_pf.cpp</code>	Linearized AC-angle/DC result helper.

Table 120: Public power-flow API family.

The lower-level `hacdcpf::powerflow` namespace additionally declares `solve_ac`, `solve_dc`, `FDPFSolver::solve`, `AdaptiveSolver::solve`, `DistributedSlackSolver` simplified and full-Jacobian solves, `HelmSolver::solve`, `CpfSolver::solve`, and `HomotopyContinuationSolver::solve`. The header-only `three_phase.hpp` wrapper references an analysis-side `solve_three_phase_nr` symbol that is not part of the public `hacdcpf.hpp` facade.

The standard PF paths share canonical preprocessing through model projection, `solver_data.cpp`, and `admittance_builder.cpp`. The main Newton kernel is implemented in `src/power_flow/newton_solver.cpp`; its class is declared through the legacy engine header and re-exported from `include/hacdcpf/power_flow/newton_solver.hpp`.

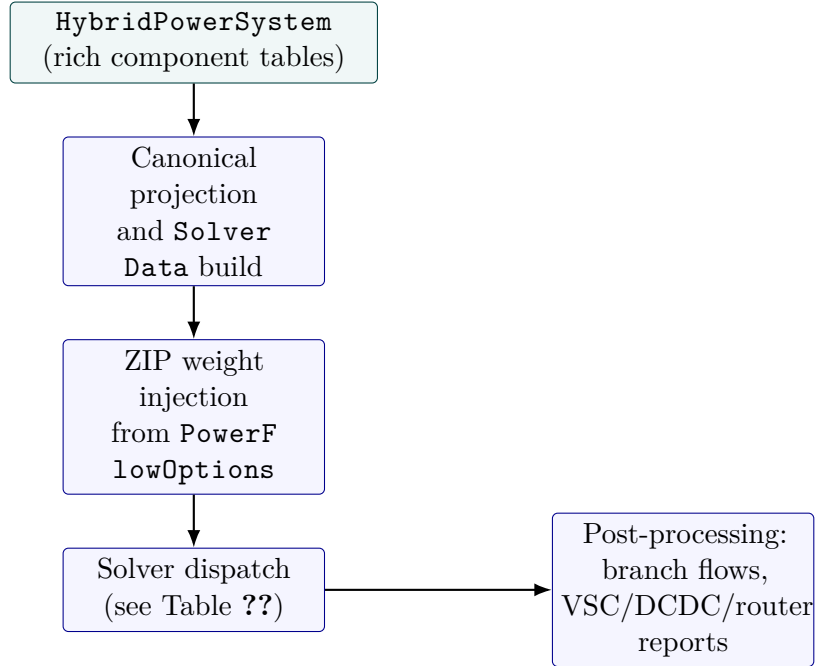


Figure 1: Shared execution pipeline for all power-flow API variants.

12.2 Mathematical Problem Form

The Newton kernel solves the hybrid residual system

$$F(x) = 0, \quad x = \begin{bmatrix} \theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \\ v_{\mathcal{N}_{PQ}} \\ v_{\mathcal{N}_{dc} \setminus \mathcal{N}_{sl}}^{\text{dc}} \end{bmatrix}, \quad (21)$$

where $\mathcal{N}_{sl}$ and $\mathcal{N}_{sl}^{\text{dc}}$ are the sets of AC and DC slack buses, and $\mathcal{N}_{PQ}$ is the set of PQ buses.

AC bus mismatch. For each non-slack AC bus i ,

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (22)$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}, \quad i \in \mathcal{N}_{PQ}, \quad (23)$$

where the calculated nodal injections are

$$P_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \cos \theta_{ij} + B_{ij} \sin \theta_{ij}), \quad (24)$$

$$Q_i^{\text{calc}} = \sum_{j \in \mathcal{N}_{ac}} v_i v_j (G_{ij} \sin \theta_{ij} - B_{ij} \cos \theta_{ij}), \quad (25)$$

with $\theta_{ij} = \theta_i - \theta_j$ and $(G_{ij} + jB_{ij})$ the (i, j) entry of the sparse bus admittance matrix Y_{bus} .

DC bus mismatch. For each non-DC-slack bus k ,

$$\Delta P_k^{\text{dc}} = P_k^{\text{dc,spec}} - P_k^{\text{dc,calc}}, \quad (26)$$

$$P_k^{\text{dc,calc}} = v_k^{\text{dc}} \left(G_{dc} v^{\text{dc}} \right)_k. \quad (27)$$

Specified-injection assembly.

$$P_i^{\text{spec}} = P_i^{\text{g,agg}} - P_i^{\text{load}}(v_i) + P_i^{\text{vsc}}, \quad (28)$$

$$Q_i^{\text{spec}} = Q_i^{\text{g,agg}} - Q_i^{\text{load}}(v_i) + Q_i^{\text{vsc}}, \quad (29)$$

where the aggregated generation vectors are

$$P_i^{\text{g,agg}} = P_i^{\text{gen}} + P_i^{\text{sg}} + P_i^{\text{ren}} + P_i^{\text{pv}} + P_i^{\text{stor}} + P_i^{\text{vpp}} + P_i^{\text{mg}} + P_i^{\text{ms}}, \quad (30)$$

$$Q_i^{\text{g,agg}} = Q_i^{\text{gen}} + Q_i^{\text{sg}} + Q_i^{\text{ren}} + Q_i^{\text{pv}} + Q_i^{\text{stor}} + Q_i^{\text{vpp}} + Q_i^{\text{ms}}. \quad (31)$$

Here P_i^{mg} is microgrid exchange at the PCC bus in `GridConnected` mode.

ZIP demand model. The runtime voltage-dependent load model is

$$P_i^{\text{load}}(v_i) = P_{i,0} (w_{P,0,i} + w_{P,1,i} v_i + w_{P,2,i} v_i^2), \quad (32)$$

$$Q_i^{\text{load}}(v_i) = Q_{i,0} (w_{Q,0,i} + w_{Q,1,i} v_i + w_{Q,2,i} v_i^2). \quad (33)$$

The ZIP weights are populated in one of three assembly cases: (1) explicit Load/charging-station tables present, (2) charging-station tables only, (3) neither table — global `zip_pw/zip_qw` from `PowerFlowOptions` are used.

12.3 VSC Converter Model

The converter model uses bus-injection-positive signs on both AC and DC sides. Three control modes are supported.

Loss model. With active transfer p (p.u.), AC-side proxy current $I^{\text{ac}} = |p|/v^{\text{dc,safe}}$, $v^{\text{dc,safe}} = \max(v^{\text{dc}}, 0.1)$:

$$P^{\text{loss,lin}} = (1 - \eta) |p|, \quad (34)$$

$$P^{\text{loss,cur}} = a + b I^{\text{ac}} + c (I^{\text{ac}})^2, \quad a = \frac{\text{loss_mw}}{S_{base}}, \quad b = \frac{\text{loss_pct}}{100}, \quad c = 1 - \eta. \quad (35)$$

PQ_MODE.

$$P^{\text{conv,ac}} = P^{\text{set}}, \quad Q^{\text{conv,ac}} = Q^{\text{set}}, \quad P^{\text{conv,dc}} = -(P^{\text{set}} + P^{\text{loss}}). \quad (36)$$

VDC_Q and VDC_VAC.

$$P^{\text{tr}} = k^{\text{vdc}} \left[(v^{\text{dc}})^2 - (v^{\text{dc,set}})^2 \right], \quad P^{\text{conv,ac}} = P^{\text{tr}} - P^{\text{loss}}, \quad P^{\text{conv,dc}} = -P^{\text{tr}}. \quad (37)$$

In VDC_VAC mode the Newton loop enforces $v^{\text{ac}} = v^{\text{ac,set}}$ for the converter AC bus by hard-clamping the trial AC voltage after updates. If `SolverData::enable_augmented_equations` is enabled, that clamp is skipped and the VDC/VAC bus is instead handled by the augmented-equation path in `newton_solver.cpp`.

12.4 DCDC Converter Model

The DCDC converter connects `bus_in` to `bus_out`. Power `p_ref_mw` is output-side referenced.

$$P^{\text{in}} = \begin{cases} P^{\text{ref}}/\eta, & P^{\text{ref}} \geq 0, \\ P^{\text{ref}} \cdot \eta, & P^{\text{ref}} < 0, \end{cases} \quad P^{\text{out}} = P^{\text{ref}} - r_{eq} \frac{(P^{\text{in}})^2}{(v^{\text{dc,in}})^2}. \quad (38)$$

12.5 Jacobian Structure

The linearized Newton system is

$$J \Delta x = F(x), \quad J = \begin{bmatrix} J_{\theta\theta} & J_{\theta V} & J_{\theta V^{\text{dc}}} \\ J_{V\theta} & J_{VV} & J_{VV^{\text{dc}}} \\ 0 & J_{V^{\text{dc}}V} & J_{V^{\text{dc}}V^{\text{dc}}} \end{bmatrix}. \quad (39)$$

The sparse Jacobian pattern is cached in a `PatternCache` keyed by matrix storage addresses and a monotone `build_id`, and is rebuilt only on topology changes.

AC diagonal entries (with ZIP load corrections).

$$\frac{\partial \Delta P_i}{\partial v_i} = \frac{P_i^{\text{calc}}}{v_i} + G_{ii} v_i - P_{i,0} (w_{P,1,i} + 2w_{P,2,i} v_i), \quad (40)$$

$$\frac{\partial \Delta Q_i}{\partial v_i} = \frac{Q_i^{\text{calc}}}{v_i} - B_{ii} v_i - Q_{i,0} (w_{Q,1,i} + 2w_{Q,2,i} v_i). \quad (41)$$

In the code, v_i is floored at `min_vm_pu` (configurable via `RobustNonlinearOptions`) to prevent zero-division.

DC entries.

$$\frac{\partial P_k^{\text{dc,calc}}}{\partial v_m^{\text{dc}}} = \begin{cases} (G_{dc} v^{\text{dc}})_k + G_{kk}^{\text{dc}} v_k^{\text{dc}}, & m = k, \\ G_{km}^{\text{dc}} v_k^{\text{dc}}, & m \neq k. \end{cases} \quad (42)$$

AC/DC coupling. When `enable_coupled_jacobian` is set, the converter Jacobian blocks $\partial P^{\text{vsc}}/\partial v^{\text{dc}}$, $\partial Q^{\text{vsc}}/\partial v^{\text{dc}}$, and $\partial P^{\text{dc}}/\partial v^{\text{ac}}$ have pattern slots. The current `converter_dc_jacobian_vm_ac` implementation returns zero because the old AC-side converter resistance term was removed, so the $\partial P^{\text{dc}}/\partial v^{\text{ac}}$ block is structurally present but numerically zero today. The coupling entries are pre-registered as `CouplingEntry` records in `JacobianPattern` and updated in-place each iteration without re-analyzing the pattern.

12.6 Hybrid Newton Algorithm

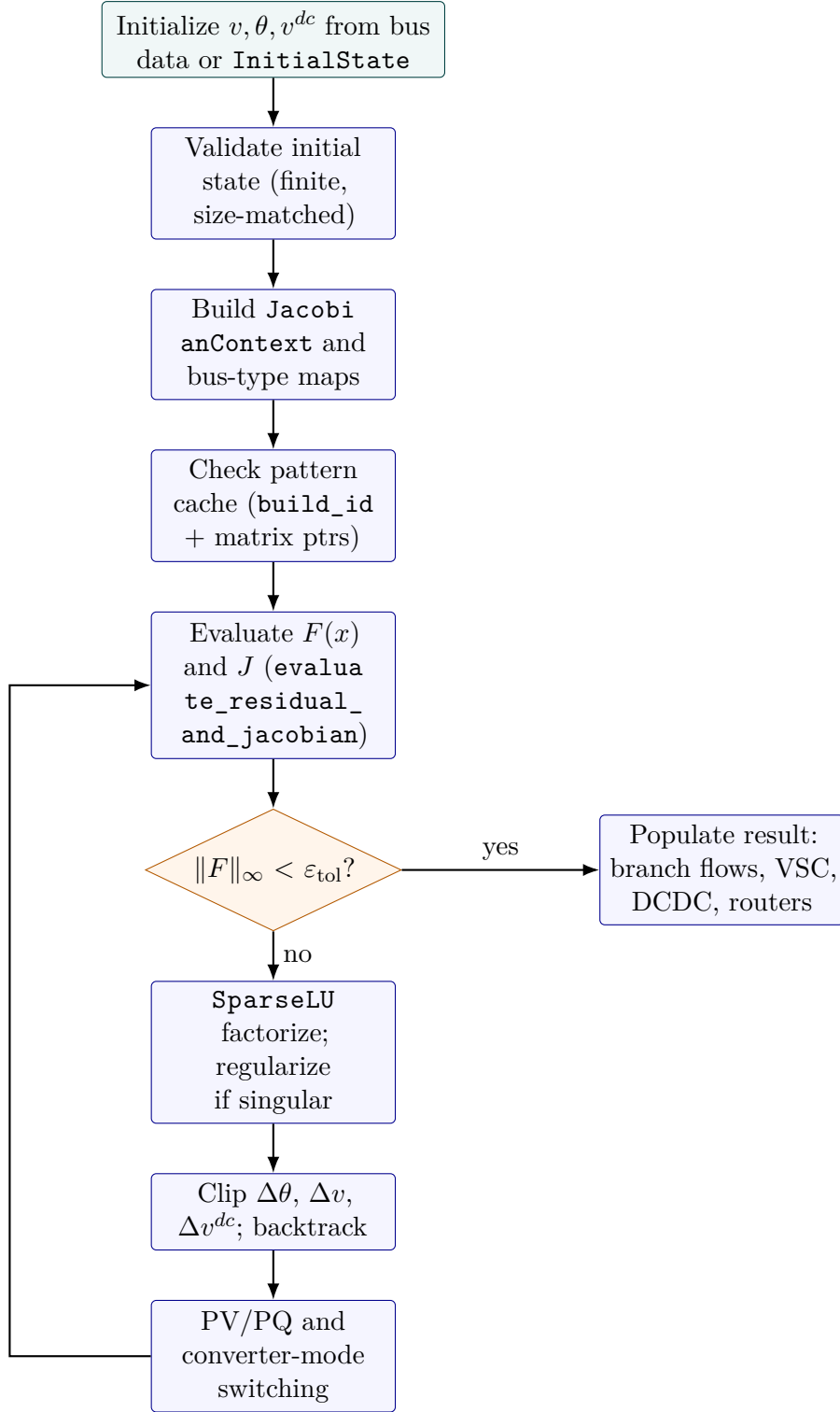


Figure 2: Hybrid AC/DC Newton power-flow algorithm (`NewtonSolver::solve`).

Algorithm PF-1: Hybrid Newton PF

- 1: **procedure** `NEWTONSOLVER::SOLVE`(*data, opt, init*)
- 2: Validate *init*: check size, NaN, Inf; **throw** `invalid_argument` on failure
- 3: $x \leftarrow$ flat start or *init* values; build `JacobianContext`

```

4:    $x_{ctx}.\text{min\_vm\_pu} \leftarrow \text{opt.robust\_nonlinear.min\_vm\_pu}$ 
5:   for  $k = 1, \dots, \text{opt.max\_iter}$  do
6:      $F, J \leftarrow \text{evaluate\_residual\_and\_jacobian}(x, ctx, pattern)$ 
7:     if  $\|F\|_\infty < \text{opt.tol}$  then break
8:     end if
9:      $\text{SparseLU}(J) \Delta x = F$ ; if singular then regularize and retry
10:    Clip  $\Delta x$ :  $|\Delta \theta| \leq \Delta \theta_{\max}, |\Delta v| \leq \Delta v_{\max}$ 
11:    Backtracking line search; accept best step
12:    PV/PQ hysteresis switching; converter mode switching
13:  end for
14:  if not converged then return a result with converged=false, final iteration count,
    residual, and any available state vectors
15:  Post-process branch, VSC, DCDC, router flows; return result
16: end procedure

```

Robust nonlinear pipeline. The solver implements a five-phase robustification pipeline controlled by `RobustNonlinearOptions`:

1. **Phase 1 – Scaling and diagnostics:** residual-equation scaling, variable scaling, and Jacobian row/column equilibration; condition-number proxy monitor.
2. **Phase 2 – Nonmonotone line search and smooth NCP:** Grippo–Lampariello–Lucidi nonmonotone Armijo search; optional μ -perturbed Fischer–Burmeister complementarity for PV/PQ switching.
3. **Phase 3 – LM trust-region fallback:** Levenberg–Marquardt step $(J^\top J + \lambda I) \Delta x = -J^\top F$ when Newton line search fails; SER-based pseudo-transient continuation.
4. **Phase 4 – Homotopy continuation:** ramp loads and converter setpoints from flat start to target in adaptive λ steps.
5. **Phase 5 – Krylov fallback:** Newton–GMRES with Schur-complement AC/DC block preconditioner (opt-in; not active by default).

12.7 Fast-Decoupled Power Flow (FDPF)

The FDPF solver builds two constant susceptance matrices from Y_{bus} :

$$B' = -\Im \left\{ Y_{bus}^{(R=0, b=0, \text{tap}=1)} \right\}, \quad (43)$$

$$B'' = -\Im \left\{ Y_{bus}^{(\text{full shunts and charging, zero phase shift})} \right\}, \quad (44)$$

and then alternates P- θ and Q-V decoupled solves:

$$B' \Delta \theta = \Delta P / v, \quad (45)$$

$$B'' \Delta v = \Delta Q / v. \quad (46)$$

Both B' and B'' are factorized once via `Eigen::SparseLU`; only the right-hand sides change across iterations. The solver is AC-only (no DC or converter coupling).

Algorithm PF-2: Fast-Decoupled PF

- 1: **procedure** `FDPFSOLVER::SOLVE`($data, opt$)
- 2: Build reduced $B'_{\mathcal{N} \setminus \{sl\}}, B''_{\mathcal{N}_{PQ}}$; factorize
- 3: **if** factorization failed **then** return non-converged result

```

4:   end if
5:   for  $k = 1, \dots, \text{opt.fdpf\_max\_iter}$  do
6:       Convert/use dense ybus_dense; compute  $\Delta P/v$ ,  $\Delta Q/v$  with all-pairs loops
7:       if  $\|\Delta P/v\|_\infty < \varepsilon$  and  $\|\Delta Q/v\|_\infty < \varepsilon$  then break
8:       end if
9:        $\Delta \theta \leftarrow (B')^{-1}(\Delta P/v)$ ; update  $\theta$ 
10:       $\Delta v \leftarrow (B'')^{-1}(\Delta Q/v)$ ; update  $v$ 
11:   end for
12: end procedure

```

12.8 DC-Only Power Flow

The DC solver iterates on non-slack DC voltages using dense LU on the small reduced Jacobian:

$$J_{dc} \Delta v^{\text{dc}} = \Delta P^{\text{dc}}, \quad (47)$$

$$J_{dc,kl} = \begin{cases} (G_{dc} v^{\text{dc}})_k + G_{kk}^{\text{dc}} v_k^{\text{dc}}, & l = k, \\ G_{kl}^{\text{dc}} v_k^{\text{dc}}, & l \neq k. \end{cases} \quad (48)$$

DC bus injections are assembled from **DCLoad**, DC storage, DC static generators, DC PV arrays, VSC DC-side injections, DCDC transfers, and DC **EnergyRouter** ports. A backtracking line search over $\alpha \in \{1, \frac{1}{2}, \dots\}$ is applied with voltages floored at **kMinVdc** to prevent collapse.

12.9 Adaptive Islanded Solver

The adaptive solver handles multi-island AC/DC topologies that a single global solve cannot address:

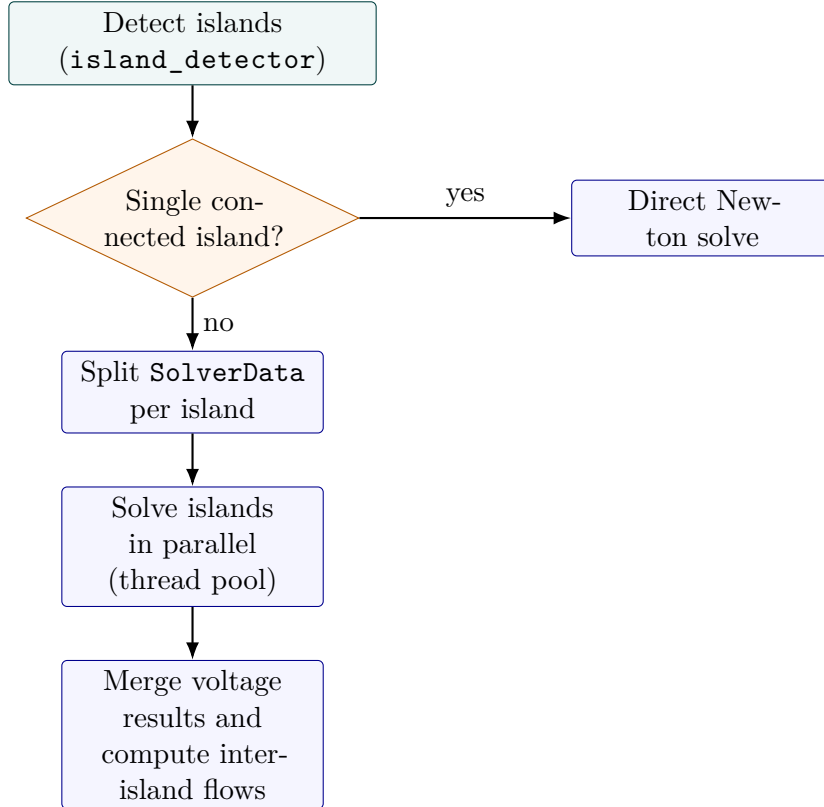


Figure 3: Adaptive islanded power-flow workflow.

12.10 HELM Solver

The HELM (Holomorphic Embedding Load-flow Method) solver constructs a power series in a complex embedding parameter $s \in \mathbb{C}$:

$$V_i(s) = \sum_{n=0}^N a_n^{(i)} s^n. \quad (49)$$

The recursion for order n coefficients follows from the holomorphic embedding of the network equations. Convergence is tested via Padé approximants $[M/M]$ evaluated at $s = 1$:

$$V_i(1) \approx \frac{P_M(1)}{Q_M(1)}. \quad (50)$$

In the current source, HELM is a standalone `HelmSolver::solve` path. `HelmWarmStart::DCPF` and `HelmWarmStart::GaussSeidel` are options consumed inside `HelmSolver::solve`; the hybrid Newton and adaptive solver paths do not call HELM as an automatic fallback.

12.11 Error Handling and Diagnostics

The public result structs do not contain a `SolverFailureReason` enum. `PowerFlowResult` exposes `converged`, `iterations`, `residual`, solved state vectors, flow/transfer vectors, and profiling metadata. `DCPowerFlowResult`, `AdaptiveSolveResult`, `IslandedSolveResult`, and `DistributedSlackResult` use their own compact status fields and nested results.

The solver **throws** `std::invalid_argument` for structural API violations (e.g., size mismatch, NaN in initial state) and **returns a non-converged result** for numerical failures. This two-tier policy separates programming errors from expected numerical difficulties.

12.12 Maintainability Roadmap: Code-Duplication Inventory

All six items D1–D6 from the original inventory were resolved across two refactoring passes (*code-dedup-D1-D5* and *code-dedup-D3-D6*).

D1 — `inf_norm` (DONE). The anonymous-namespace duplicate removed from `jacobian_builder.cpp` and `dc_solver.cpp`. The canonical definition now lives in `include/hacdcpf/power_flow/pf_utils.hpp` as an inline function. Test files `test_powerflow_performance.cpp` updated to use `hacdcpf::powerflow::inf_norm` via a `using` declaration.

D2 — DC injection assembly (DONE). The 80-line DC injection loop extracted into `assemble_dc_injections(data,vm,va,vdc,pdc_spec)`, declared in a new header `include/hacdcpf/power_flow/pf_injection_assembly.hpp` and implemented in `src/power_flow/pf_injection_assembly.cpp`. Both `dc_solver.cpp` and `residual_evaluator.cpp` now call this single canonical function.

D3 — Duplicated `B'` matrix construction (DONE). The anonymous-namespace functions `build_bp` and `build_bpp` were removed from `fdpf_solver.cpp` and replaced with calls to a new utility `build_susceptance_matrix(data,flags)` implemented in `admittance_builder.cpp`. A `BpBuildFlags` struct (declared in `admittance_builder.hpp`) controls all assembly variants: zero resistance, zero line charging, unit tap, zero phase shift, and optional bus shunt susceptance. Convenience factories `make_bp_flags(min_x_pu)` and `make_bpp_flags()` cover the two FDPF use cases.

D4 — DC initial-state warm-start (DONE). The size-guarded copy of `init->vdc` with NaN/Inf validation extracted into `load_dc_initial_state(init,dc_buses,vdc)` in `pf_utils.hpp`. Applied to both `dc_solver.cpp` and `newton_solver.cpp`.

D5 — Bus-type classification loop (DONE). The manual AC bus classification into SLACK/PV/PQ sets extracted into `classify_ac_buses(data)` returning an `AcBusSets` struct (all in `pf_utils.hpp`). Applied to `fdpf_solver.cpp`, `residual_evaluator.cpp`, and the helper in `test_unified_pf_upgrade.cpp`.

D6 — jacobian_builder.cpp file split (DONE). The monolithic `jacobian_builder.cpp` was split into three focused compilation units:

- `jacobian_pattern.cpp` — `build_jacobian_pattern` and its anonymous-namespace helpers (`rc_key`, `add_pattern_position`, `build_entry_to_nz`, `lookup_nz`);
- `ac_kernel.cpp` — `evaluate_ac_kernel_serial`, `evaluate_ac_kernel_parallel`, and `effective_ac_threads`;
- `jacobian_builder.cpp` (retained) — `build_power_spec`, `evaluate_residual_impl`, `evaluate_residual_and_jacobian`, and `evaluate_residual_only`.

The cross-file linkage between `jacobian_builder.cpp` and `ac_kernel.cpp` is provided by an internal header `include/hacdcpf/power_flow/ac_kernel_impl.hpp`.

12.13 Numerical Performance (Release Mode)

Benchmarks were run on 84 MATPOWER cases (`build_rel`, Apple M4, `-O3-DNDEBUG`, single thread) using `build_rel/helm_powerflow_benchmark../data`. Four paths are compared: NR = hybrid Newton, FD = fast-decoupled, HE = HELM standalone, HE+DC = HELM with DC warm start.

Case	<i>N</i>	Branches	NR ms	FD ms	HE ms	HE+DC ms
case14	14	20	0.0	0.3	0.1	0.1
case57	57	80	0.1	2.9	0.6	0.6
case118	118	186	0.4	11.5	1.8	1.9
case300	300	411	0.9	59.5	div	div
case1354pegase	1354	1991	15.2	skip	div	div
case1888rte	1888	2531	10.8	skip	135.8	136.5
case2383wp	2383	2896	3.8	skip	117.6	109.3
case2736sp	2736	3504	5.6	skip	75.2	76.6
case3012wp	3012	3572	35.0	skip	124.3	126.9
case6468rte	6468	9000	31.7	skip	297.7	296.2
case8387pegase	8387	14561	46.7	skip	1776.5	1786.7
case13659pegase	13659	20467	95.9	skip	skip	skip
	10000	12706	21.8	skip	4327.9	4389.7
case_ACTIVSg10k						
	25000	32230	64.3	skip	skip	skip
case_ACTIVSg25k						

Case	N	Branches	NR ms	FD ms	HE ms	HE+DC ms
case_SyntheticUSA	82000	104121	1745.2	skip	skip	skip

Table 121: Power-flow solver timing (release mode, Apple M4, single thread, post D1–D5 refactor). “div” = diverged, “skip” = case excluded from path at this size.

Key observations:

- The hybrid Newton path converges on the 82000-bus USA synthetic system in 1.75s with 17 iterations after the D1–D5 refactoring pass (no regression vs. pre-refactoring baseline).
- Newton solve time scales approximately as $O(N^{1.2})$ in practice for typical sparse transmission systems.
- FDPF is 10–200× slower than Newton on small-to-medium cases due to the current $O(N^2)$ dense Ybus iteration in the power-injection loop; replacing it with a sparse row iteration remains an open performance item.
- HELM converges on most well-conditioned cases under 2000 buses but diverges on stressed systems (case300, case1354pegase); it remains useful as a warm-start provider for Newton on difficult cases.
- The D1–D6 refactoring passes introduced no performance regression: Newton solve times match the pre-refactoring baseline on all benchmark cases.

13 Advanced and Fallback Power-Flow Solvers

This chapter documents the remaining power-flow solver families declared in `include/hacdcpf/power_flow/`: the Continuation Power Flow (`CpfSolver`), the Homotopy Continuation Solver, the Distributed Slack Solver mathematical model, and the Newton–Krylov GMRES fallback. It also provides the complete `RobustNonlinearOptions` and `PowerFlowOptions` reference tables.

13.1 Continuation Power Flow (CPF) / Voltage Stability

The `CpfSolver` traces the P–V curve of a parameterized power system from the base operating point ($\lambda = 0$) toward the voltage-collapse nose point. The implementation lives in `src/power_flow/homotopy_continuation.cpp` (the CPF corrector) and `include/hacdcpf/power_flow/voltage_stability.hpp`. It is exposed at the lower-level `hacdcpf::powerflow` namespace and is not part of the public `hacdcpf.hpp` facade.

13.1.1 Parameterized Power System

The loading direction is described by `CpfDirection`:

$$P_{L,i}(\lambda) = P_{L,i}^0 + \lambda \Delta P_{L,i}, \quad (51)$$

$$Q_{L,i}(\lambda) = Q_{L,i}^0 + \lambda \Delta Q_{L,i}, \quad (52)$$

$$P_{G,i}(\lambda) = P_{G,i}^0 + \lambda \Delta P_{G,i}, \quad (\text{non-slack only}). \quad (53)$$

Here $\lambda = 0$ is the base case; $\lambda = 1$ doubles all loads when using the proportional direction. Two factory methods build standard directions:

- `CpfDirection::proportional(data)` — all buses scale in proportion to their base-case power.
- `CpfDirection::at_buses(data, bus_indices)` — uniform 1 MW/Mvar per bus in the specified list.

13.1.2 Predictor–Corrector Continuation

The predictor step uses a secant (or constant) approximation:

$$x_{\lambda+\Delta\lambda} \approx x_{\lambda} + \Delta\lambda \frac{x_{\lambda} - x_{\lambda-\Delta\lambda}}{\Delta\lambda_{\text{prev}}}. \quad (54)$$

The corrector step solves the parameterized Newton system at the predicted point by running the hybrid Newton PF with the modified injection vector and convergence tolerance `corrector_tol`.

Step adaptation:

$$\Delta\lambda_{\text{new}} = \begin{cases} \min(\text{step_grow} \cdot \Delta\lambda, \text{step_max}), & \text{corrector converged,} \\ \text{step_shrink} \cdot \Delta\lambda, & \text{corrector failed.} \end{cases} \quad (55)$$

If $\Delta\lambda < \text{step_min}$, the solver reports the nose point and stops.

13.1.3 CpfOptions Reference

Field	Default	Meaning
<code>lambda_max</code>	5.0	Maximum loading factor λ to trace.
<code>step_init</code>	0.05	Initial continuation step $\Delta\lambda$.
<code>step_min</code>	1e-5	Minimum step size; stop and declare nose point reached.
<code>step_max</code>	0.20	Maximum step size (prevents skipping over the nose).
<code>step_grow</code>	2.0	Growth factor applied after a successful corrector solve.
<code>step_shrink</code>	0.5	Shrink factor applied after a corrector failure.
<code>corrector_max_iter</code>	50	Max Newton iterations in each corrector call.
<code>corrector_tol</code>	1e-8	Convergence tolerance for the corrector (p.u.).
<code>monitor_bus</code>	-1	Bus index to plot on the P–V curve; -1 = auto-select max demand.
<code>trace_all_buses</code>	false	Store full <code>vm/va</code> vectors at every CPF step.
<code>max_steps</code>	2000	Safety cap on total predictor–corrector steps.
<code>vm_min_pu</code>	0.25	Voltage floor; stop if any bus falls below this (p.u.).
<code>use_secant_predictor</code>	true	Use secant (two-point) predictor; false = constant predictor.

13.1.4 CPF Result Structure

Each tracing step produces a `CpfPoint`:

- `lambda` – continuation parameter value at this step;
- `vm_monitor` – monitor-bus voltage magnitude (p.u.).

The top-level result includes the full trace of `CpfPoint` records, the final converged solution, estimated nose-point λ^* and minimum voltage, and the convergence status.

13.2 Homotopy Continuation Solver

The `HomotopyContinuationSolver` solves the hybrid power flow by gradually ramping “difficult” injections from a flat-start base to the target values. It is invoked as a last-resort fallback when the direct Newton solve fails (Phase 4 of the robust pipeline).

13.2.1 Homotopy Path

The homotopy path scales all active/reactive loads and converter setpoints:

$$P_L(\lambda) = \lambda P_L^{\text{target}}, \quad Q_L(\lambda) = \lambda Q_L^{\text{target}}, \quad (56)$$

$$P_{\text{vsc}}(\lambda) = \lambda P_{\text{vsc}}^{\text{target}}, \quad P_{\text{dcdc}}(\lambda) = \lambda P_{\text{dcdc}}^{\text{target}}. \quad (57)$$

At $\lambda = 0$ the system is trivially solved from the flat start. At $\lambda = 1$ the full target system is solved.

13.2.2 Step Adaptation

$$\text{Success: } \Delta\lambda \leftarrow \min(2\Delta\lambda, \Delta\lambda_{\max}), \quad (58)$$

$$\text{Failure: } \Delta\lambda \leftarrow \frac{\Delta\lambda}{2}; \quad \text{fail if } \Delta\lambda < \Delta\lambda_{\min}. \quad (59)$$

The current implementation records accepted/rejected steps in a `HomotopyState` diagnostic struct and returns both the final `PowerFlowResult` and the state for instrumentation.

13.3 Distributed Slack Solver

The distributed-slack solver distributes the active-power mismatch across a set of participating buses according to participation factors instead of assigning it all to a single slack bus. Two variants are exposed:

- `solve_power_flow_distributed_slack` – simplified form (one extra slack sweep);
- `solve_power_flow_distributed_slack_full` – full-Jacobian distributed-slack, as described below.

13.3.1 Problem Formulation

Let $\alpha_g \geq 0$ be participation factors (normalized to sum to 1) and let P_s be the total system mismatch. Each participating generator g receives an additional dispatch increment:

$$\delta P_g = \alpha_g P_s, \quad (60)$$

subject to individual upper bounds `max_participation_pg` when provided. The reference bus (`reference_bus`) provides the angle reference; it need not be the same as the swing generator.

13.3.2 Convergence Criterion

In the full-Jacobian variant, the distributed-slack equations are appended to the standard Newton residual and solved simultaneously. Convergence is judged on the extended infinity-norm $\|\Delta F\|_\infty < \varepsilon$.

13.3.3 Result Structure

`DistributedSlackResult` extends the standard bus-voltage vectors with:

- `distributed_slack_p` – map from participating bus index to final slack-share allocation (MW);
- `hit_limits` – list of buses that reached `max_participation_p` and were clamped.

13.4 Newton–Krylov GMRES Fallback (Phase 5)

Phase 5 of the robust pipeline replaces the direct-factorization linear solve with a GMRES iterative solver inside the Newton loop. The implementation lives in `src/power_flow/newton_krylov.cpp` and is declared in `include/hacdcpf/power_flow/newton_krylov.hpp`. It is opt-in and not active by default (`RobustNonlinearOptions::enable_krylov_fallback`).

At each Newton iteration the augmented linear system

$$(J + \alpha M^{-1}) \Delta x = -F \quad (61)$$

is solved via GMRES with a Schur-complement AC/DC block preconditioner M :

$$M = \begin{bmatrix} J_{ac} & 0 \\ 0 & J_{dc} \end{bmatrix}, \quad (62)$$

where J_{ac} is the AC subsystem Jacobian and J_{dc} is the DC subsystem Jacobian. The Krylov fallback is most effective when the Jacobian is ill- conditioned due to strong AC/DC coupling, but adds per-iteration overhead compared to the direct SparseLU path.

13.5 Complete RobustNonlinearOptions Reference

Field	Default	Description
Phase 1: Scaling and diagnostics		
<code>enable_residual_scaling</code>	<code>true</code>	Scale residual equations by $\max F_i^{\text{spec}} $.
<code>enable_variable_scaling</code>	<code>true</code>	Scale Newton variables (θ, V, V^{dc}) to $O(1)$.
<code>enable_jacobian_row_col_eq_uilibration</code>	<code>true</code>	Apply $D_F J D_x^{-1}$ row/column equilibration.
<code>enable_condition_monitor</code>	<code>true</code>	Compute ℓ_1 row/col proxy for condition number each iteration.
<code>min_scale</code>	1e-6	Floor for any scale factor.
<code>max_scale</code>	1e6	Ceiling for any scale factor.
<code>min_vm_pu</code>	1e-8	Voltage magnitude floor (pu) in Jacobian $\partial P / \partial V$.
<code>min_branch_x_pu</code>	1e-20	Min branch reactance; below this, branch excluded from FDPF B' .

Field	Default	Description
<code>bad_condition_threshold</code>	1e10	Proxy condition number above which Jacobian is flagged “bad”.
<code>tiny_pivot_threshold</code>	1e-12	Pivot magnitude threshold for informational logging.
Phase 2: Nonmonotone line search		
<code>enable_nonmonotone_linesearch</code>	<code>true</code>	Use GLL nonmonotone Armijo search with sliding window.
<code>nonmonotone_window</code>	5	Sliding window M for nonmonotone reference merit.
<code>armijo_c</code>	1e-4	Armijo sufficient-decrease constant $c_1 \in (0, 1)$.
<code>line_search_beta</code>	0.5	Step-length reduction factor β per backtrack trial.
<code>max_line_search_trials</code>	12	Maximum backtrack trials before step rejection.
Phase 2: Smooth Fischer–Burmeister NCP (off by default)		
<code>enable_smooth_ncp</code>	<code>false</code>	Use μ -perturbed smooth-FB for PV/PQ complementarity.
<code>ncp_mu0</code>	1e-2	Initial smoothing parameter μ_0 .
<code>ncp_mu_min</code>	1e-12	Minimum smoothing parameter (limit of continuation).
<code>ncp_mu_factor</code>	0.1	Fine-phase annealing factor (near-convergence).
<code>ncp_mu_factor_coarse</code>	0.5	Coarse-phase annealing factor (large residual).
<code>ncp_mu_phase_transition</code>	0.1	Fraction of initial residual to switch to fine annealing.
<code>enable_activity_hysteresis</code>	<code>true</code>	Hold PV/PQ active-set for <code>min_active_set_hold_iters</code> iterations.
<code>min_active_set_hold_iters</code>	3	Minimum iterations between consecutive PV/PQ status changes.
<code>q_limit_hysteresis</code>	1e-4	Hysteresis band on Q-limit proximity (pu) before switching.
Phase 3: LM trust-region fallback		
<code>enable_lm_trust_region_fallback</code>	<code>true</code>	Compute LM step $(J^\top J + \lambda I)\Delta x = -J^\top F$ on Newton failure.
<code>lm_lambda0</code>	1e-4	Initial LM damping parameter λ_0 .
<code>lm_lambda_min</code>	1e-12	Minimum LM damping.
<code>lm_lambda_max</code>	1e8	Maximum LM damping.
<code>trust_region_radius0</code>	1.0	Initial trust-region radius.
<code>trust_region_radius_min</code>	1e-6	Minimum radius; below this the LM fallback is abandoned.
<code>trust_region_radius_max</code>	100.0	Maximum trust-region radius.

Field	Default	Description
Phase 3: PTC-SER pseudo-transient continuation		
<code>enable_ptc_ser</code>	<code>true</code>	Use Self-Equalising Residual (SER) exponent for adaptive PTC.
<code>ptc_dt0</code>	0.1	Initial pseudo-timestep δt_0 .
<code>ptc_dt_min</code>	1e-4	Minimum pseudo-timestep (large damping).
<code>ptc_dt_max</code>	1e6	Maximum pseudo-timestep (approaches pure Newton).
Phase 4: Homotopy continuation (see Section ??)		
<code>enable_homotopy_fallback</code>	configurable	Trigger <code>HomotopyContinuationSolver</code> when Phase 3 fails.
Phase 5: Krylov fallback (off by default)		
<code>enable_krylov_fallback</code>	<code>false</code>	Replace SparseLU with Newton-GMRES + Schur-complement preconditioner.

13.6 PowerFlowOptions Complete Reference

Field	Default	Description
<code>max_iter</code>	50	Maximum Newton iterations.
<code>tol</code>	1e-8	Convergence tolerance ε (p.u., ℓ^∞).
<code>ac_eval_threads</code>	1	Threads for AC kernel; 0 = hardware auto-detect.
<code>enable_solver_profiling</code>	<code>false</code>	Populate <code>PowerFlowResult::profiling</code> counters and timings.
<code>max_line_search_steps</code>	10	Max backtrack trials in standard monotone search.
<code>max_regularization_steps</code>	8	Max Jacobian regularization attempts.
<code>regularization_lambda0</code>	1e-6	Initial regularization shift λ_{reg} .
<code>regularization_growth</code>	10.0	Growth factor applied to λ_{reg} each retry.
<code>max_delta_va_rad</code>	1.50	Max step size clamp for $\Delta\theta$ (rad).
<code>max_delta_vm_pu</code>	0.50	Max step size clamp for ΔV (pu).
<code>max_delta_vdc_pu</code>	0.50	Max step size clamp for ΔV^{dc} (pu).
<code>pv_q_hysteresis_pu</code>	0.01	Q-limit proximity band for PV→PQ switching (pu).
<code>pv_recover_vm_tol_pu</code>	0.01	Voltage recovery tolerance for PQ→PV switching (pu).

Field	Default	Description
converter_vdc_switch_high_pu	0.03	DC voltage deviation threshold (high) for mode switching.
converter_vdc_switch_low_pu	0.01	DC voltage deviation threshold (low) for mode switching.
mode_hysteresis_iters	2	Minimum iterations between converter mode switches.
enable_pv_pq_conversion	true	Enable PV/PQ generator switching.
enable_auto_swing_selection	true	Auto-assign swing bus if none is specified.
enable_converter_mode_switching	true	Enable VSC control-mode switching during Newton.
verbose	false	Print iteration diagnostics to std-out.
loss_model	Linear	VSC loss model; Linear or CurrentDependent .
zip_pw[3]	{1,0,0}	ZIP active-power weights [const-P, const-I, const-Z].
zip_qw[3]	{1,0,0}	ZIP reactive-power weights [const-P, const-I, const-Z].
fdpf_max_iter	1000	Maximum FDPF P/Q sweep iterations.
enable_coupled_jacobian	false	Include cross-coupling Jacobian blocks ($\partial P^{vsc}/\partial V^{dc}$ etc.).
enable_augmented_equations	false	Use augmented-equation path for VDC/VAC bus setpoints.
enable_semi_smooth_newton	false	Use NCP-based PV/PQ switching.
globalization	LineSearch	LineSearch , TrustRegion , or PseudoTransient .
trust_region_radius0	1.0	Initial trust-region radius (used when TrustRegion is selected).
trust_region_max	10.0	Maximum trust-region radius.
ptc_delta0	1.0	Initial PTC pseudo-timestep.
ptc_growth	2.0	PTC pseudo-timestep growth factor.
initial_state	empty	Optional warm-start InitialState (vm, va, vdc).
robust_nonlinear	(defaults above)	RobustNonlinearOptions substruct.

14 Optimal Power Flow Models

14.1 Problem Form and Runtime Scope

This chapter covers the OPF family built on top of canonical AC/DC network projection. The implemented runtime problem classes are:

- nonlinear AC/DC OPF for the native primal-dual and parity-IPM routes,
- DC OPF with a true-QP model and LP fallback backends,
- reactive-power optimization over discrete taps and shunts by outer-approximation / coordinate-search heuristics.

The runtime solver family is selected from option flags and DC-network presence.

14.2 Code Map

The current implementation is concentrated in:

- `src/optimal_power_flow/ac_opf.cpp` for top-level AC OPF routing and native reduced-space solve orchestration,
- `src/optimal_power_flow/parity_formulation.cpp` and `src/optimal_power_flow/parity_ipm.cpp` for the parity full-space NLP route,
- `src/optimal_power_flow/dc_opf.cpp` for DC OPF model build and backend selection,
- `src/optimal_power_flow/reactive_power_opt.cpp` for the four-phase outer-approximation / coordinate-search workflow,
- `include/hacdcpf/engine` forwarding headers for LCQP, dual simplex, HiGHS, Gurobi, and related engine facilities supplied by the `mipsolvers` dependency.

There is no `src/engine` directory in this repository.

14.3 Pseudocode Map

The relevant code-aligned workflows are:

- Algorithms A8–A10 in Section ?? for AC OPF routing, parity-NLP construction, and parity-IPM solve,
- Algorithm A11 for DC OPF,
- Algorithm A12 for reactive-power optimization,
- Section ?? for the forwarding engine surface used by DC OPF and RPO result statistics.

14.4 Current Status

The AC OPF family has multiple implemented routes, but backend coverage differs by path. Parity and native nonlinear routes are the main AC/DC engines; DC OPF builds a specialized QP/LP model; RPO currently runs an OA/coordinate-search workflow and reuses branch-and-cut-style statistics fields for counters.

14.5 Top-Level Solver Routing

The public `solve_ac_opf` path selects among three solver families. A key implementation rule is automatic DC routing: if the case has DC buses or VSC converters and `enable_primal_dual=false`, the options are internally overridden to `enable_primal_dual=true, use_parity_ipm=true`.

Path	Trigger	Current behavior
Fast dispatch + PF fallback (AC-only)	<code>enable_primal_dual=f</code> <code>alse</code> and no forced DC parity routing	Drops DC and converter subsystems, solves economic dispatch, then runs AC PF projection.
Native primal-dual OPF	<code>enable_primal_dual=true</code> , <code>use_parity_ipm=f</code> <code>alse</code>	Uses a custom sparse KKT loop with AC equations, DC-bus equations, and converter power-balance equations.
Parity primal-dual IPM	<code>enable_primal_dual=true</code> , <code>use_parity_ipm=true</code> or auto-enabled by DC presence	Builds the parity full-space NLP and solves by Mehrotra-type primal-dual IPM. Optional fallback to native path when enabled.

14.6 Fast Dispatch + PF Fallback (AC-Only Path)

This path is used for fastest feasible dispatch recovery in AC-only operation. It solves a clipped economic dispatch first, then projects via AC PF.

Economic dispatch stage.

$$\min_{P_g} \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}), \quad (63)$$

subject to generator bounds and $\sum_g P_g = P_D$ (clamped to feasible range). For quadratic units, the code uses incremental-cost clipping $P_g = \text{clip}((\lambda - c_1)/(2c_2), P_g^{\min}, P_g^{\max})$.

Feasibility projection stage. The dispatch is passed to AC Newton PF; resulting bus injections are split back across generators at each bus and clamped to individual bounds. If direct projection fails, one reshaped economic-dispatch attempt is executed before declaring failure.

14.7 Native Primal-Dual OPF Formulation

14.7.1 Decision Variables

For the native path, variables are

$$x_{nat} = \left[\theta_{\mathcal{N}_{ac} \setminus \mathcal{N}_{sl}} \quad v_{\mathcal{N}_{PQ}} \quad P_g \quad Q_g \quad v^{\text{dc}} \quad P^{\text{conv}, \text{ac}} \quad Q^{\text{conv}, \text{ac}} \quad P^{\text{conv}, \text{dc}} \right]^\top. \quad (64)$$

The implementation includes all DC bus voltages in v^{dc} ; for DC_V buses, tight bounds are built around the setpoint.

14.7.2 Objective

$$\min_x f(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}). \quad (65)$$

14.7.3 Equality Constraints

AC equations. AC rows reuse the PF residual kernel (with converter table disabled in that kernel call), then subtract explicit converter variables:

$$g_{P,i} = \left(P_i^{\text{spec,PF}} - P_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} P_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{ac} \setminus \mathcal{N}_{sl}, \quad (66)$$

$$g_{Q,i} = \left(Q_i^{\text{spec,PF}} - Q_i^{\text{calc}} \right) - \sum_{c \in \mathcal{C}_i} Q_c^{\text{ac}} = 0, \quad i \in \mathcal{N}_{PQ}. \quad (67)$$

DC equations. For each DC bus k :

$$g_k^{\text{dc}} = P_{d,k}^{\text{dc}} + V_k^{\text{dc}}(G_{dc}V^{\text{dc}})_k - \sum_{c \in \mathcal{C}_k^{\text{dc}}} P_c^{\text{dc}} = 0. \quad (68)$$

Converter energy balance. For converter c :

$$g_c^{\text{conv}} = P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (69)$$

$$P_c^{\text{loss}} = a_c + b_c I_c^{\text{ac}} + (1 - \eta_c)(I_c^{\text{ac}})^2, \quad (70)$$

$$I_c^{\text{ac}} = \frac{\sqrt{(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 + \epsilon}}{V_c^{\text{ac}}}, \quad \epsilon = 10^{-6}. \quad (71)$$

14.7.4 Bounds and Nonlinear Limits

Variable bounds are enforced for $\theta, v, P_g, Q_g, v^{\text{dc}}, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}$.

Nonlinear engineering limits are modeled as $h(x) \leq 0$ and handled by exterior quadratic penalty terms inside the merit model:

$$h_{ij}^{\text{from}} = P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (72)$$

$$h_{ij}^{\text{to}} = P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (73)$$

$$h_c^{\text{conv}} = (P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0. \quad (74)$$

14.7.5 KKT System and Globalization

At each inner iteration, the solver builds

$$\begin{bmatrix} H + \delta I & J_g^\top \\ J_g & -\delta I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_x L \\ g(x) \end{bmatrix}, \quad (75)$$

with dynamic regularization δ , line search, trust/restore logic, and bounded step clipping.

A barrier term is applied to variable bounds; nonlinear branch/converter limits enter the merit model through adaptive exterior penalties.

14.8 Parity Full-Space Formulation

14.8.1 Decision Variables

The parity NLP includes AC, DC, converter, shedding, and enhanced DER variables:

$$x_{\text{par}} = \begin{bmatrix} \theta, v^{\text{ac}}, P_g, Q_g, v^{\text{dc}}, P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}, \\ \Delta P_d, \Delta Q_d, P^{\text{ren}}, Q^{\text{ren}}, P^{\text{stor,ac}}, Q^{\text{stor,ac}}, P^{\text{stor,dc}}, \\ P^{\text{dcdc}}, P^{\text{flex}} \end{bmatrix}^\top. \quad (76)$$

14.8.2 Objective

$$\min_x f_{par}(x) = \sum_{g \in \mathcal{G}} (c_{2g} P_g^2 + c_{1g} P_g + c_{0g}) + \text{VOL}_{\text{L}_{eff}} S_{base} \sum_i (\Delta P_{d,i} + \Delta Q_{d,i}) \quad (77)$$

$$+ \sum_{r \in \mathcal{R}^{\text{curt}}} \max(0, P_r^{\text{rated}} - P_r) c_r^{\text{curt}}. \quad (78)$$

14.8.3 Equality Constraints

AC full-bus balance (scaled). For each AC bus i :

$$g_{P,i} = \left(P_i^{\text{calc}} - P_i^{\text{gen}} - P_i^{\text{fixed}} - P_i^{\text{ren}} - P_i^{\text{stor}} + P_i^{\text{load}} - \Delta P_{d,i} + P_i^{\text{flex}} - P_i^{\text{conv,ac}} \right) \alpha_P = 0, \quad (79)$$

$$g_{Q,i} = \left(Q_i^{\text{calc}} - Q_i^{\text{gen}} - Q_i^{\text{fixed}} - Q_i^{\text{ren}} - Q_i^{\text{stor}} + Q_i^{\text{load}} - \Delta Q_{d,i} - Q_i^{\text{conv,ac}} \right) \alpha_Q = 0. \quad (80)$$

Here $P_i^{\text{fixed}}, Q_i^{\text{fixed}}$ aggregate non-variable injections (static generators, non-curtailable renewables/PV, VPP, microgrid exchange, mobile storage).

DC balance with DCDC coupling. For each DC bus k :

$$g_k^{\text{dc}} = P_{d,k}^{\text{dc}} + \sum_{m \neq k} g_{km} \left((V_k^{\text{dc}})^2 - V_k^{\text{dc}} V_m^{\text{dc}} \right) - P_k^{\text{conv,dc}} + P_k^{\text{dcdc,in}} - P_k^{\text{dcdc,out}} = 0, \quad (81)$$

where the OPF decision variable is P^{in} (*input-side referenced*, i.e. the power absorbed from **bus_in**). The output power is a deterministic function of P^{in} including both efficiency and $I^2 R$ conduction loss:

$$P^{\text{out}} = \begin{cases} \eta P^{\text{in}} - r_{eq} (P^{\text{in}})^2 / (V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ P^{\text{in}} / \eta - r_{eq} (P^{\text{in}})^2 / (V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (82)$$

The parity **ConstraintIndex** allocates dedicated DCDC balance slots and counts them in the equality total, but the current equality evaluator leaves those dedicated rows unpopulated. The actual DCDC physics is folded directly into the two DC bus balance equations:

$$g_{\text{bus_in}} += P^{\text{in}}, \quad g_{\text{bus_out}} -= P^{\text{out}}. \quad (83)$$

DCDC Jacobian and Hessian. The first-order derivatives of the DCDC coupling terms are:

$$\frac{\partial g_{\text{bus_in}}}{\partial P^{\text{in}}} = 1, \quad \frac{\partial g_{\text{bus_out}}}{\partial P^{\text{in}}} = -\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}}, \quad (84)$$

$$\frac{\partial P^{\text{out}}}{\partial P^{\text{in}}} = \begin{cases} \eta - 2r_{eq} P^{\text{in}} / (V^{\text{dc}})^2, & P^{\text{in}} \geq 0, \\ 1/\eta - 2r_{eq} P^{\text{in}} / (V^{\text{dc}})^2, & P^{\text{in}} < 0. \end{cases} \quad (85)$$

When $r_{eq} > 0$, there is also a cross-derivative with respect to the input-side DC voltage:

$$\frac{\partial g_{\text{bus_out}}}{\partial V_{\text{in}}^{\text{dc}}} = \frac{2r_{eq} (P^{\text{in}})^2}{(V_{\text{in}}^{\text{dc}})^3}. \quad (86)$$

The Hessian contributions from the $I^2 R$ loss term are:

$$\frac{\partial^2 g}{\partial (P^{\text{in}})^2} = \frac{2r_{eq}}{(V^{\text{dc}})^2}, \quad \frac{\partial^2 g}{\partial P^{\text{in}} \partial V^{\text{dc}}} = \frac{-4r_{eq} P^{\text{in}}}{(V^{\text{dc}})^3}, \quad \frac{\partial^2 g}{\partial (V^{\text{dc}})^2} = \frac{6r_{eq} (P^{\text{in}})^2}{(V^{\text{dc}})^4}. \quad (87)$$

OPF warm-start convention. Note that the PF uses output-referenced P^{ref} while the OPF uses input-referenced P^{in} . The warm-start path converts between these reference frames when seeding the OPF initial point from a PF result.

Converter balance. For each VSC converter:

$$P_c^{\text{ac}} + P_c^{\text{dc}} + P_c^{\text{loss}} = 0, \quad (88)$$

with the same loss model as in the native formulation.

14.8.4 Inequality Constraints

Nonlinear inequalities:

$$P_{f,ij}^2 + Q_{f,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (89)$$

$$P_{t,ij}^2 + Q_{t,ij}^2 - (S_{ij}^{\text{max}})^2 \leq 0, \quad (90)$$

$$(P_c^{\text{ac}})^2 + (Q_c^{\text{ac}})^2 - (S_c^{\text{max}})^2 \leq 0, \quad (91)$$

$$\left(g_{km} V_k^{\text{dc}} (V_k^{\text{dc}} - V_m^{\text{dc}})\right)^2 - (P_{km}^{\text{max}})^2 \leq 0. \quad (92)$$

Variable bounds are converted to inequality rows:

$$x^{\text{min}} - x \leq 0, \quad x - x^{\text{max}} \leq 0. \quad (93)$$

14.8.5 Primal-Dual IPM System

The parity solver introduces slack $z > 0$ and multipliers $\mu > 0$:

$$g(x) = 0, \quad (94)$$

$$h(x) + z = 0, \quad (95)$$

$$\nabla f(x) + J_g^\top \lambda + J_h^\top \mu = 0, \quad (96)$$

$$z \circ \mu = \gamma \mathbf{1}. \quad (97)$$

A Mehrotra predictor-corrector method is used, with:

1. affine predictor step,
2. adaptive centering parameter $\sigma = (\mu_{aff}/(\mu_{cur} + 10^{-16}))^3$ after clamping the ratio to $[0, 1]$,
3. corrector step with second-order compensation,
4. iterative refinement (2 steps) for conditioning of the KKT solve,
5. fraction-to-boundary step lengths for primal and dual updates.

Implementation note. The current `parity_ipm.cpp` uses the Mehrotra predictor-corrector scheme and implements up to two Gondzio extra correctors. The KKT system is solved using either Eigen `SparseLU` for sparse or `PartialPivLU` for dense factorizations.

Convergence monitors primal infeasibility, dual stationarity, and average complementarity. A best-iterate restoration is applied before final failure.

14.9 Component Coverage by Solver Path

Component / feature	Native primal-dual	Parity IPM
Generator (P_g, Q_g)	decision variable	decision variable
VSCConverter ($P^{\text{conv,ac}}, Q^{\text{conv,ac}}, P^{\text{conv,dc}}$)	decision variable	decision variable
DCBus voltage	decision variable	decision variable
AC branch MVA limits	nonlinear penalty term	nonlinear inequality + slack
DC branch power limits	not explicit in native path	nonlinear inequality + slack
Load shedding ($\Delta P_d, \Delta Q_d$)	not modeled	decision variable with VOLL penalty
Curtailable renewable / controllable PV	not modeled as variable	decision variable
AC storage dispatch	not modeled as variable	decision variable
DC storage dispatch	not modeled as variable	decision variable; packed into <code>AC OPFResult::pstor_mw</code> with <code>stor_map.source_type</code> identifying AC vs DC storage
DCDC dispatch	not modeled as variable	decision variable + DC coupling
Flexible load dispatch	not modeled as variable	decision variable
StaticGen / fixed DER aggregates	not explicit variable in native objective model	included as fixed injections

14.10 AC OPF Result JSON Contract

`opf_result_to_json` writes the scalar solve summary `converged`, `iterations`, `outer_iterations`, `objective`, `max_constraint_violation`, `max_stationarity`, `status`, and `solver_path`. The `solver_path` token is `NativeAC`, `ParityIPM`, or `Unknown`.

The core vectors are `vm`, `va`, `vdc`, `pg_mw`, `qg_mvar`, `dpg_mw`, `dqg_mvar`, `pac_mw`, and `qac_mvar`. Enhanced dispatch is serialized both as structured objects and as legacy bare arrays:

- `renewable_dispatch`: `pren_mw`, `qren_mvar`, and a map array of `original_index/source_type` records;
- `storage_dispatch`: `pstor_mw`, `qstor_mvar`, and a map array; this packed storage array includes AC and DC storage;
- `dcdc_dispatch`: `pdcdc_mw` and a map array;
- `flex_dispatch`: `pflex_mw` and a map array;
- legacy top-level `pren_mw`, `qren_mvar`, `pstor_mw`, `qstor_mvar`, `pdcdc_mw`, and `pflex_mw` arrays.

The `profiling` object contains `linear_solver_backend`, `analyze_calls`, `factorization_calls`, `linear_solve_calls`, `total_iterations`, `accepted_steps`, `rejected_steps`, and `final_barrier_mu`.

14.11 Fallback Behavior

1. If parity IPM is selected and fails, and `allow_fallback=true`, the solver retries with `use_parity_ipm=false`.
2. If native primal-dual fails and fallback is allowed, it attempts the fast dispatch + AC PF path.
3. Status strings in `ACOPFResult::status` record which fallback path was taken.

14.12 DC Optimal Power Flow (`solve_dc_opf`)

The DC OPF model solves a linearized economic dispatch problem using the DC power flow approximation. It is independent of the AC OPF paths above and is invoked through `hacdcpf::opf::solve_dc_opf`.

14.12.1 Decision Variables

$$x_{dc} = \begin{bmatrix} \theta_{\mathcal{N} \setminus \{sl\}} & P_g & P_f & \Delta P_d \end{bmatrix}^\top, \quad (98)$$

where $P_{f,ij}$ are branch flow variables (optional, depending on formulation mode) and $\Delta P_{d,i}$ are per-bus load-shedding variables.

14.12.2 Network Model

The DC power flow relationship is:

$$P_{f,ij} = \frac{\theta_i - \theta_j}{x_{ij}}, \quad (99)$$

with the bus susceptance matrix B assembled from branch reactances x_{ij} . The slack bus angle is fixed at $\theta_{sl} = 0$.

14.12.3 Objective Function

The DC OPF builder constructs both an LP model and a QP model. The QP model keeps true quadratic generator costs for QP-capable backends; LP backends use the linearized model.

LP linearized cost.

$$\min \sum_{g \in \mathcal{G}} c_g^{\text{lin}} P_g + \sum_i \text{VOLL}_{\text{eff}} \cdot \Delta P_{d,i} \cdot S_{\text{base}}, \quad (100)$$

where the linearized coefficient is evaluated at the generator midpoint:

$$c_g^{\text{lin}} = c_{1g} + 2c_{2g} \frac{P_g^{\text{min}} + P_g^{\text{max}}}{2}. \quad (101)$$

QP quadratic cost.

$$\min \frac{1}{2} x^\top Q x + c^\top x, \quad (102)$$

where $Q_{gg} = 2c_{2g}S_{\text{base}}^2$ and $c_g = c_{1g}S_{\text{base}}$. This mode is used by `NativeQP` and `Gurobi`. The `HiGHS` path currently calls the LP adapter; QP support there is marked `TODO` in the source.

VOLL auto-tuning. The load-shedding penalty is auto-tuned as $1.1 \times$ the maximum generator marginal cost to guarantee load serving priority.

14.12.4 Constraints

Power balance (per bus).

$$\sum_{g \in \mathcal{G}_i} P_g - P_{d,i} + \Delta P_{d,i} = \sum_{j \in \mathcal{N}_i} P_{f,ij}, \quad \forall i. \quad (103)$$

Flow definition.

$$P_{f,ij} - b_{ij}(\theta_i - \theta_j) = 0, \quad \forall (i, j) \in \mathcal{E}. \quad (104)$$

Variable bounds.

$$\theta_{sl} = 0, \quad -\pi \leq \theta_i \leq \pi, \quad i \neq sl, \quad (105)$$

$$P_g^{\min} \leq P_g \leq P_g^{\max}, \quad (106)$$

$$-r_a^{\max} \leq P_{f,ij} \leq r_a^{\max}, \quad (107)$$

$$0 \leq \Delta P_{d,i} \leq \max(P_{d,i}, 0.01/S_{\text{base}}). \quad (108)$$

14.12.5 Solver Backend Selection

`DCOPFOptions::solver` defaults to `DCOPFSolverBackend::Auto`. The solver always builds the QP form and then dispatches by backend:

1. **NativeQP** (LCQP solver): true quadratic costs via primal-dual IPM.
2. **Gurobi**: commercial QP solver.
3. **HiGHS**: open-source LP solver (linearized costs).
4. **Native Dual Simplex**: via `solve_lp_with_basis()`.
5. **Auto**: tries **NativeQP**, then **Gurobi**, then **HiGHS**, then native dual simplex.

14.12.6 Locational Marginal Prices

LMP extraction is not implemented in the current source. `dc_opf.cpp` contains a TODO for dual extraction and fills `lmp`, `branch_mu_lower`, and `branch_mu_upper` with zeros.

14.13 Reactive Power Optimization (`solve_rpo`)

The reactive power optimization (RPO) module searches over discrete voltage and reactive-power controls (transformer on-load tap changers and switchable shunt steps) with continuous AC OPF evaluations. The current source implements the OA/coordinate-search/ternary/pairwise refinement workflow below rather than a generic branch-and-bound MINLP solve loop.

14.13.1 Problem Structure

Discrete variables.

- Transformer OLTC tap positions: $\text{tap ratio} = 1 + (\text{tap_pos} - \text{tap_neutral}) \times \text{step_pct}/100$.
- Switchable shunt steps: $B_s = \text{step} \times b_{\text{per_step}}$ (susceptance per step).

Continuous variables. Bus voltages and generator reactive outputs, solved via AC OPF at each discrete configuration.

14.13.2 Objective Functions

Selectable objectives:

$$\text{MinVoltageDeviation: } \sum_i w_{\text{vdev}} (V_i - V_{\text{tgt}})^2, \quad (109)$$

$$\text{MinActiveLoss: } \sum_g P_g - \sum_d P_d, \quad (110)$$

$$\text{Combined: } w_{\text{vdev}} \sum_i (V_i - V_{\text{tgt}})^2 + \left(\sum_g P_g - \sum_d P_d \right). \quad (111)$$

14.13.3 Algorithm: Four-Phase Outer Approximation

Phase 0: Sensitivity ranking. Perturb each discrete variable by ± 1 from the baseline (two NLP solves per variable). Compute $|\Delta \text{obj}|$ for each variable and order by descending sensitivity.

Phase 1: Ternary-search sweep. For each variable in sensitivity order, exploit approximate unimodality of the RPO objective using integer ternary search ($O(\log R)$ evaluations per variable instead of $O(R)$). At each evaluation, generate an outer-approximation cutting plane:

$$f(y) \geq f_k + g_k^\top (y - y_k), \quad (112)$$

where g_k is a finite-difference gradient estimate.

Phase 2: Coordinate descent with OA pruning (up to 3 cycles). Sweep each variable and evaluate only candidates where the OA lower bound $<$ incumbent. Add OA cuts after each NLP evaluation. Terminate if no improvement this cycle. The OA lower bound is:

$$\max_k f_k + g_k^\top (y - y_k). \quad (113)$$

Phase 3: Pairwise neighborhood polishing. Check ± 1 perturbations of every pair of discrete variables. OA cuts prune many candidates before expensive NLP evaluation.

14.13.4 Inner NLP Solver

At each discrete configuration, the AC OPF solver (IPM by default, fallback enabled) is called to solve the continuous sub-problem. Solution caching by discrete configuration key avoids redundant NLP evaluations.

14.13.5 Result Structure

The result includes per-device tap/shunt adjustments, the number of NLP solves, OA cuts added, nodes explored, and the final status. Current source status strings include "Optimal(OA)", "Timelimit(OA)", "Nodelimit(OA)", and "Nofeasibleconfigurationfound(OA)".

15 Power-Model Builders (AML Layer)

The `src/power_models/` subsystem provides four algebraic model builders that construct optimisation or simulation models using the AML expression layer. All builders live under the namespace `hacdcpf::power_models` and are declared in `include/hacdcpf/power_models/`.

15.1 AML Abstraction Layer

The AML layer (`include/hacdcpf/aml/aml.hpp`) is a thin namespace alias:

```
#include <mipsolvers/aml/aml.hpp>
namespace hacdcpf {
    namespace aml = mipsolvers::aml;
}
```

All expression types (`Variable`, `NonlinearExpr`, `Constraint`, etc.) and the `AMLModel` container are inherited from the `mipsolvers` library. The builders populate an `AMLModel` instance and hand it to the solver engine.

15.2 LinDistFlow Builder

15.2.1 Mathematical Formulation

The `LinDistFlowBuilder` constructs a linearised DistFlow LP (Baran & Wu, 1989) for a radial distribution network.

Variables

- P_{ij} (MW) – active power flow on branch (i, j) ;
- Q_{ij} (MVar) – reactive power flow on branch (i, j) ;
- v_i (pu²) – squared voltage magnitude at bus i .

Constraints (per non-root bus j) Active power balance:

$$P_{\pi(j),j} - P_j^{\text{load}} - \sum_{k:(j,k) \in \mathcal{E}} P_{jk} = 0. \quad (114)$$

Reactive power balance:

$$Q_{\pi(j),j} - Q_j^{\text{load}} - \sum_{k:(j,k) \in \mathcal{E}} Q_{jk} = 0. \quad (115)$$

Linearised voltage drop on branch (i, j) with resistance r_{ij} and reactance x_{ij} :

$$v_j = v_i - 2(r_{ij} P_{ij} + x_{ij} Q_{ij}). \quad (116)$$

Thermal limits (VA rating S_{ij}^{max}):

$$P_{ij}^2 + Q_{ij}^2 \leq (S_{ij}^{\text{max}})^2 \quad (\text{relaxed as LP box in current version}). \quad (117)$$

Voltage bounds:

$$v_{\min} \leq v_i \leq v_{\max} \quad \forall i, \quad (118)$$

with defaults $v_{\min} = 0.81$ (pu², i.e. 0.9 pu) and $v_{\max} = 1.21$ (pu², i.e. 1.1 pu).

Objective Minimise total active-power loss (or external generator cost when provided):

$$\min \sum_{(i,j) \in \mathcal{E}} r_{ij} I_{ij}^2 \approx \min \sum_{(i,j) \in \mathcal{E}} r_{ij} \frac{P_{ij}^2 + Q_{ij}^2}{v_i}. \quad (119)$$

In the pure LP (default) the loss objective uses the DistFlow convex-relaxation approximation.

15.2.2 Input/Output Structs

LinDistFlowData field	Type	Meaning
bus_ids	vector<string>	Ordered list of bus IDs (root first).
root_bus	string	Root bus ID (voltage fixed to $v_0 = 1.0$ pu ²).
branches	vector<LinBranchData>	Each branch: id, from_bus, to_bus, r_pu, x_pu, rate_a_mva.
p_load_mw	map<string, double>	Active load by bus ID (MW).
q_load_mvar	map<string, double>	Reactive load by bus ID (MVar).
v_min_pu_sq	double	Squared lower voltage bound; default 0.81.
v_max_pu_sq	double	Squared upper voltage bound; default 1.21.
base_mva	double	System base MVA.

`LinDistFlowResult` provides per-bus squared voltages v_i , per-branch P_{ij} and Q_{ij} , total loss, and the AML solver status.

15.3 AC OPF Builder

15.3.1 Mathematical Formulation

The `ACOPFBuilder` populates an `AMLModel` with the standard MATPOWER AC OPF. All quantities are in per-unit on the system `base_mva`.

Objective

$$\min \sum_g [c_{2,g} (P_g S_b)^2 + c_{1,g} (P_g S_b) + c_{0,g}]. \quad (120)$$

Power balance at bus i

$$\sum_{g \in i} P_g - P_{d,i} - G_{s,i} V_i^2 - \sum_{\ell} (P_{f,\ell} + P_{t,\ell}) = 0, \quad (121)$$

$$\sum_{g \in i} Q_g - Q_{d,i} + B_{s,i} V_i^2 - \sum_{\ell} (Q_{f,\ell} + Q_{t,\ell}) = 0. \quad (122)$$

Branch π -model flows For branch $\ell = (i \rightarrow k)$ with admittance $y_s = 1/(r + jx)$, off-nominal tap τ , phase shift φ and shunt b_c :

$$Y_{ff} = \frac{y_s + jb_c/2}{\tau^2}, \quad Y_{tt} = y_s + jb_c/2, \quad (123)$$

$$Y_{ft} = -\frac{y_s}{\tau e^{-j\varphi}}, \quad Y_{tf} = -\frac{y_s}{\tau e^{j\varphi}}. \quad (124)$$

$$P_{f,\ell} = \Re(Y_{ff}) V_i^2 + \Re(Y_{ft}) V_i V_k \cos(\theta_i - \theta_k) + \Im(Y_{ft}) V_i V_k \sin(\theta_i - \theta_k), \quad (125)$$

$$Q_{f,\ell} = -\Im(Y_{ff}) V_i^2 - \Im(Y_{ft}) V_i V_k \cos(\theta_i - \theta_k) + \Re(Y_{ft}) V_i V_k \sin(\theta_i - \theta_k). \quad (126)$$

(Identical expressions for $P_{t,\ell}$, $Q_{t,\ell}$ with subscripts swapped.)

Branch thermal limits

$$P_{f,\ell}^2 + Q_{f,\ell}^2 \leq (S_\ell^{\max})^2, \quad (127)$$

$$P_{t,\ell}^2 + Q_{t,\ell}^2 \leq (S_\ell^{\max})^2. \quad (128)$$

Voltage/generation limits

$$V_{i,\min} \leq V_i \leq V_{i,\max}, \quad P_{g,\min} \leq P_g \leq P_{g,\max}, \quad Q_{g,\min} \leq Q_g \leq Q_{g,\max}. \quad (129)$$

15.3.2 Input Structs

Struct	Key fields
ACOPFBusData	id, pd_pu, qd_pu, gs_pu, bs_pu, vm_min, vm_max, vm0, va0_rad, is_ref.
ACOPFGenData	id, bus_id, pg_min_pu, pg_max_pu, qg_min_pu, qg_max_pu, cost_c2, cost_c1, cost_c0.
ACOPFBranchData	id, from_bus, to_bus, r_pu, x_pu, bc_pu, tap, shift_deg, rate_a_pu.
ACOPFData	Container: buses, generators, branches, base_mva.

ACOPFBuilderResult exposes the populated AMLModel, variable handles (Pg, Qg, Vm, Va), and the constraint ranges for post-solution dual extraction (LMPs).

15.4 Hybrid AC/DC OPF Builder

The ACDCOPFBuilder extends ACOPFBuilder with DC buses, DC branches, and VSC converter coupling. It inherits all AC OPF data structs and adds the following.

15.4.1 Mathematical Formulation

DC bus voltage bounds

$$V_{k,\min}^{dc} \leq V_k^{dc} \leq V_{k,\max}^{dc} \quad \forall k \in \mathcal{B}^{dc}. \quad (130)$$

DC power balance at bus k

$$\sum_{m \in \delta^+(k)} \frac{V_k^{dc} - V_m^{dc}}{r_{km}} - \sum_{m \in \delta^-(k)} \frac{V_m^{dc} - V_k^{dc}}{r_{mk}} + P_k^{dc, \text{conv}} = 0. \quad (131)$$

VDC-Q converters enforce $V_c^{dc} = V_c^{dc, \text{set}}$ via equality constraints.

VSC converter coupling Per converter c with AC bus i and DC bus k :

$$P_{ac,c} \in [p_{c,\min}, p_{c,\max}]/S_b, \quad (132)$$

$$Q_{ac,c} \in [q_{c,\min}, q_{c,\max}]/S_b. \quad (133)$$

Quadratic loss model:

$$P_{ac,c} + P_{dc,c} + a_c + b_c I_{ac,c} + c_c I_{ac,c}^2 = 0, \quad (134)$$

where $I_{ac,c} = \sqrt{P_{ac,c}^2 + Q_{ac,c}^2 + \varepsilon} / V_i$ and the loss parameters are $a_c = \text{loss_mw}/S_b$, $b_c = \text{loss_percent}/100$, $c_c = 1 - \eta_c$.

15.4.2 Additional Input Structs

Struct	Key fields
ACDCOPFDCBusData	id, vdc_min=0.9, vdc_max=1.1, vdc0=1.0, pd_pu=0.0, is_vdc_slack, vdc_set=1.0.
ACDCOPFDCBranchData	id, from_bus, to_bus, r_pu=0.01.
ACDCOPFConverterData	id, ac_bus_id, dc_bus_id, power bounds (pac_min/max, qac_min/max), warm-start (pac0, qac0), loss coefficients (a_pu, b_loss, c_loss).
ACDCOPFData	AC data via ACOPFData base, plus dc_buses, dc_branches, converters.

15.5 Security-Constrained Unit Commitment (SCUC) Builder

15.5.1 Mathematical Formulation

The `SCUCBuilder` constructs a MILP unit commitment over a multi-period horizon $\mathcal{T}$.

Decision variables

- $p_{g,t} \geq 0$ – dispatch level (MW) of unit g in period t ;
- $u_{g,t} \in \{0, 1\}$ – on/off binary status;
- $s_{g,t} \geq 0$ – start-up cost incurred if unit starts in period t .

Objective

$$\min \sum_{t \in \mathcal{T}} \sum_g [c_g p_{g,t} + S_g s_{g,t} + C_g u_{g,t}]. \quad (135)$$

Constraints System balance:

$$\sum_g p_{g,t} = D_t \quad \forall t. \quad (136)$$

Capacity:

$$P_{g,\min} u_{g,t} \leq p_{g,t} \leq P_{g,\max} u_{g,t} \quad \forall g, t. \quad (137)$$

Ramp limits:

$$p_{g,t} - p_{g,t-1} \leq R_g^{up}, \quad p_{g,t-1} - p_{g,t} \leq R_g^{dn}. \quad (138)$$

Start-up logic:

$$s_{g,t} \geq u_{g,t} - u_{g,t-1} \geq 0. \quad (139)$$

15.5.2 Input/Output Structs

SCUCData field	Type	Meaning
generator_ids	vector<string>	Generator IDs.
period_ids	vector<string>	Period labels (e.g. “H01” ... “H24”).
demand_MW	map<string, double>	System demand per period (MW).
cost_per_MWh	map<string, double>	Energy cost c_g per generator (\$/MWh).
startup_cost	map<string, double>	Start-up cost S_g (\$).
fixed_cost	map<string, double>	No-load cost C_g (\$/h).
pmin_MW	map<string, double>	Minimum stable generation $P_{g,\min}$ (MW).
pmax_MW	map<string, double>	Maximum generation $P_{g,\max}$ (MW).
ramp_up_MW	map<string, double>	Ramp-up limit R_g^{up} (MW/period).
ramp_dn_MW	map<string, double>	Ramp-down limit R_g^{dn} (MW/period).

SCUCResult provides per-generator, per-period schedules $(p_{g,t}, u_{g,t}, s_{g,t})$, total cost, and solver status.

16 I/O Layer: Parsers, Case Builders, and JSON Contracts

16.1 MATPOWER Parser

`hacdcpf::io::MatpowerParser` reads MATPOWER .m case files from disk (`include/hacdcpf/io/matpower_parser.hpp`). The parser understands the `mpc.version~='2'` format used by MATPOWER 6 and populates a `HybridPowerSystem` object.

Covered table sections

Section	Mapped struct(s)
<code>mpc.bus</code>	<code>ACBus</code> (type, Pd, Qd, Gs, Bs, Vm, Va, Vmin, Vmax).
<code>mpc.gen</code>	<code>Generator</code> (bus, Pg, Qg, Qmax, Qmin, Vg, Pmax, Pmin, status).
<code>mpc.branch</code>	<code>ACBranch</code> (from, to, r, x, b, rateA/B/C, tap, shift, status).
<code>mpc.gencost</code>	Generator cost model: polynomial (type 2) or piecewise-linear (type 1).
<code>mpc.dcline</code>	<code>VSCConverter</code> (basic two-terminal DC; mapped to converter pair).
<code>mpc.bus_name</code>	Populates <code>ACBus::name</code> .

Data directory `data/` ships with 80+ MATPOWER case files covering IEEE 9, 14, 30, 57, 118, 300, and a range of synthetic US/European grids up to 70 000 buses.

16.2 JSON I/O

`include/hacdcpf/io/json_io.hpp` provides:

- `load_hybrid_system_from_json(path)` – read a `HybridPowerSystem` from disk;
- `save_hybrid_system_to_json(sys,path)` – write to disk;
- `result_to_json(result)` – serialise a `PowerFlowResult` to JSON;
- `opf_result_to_json(result)` – serialise an `OPFResult`.

The on-disk schema mirrors the field names of the C++ structs and includes metadata (`name`, `base_mva`, `timestamps`).

16.3 Pre-built Test Networks (`case_builders.hpp`)

`hacdcpf::io` exposes a library of factory functions for use in tests and benchmarks. All returned systems include at least one DC bus and VSC converter unless the `_ac_only` variant is used.

Function	Description
<code>build_ieee14_acdc()</code>	IEEE 14-bus AC + one DC link.
<code>build_ieee24_3area_acdc()</code>	IEEE 24-bus RTS divided into 3 control areas, interconnected via DC.
<code>build_ieee24_3area_acdc_expanded()</code>	Expanded version with additional DER and storage.
<code>build_ieee118_acdc()</code>	IEEE 118-bus + multi-terminal DC grid.
<code>build_ac_only_version(sys)</code>	Strip all DC components from an existing system.
<code>build_case33bw_acdc()</code>	Baran–Wu 33-bus distribution + DC micro-grid.
<code>build_case33mg_acdc()</code>	33-bus microgrid variant with storage.
<code>build_case69_acdc()</code>	69-bus radial distribution + DC feeder.
<code>build_case300_acdc()</code>	IEEE 300-bus + HVDC link.
<code>build_demo_multizone_acdc()</code>	Multi-zone demonstration system.
<code>build_case2000_acdc()</code>	2000-bus synthetic + DC overlay.
<code>build_dist33_microgrid_der()</code>	33-bus distribution with DER and battery storage.
<code>build_comprehensive_hybrid_acdc()</code>	Stress-test system with all component types.
<code>build_market_3bus_toy()</code>	3-bus toy system for market/OPF unit tests.
<code>build_market_5bus_acdc_toy()</code>	5-bus AC/DC toy for hybrid OPF unit tests.

17 Test Suite

All tests use the Catch2 framework (fetched automatically by CMake via `FetchContent`). Build and run with:

```
cmake -B build -DHACDCPF_BUILD_TESTS=ON
cmake --build build
cd build && ctest --output-on-failure
```

17.1 Test File Overview

File	Coverage
<code>test_hacdcpf.cpp</code>	Smoke tests for model construction, power flow, OPF, and basic analysis on 2-bus AC and IEEE 14 AC/DC systems. Uses <code>make_simple_ac_system()</code> and <code>build_ieee14_acdc()</code> .
<code>test_hacdcoss.cpp</code>	Smoke tests for the <code>hacdcoss</code> module: <code>NetworkModel</code> , <code>ScenarioData</code> , <code>TimeSeriesLoader</code> , <code>AdmittanceModel</code> , <code>PFSolver</code> (<code>hacdcoss</code> namespace), and <code>OPFModel</code> . Validates a 2-bus AC network and a 4-bus DC ring.
<code>test_all_components_pf.cpp</code>	Exercises every component type (VSC, DCDC converters, <code>ChargingStation</code> , <code>AsymmetricLoad</code> , <code>Switch</code> , <code>CircuitBreaker</code> , <code>Storage</code> , <code>RenewableGen</code> , etc.) through the full power-flow pipeline.
<code>test_acopf_dcopf_crossval.cpp</code>	Cross-validates AC OPF, DC OPF (LP), and power-flow feasibility on a batch of MATPOWER cases from the <code>data/</code> directory. Compares generation costs and bus-voltage ranges.
<code>test_enhanced_solver.cpp</code>	Tests the multi-phase robust pipeline: island detection, distributed slack solver, <code>RobustNonlinearOptions</code> phase toggling, and fallback path activation.
<code>test_powerflow_performance.cpp</code>	Wall-clock performance regression tests on medium-to-large cases (IEEE 118, case300, case2000). Fails if solve time exceeds reference limits.
<code>test_scaling_regression.cpp</code>	Numerical scaling regression: checks that per-unit results match reference solutions to prescribed tolerance after toggling residual/variable/Jacobian scaling options.
<code>test_unified_pf_upgrade.cpp</code>	Integration tests for the unified PF upgrade path. Uses <code>hacdcpf::powerflow::inf_norm</code> to verify residual norms on upgraded systems.

17.2 Example Test Pattern

The typical Catch2 test pattern in this project:

```
TEST_CASE("brief description", "[tag1][tag2]") {
    auto sys = hacdcpf::io::build_ieee14_acdc();
    hacdcpf::PowerFlowOptions opt;
    opt.tol = 1e-8;

    auto result = hacdcpf::solve_power_flow(sys, opt);

    REQUIRE(result.converged);
    REQUIRE_THAT(result.buses[0].vm_pu,
        Catch::Matchers::WithinAbs(1.06, 1e-4));
}
```

Tags used across the suite include `[model]`, `[pf]`, `[opf]`, `[integration]`, `[enhanced]`, `[island]`, `[performance]`, and `[scaling]`.

18 Architecture Assessment and Roadmap

This chapter records an external architectural review of the `hacdcpf` package conducted against the updated codebase described in Chapters ??–??. It documents observed strengths, remaining design risks, module-level recommendations, testing strategy, dependency rules, and a prioritised roadmap.

18.1 Overall Quality Assessment

The package architecture now resembles a mature scientific computing library. Table ?? summarises the current design quality by area.

Table 134: Architecture quality by area.

Area	Quality	Comment
Public model design	Strong	Rich component model; clear I/O field dictionaries
Solver layering	Strong	Projection and assembly separation is a major improvement
PF solver design	Strong	Multiple routes and fallback methods well documented
OPF design	Good–Strong	Native, parity-IPM, DC-QP, and reactive routes well separated
I/O contracts	Strong	JSON and Excel field dictionaries are detailed
Error handling	Improved	<code>Result<T></code> , <code>Error</code> , <code>ErrorCode</code> are good foundations
Analysis modules	Promising	Requires careful dependency control
Backend abstraction	Improved	Linear/NLP/QP backend split is the right direction
Documentation	Strong	Implementation-facing handbook is detailed
Testability	Likely improved	Needs evidence through test matrix and CI policy

The overall data-flow pipeline is now clearly separated:

```
HybridPowerSystem ↓ Projection Layer ↓ Canonical Models ↓ Assembly Layer ↓ Flat
Solver Data ↓ PF / OPF / Analysis Solvers ↓ Result Objects ↓ I/O and Reports
```

18.2 Confirmed Strengths

18.2.1 Horizontal Infrastructure Layer

The dedicated solver-infrastructure layer covering assembly, projection, backend abstractions, and the thread pool (Chapter ??) is a major improvement. This separation is essential for long-term maintainability: the rich physical model can evolve without destabilising the numerical kernels.

18.2.2 Monadic Result Handling

The `Result<T>` / `Error` / `ErrorCode` design (Section ??) addresses one of the most important concerns in simulation packages: handling invalid input and solver failures predictably. A monadic result type is much better than relying on bare exceptions, booleans, or string messages.

18.2.3 Backward-Compatible Option Refactoring

`PowerFlowOptions::from_parts()` preserves flat API compatibility for existing users while enabling structured internal configuration. The recommended long-term pattern is:

```
struct PowerFlowOptions {
    // Legacy flat fields
    double tolerance;
    int max_iterations;
    bool enable_homotopy;

    static PowerFlowOptions from_parts(
        const ConvergenceOptions& convergence,
        const LinearSolverOptions& linear,
        const GlobalizationOptions& globalization,
        const PVPQSwitchingOptions& pvpq,
        const ConverterModeOptions& converter,
        const ZipLoadOptions& zip,
        const RuntimeOptions& runtime
    );

    PowerFlowOptionParts to_parts() const; // optional round-trip
};
```

18.2.4 Explicit Validation Subsystem

A dedicated validation layer capable of catching missing slack buses, duplicate IDs, invalid branch endpoints, inconsistent voltage limits, unsupported OPF configurations, and islands without a source gives the package its primary defence against badly formed inputs. The recommended list of checks is given in Section ??.

18.2.5 Projection plus Assembly Separation

- **Projection:** rich physical model → canonical electrical model.
- **Assembly:** canonical model → numerical vectors, matrices, and residual functions.

This boundary allows rich devices (motors, chargers, PV arrays, flexible loads, switches, transformers, VSC and DCDC converters) to evolve independently of solver kernels.

18.2.6 Centralised Defaults

Every default is now traceable to one of: engineering convention, MATPOWER compatibility, package-specific design, safety fallback, or numerical stabilisation. Avoid silent defaults for safety-critical fields; such fields should be required rather than defaulted.

18.3 Remaining Design Risks

18.3.1 Analysis Layer May Become Too Broad

The `analysis/` module currently covers carbon tracing, distribution PF, three-phase unbalanced PF, PV modelling, reliability, resilience, and time-series simulation. These are not all the same type of functionality. Consider explicit subdomains:

```
include/hacdcpf/analysis/
  carbon/
  distribution/
  three_phase/
  pv/
  reliability/
  resilience/
  time_series/
```

Dependency rule for analysis Analysis modules may depend on:

- public model and validation API,
- public solver API and result types.

They must not depend on private Newton/IPM internals, assembly private data, or backend private classes.

18.3.2 Three-Phase and Balanced PF Should Not Share One Abstraction

Three-phase unbalanced PF is structurally different from balanced positive-sequence PF. Prefer distinct free functions over one overly generic virtual interface:

```
BalancedPowerFlowResult solve_power_flow(
    const HybridPowerSystem& sys,
    const PowerFlowOptions& opt
);

ThreePhasePowerFlowResult solve_three_phase_power_flow(
    const ThreePhaseSystem& sys,
    const ThreePhasePowerFlowOptions& opt
);
```

A common `SolverSummary` (converged, iterations, max residual, status) can be shared where natural without forcing identical result shapes.

18.3.3 Error Handling Policy Needs Explicit Definition

Now that both `Result<T>` and exceptions are present, define exactly when each mechanism is used. A recommended policy is given in Table ??.

Table 135: Recommended error handling policy.

Situation	Mechanism
Expected input validation failure	<code>Result<T></code>
Solver non-convergence	result object with status
Optional feature unavailable	<code>Result<T></code> consistently
Programmer error / invariant violation	assertion or internal exception
File not found	<code>Result<T></code> for library API
JSON parse failure	<code>Result<T></code> with error code and location
Backend crash / failure	<code>Result<T></code> with backend diagnostic

Provide both styles where helpful:

```
Result<HybridPowerSystem> try_load_json(const std::string& path);
HybridPowerSystem load_json_or_throw(const std::string& path);
```

18.3.4 Field Dictionary / Serializer Drift

Detailed I/O field dictionaries are excellent, but documentation, serializers, and model structs can drift apart over time. Automated tests should verify:

- every documented field exists in the serializer,
- every serializer field is documented,
- JSON round-trip preserves each field,
- each documented default equals the code default.

A single machine-readable field-descriptor registry (see Section ??) is the preferred long-term solution.

18.3.5 Backend Capability Discovery

Since the package has optional backends (HiGHS, Gurobi, Ipopt, native IPM, etc.), expose a runtime capability query:

```
struct SolverCapabilities {
    bool has_sparse_lu;
    bool has_highs;
    bool has_gurobi;
    bool has_native_ipm;
    bool has_excel;
    bool supports_quadratic_objective;
    bool supports_integer_variables;
};

SolverCapabilities get_solver_capabilities();
```

This enables portable user code that selects a backend based on availability rather than compile-time flags alone.

18.4 Module-Level Recommendations

18.4.1 Model Layer

Separate persistent fields from runtime fields If any model struct contains both user-input data and runtime caches, split them. Do not mix result fields into input model objects unless explicitly supporting in-place result annotation.

Strong IDs at boundaries At projection boundaries, typed reference structs avoid accidentally connecting a DC converter terminal to an AC bus field:

```
struct ACBusId { std::string value; };
struct DCBusId { std::string value; };
```

Stable enum serialisation Treat enum string names as serialised API. Document whether strings are case-sensitive. Avoid casual renaming once a version ships.

18.4.2 Validation Layer

Use tiered validation levels:

```
enum class ValidationLevel {
    Basic,          // required fields, references, limits
    Electrical,     // topology, slack/reference, physical consistency
    SolverReady,    // checks required by PF/OPF kernels
    Strict          // warnings promoted to errors
};
```

A structured diagnostic shape supports GUIs, APIs, and batch workflows:

```
struct Diagnostic {
    Severity severity; // Info, Warning, Error
    ErrorCode code;
    std::string message;
    std::string component_type;
    std::string component_id;
    std::string field;
};
```

18.4.3 Projection Layer

Projection report Every projection should optionally return a diagnostic report that records mappings such as:

Source	Canonical
PVSystempv_12	StaticGeneratorsg_pv_12
Motormotor_7	EquivalentLoadload_motor_7
ClosedSwitchsw_3	BusMergegroup_2
OpenSwitchsw_4	DisabledBranchbranch_sw_4

Reversible mapping Maintain a `ComponentMapping` record with source type, source ID, canonical type, canonical ID, and participation factor. This is essential for result attribution, carbon tracing, and reporting.

18.4.4 Assembly Layer

Assembly should be *deterministic*: given the same canonical system, it must produce the same ordering, index maps, vector layout, and sparsity pattern. Explicit layout objects make residual code readable:

```
struct PowerFlowVariableLayout {
    std::vector<ACBusIndex> angle_variables;
    std::vector<ACBusIndex> voltage_variables;
    std::vector<DCBusIndex> dc_voltage_variables;
    std::vector<VSCIndex> converter_variables;
};
```

18.4.5 Power-Flow Layer

Standardise solver status Every PF route should return a common summary:

```
struct SolverSummary {
    TerminationStatus status;
    bool    converged;
    int     iterations;
    double  max_mismatch;
    double  elapsed_ms;
    std::vector<Diagnostic> diagnostics;
};
```

Visible fallback trace Record every solver method transition in the result:

```
{
  "solver_trace": [
    {"method": "Newton",
     "status": "SingularJacobian", "iterations": 4},
    {"method": "HomotopyContinuation",
     "status": "Converged", "iterations": 12}
  ]
}
```

18.4.6 OPF Layer

Formulation separate from solver For each OPF route maintain the pipeline: model extraction → formulation construction → backend solve → result mapping → post-check.

Independent feasibility audit After every solve, independently check AC/DC power balance, voltage limits, generator and branch limits, converter limits, and objective recomputation. Return a structured `OpfAudit` object.

Explainable infeasibility Provide likely-cause diagnostics instead of a bare “infeasible” status:

- bus has load but no connected source,
- generator `p_min_mw > p_max_mw`,
- voltage upper limit below lower limit,
- converter power exceeds DC grid capacity.

18.4.7 I/O Layer

Format versioning Every exported JSON should include:

```
{
  "format": "hacdcpf.hybrid_power_system",
  "format_version": "1.0",
  "package_version": "x.y.z"
}
```

Import modes

```
enum class ImportMode { Strict, Permissive };
```

In Strict mode unknown columns are errors; in Permissive mode they become warnings.

Field-descriptor registry A machine-readable registry derives the serialiser, documentation table, and tests from one source:

```
constexpr FieldDescriptor ac_bus_fields[] = {
    {"id",    Required::Yes, DefaultPolicy::None},
    {"vn_kv", Required::Yes, DefaultPolicy::None},
    {"vm_pu", Required::No,  DefaultPolicy::One},
};
```

18.4.8 Analysis Suite

Carbon tracing Expose the tracing method explicitly:

```
enum class CarbonTracingMethod {
    AverageMix, ProportionalSharing,
    MarginalEmission, FlowTracing
};
```

Time-series simulation Avoid mutating the base system in uncontrolled ways. Recommended pattern:

```
struct TimeSeriesScenario {
    HybridPowerSystem base_system;
    std::vector<TimeStepInput> steps;
};
```

18.5 Recommended Testing Matrix

Testing should be organised by layer. Table ?? lists the recommended test groups.

Table 136: Recommended test groups by layer.

Layer	Test groups
Model / validation	ModelDefaultsTest, EnumSerializationTest, ValidationDuplicateIdsTest, ValidationDanglingReferencesTest, ValidationInvalidLimitsTest, ValidationIslandWithoutSlackTest
Projection	ProjectionSwitchMergingTest, ProjectionMotorAsLoadTest, ProjectionPVAsStaticGeneratorTest, ProjectionStorageMappingTest
Assembly	AssemblyVariableOrderingTest, AssemblySparsePatternTest, AssemblyIndexMapTest, AssemblyPerUnitConversionTest
Power flow	PFIEEE14ConvergenceTest, PFHybridTwoBusVSCTest, PFDCGridConvergenceTest, PFIslandedSystemTest, PFHomotopyFallbackTest, PFNewtonKrylovFallbackTest, PFNCPCComplementarityTest
OPF	OPFACSmallCaseTest, OPFDCQPBackendTest, OPFParityIPMTest,

Layer	Test groups
I/O	OPFReactiveTapShuntTest, OPFInfeasibilityDiagnosticTest JsonRoundTripAllComponentsTest, MatpowerImportIEEE14Test, UnknownFieldStrictModeTest, MissingOptionalFieldDefaultTest, ResultJsonSchemaTest
Analysis	CarbonTracingKnownCaseTest, DistributionRadialFeederTest, ThreePhaseUnbalancedKnownCaseTest, PVArrayIrradianceCurveTest, ReliabilitySingleOutageTest, TimeSeriesProfileReplayTest

For complementarity variables a and b , each NCP unit test should verify:

$$\phi(a, b) = 0 \iff a \geq 0, b \geq 0, ab = 0. \quad (140)$$

18.6 Recommended CI Pipeline

Stage	Jobs
1	clang-format check
2	clang-tidy selected checks
3	Build matrix: Debug / Release, Excel OFF / ON, backends disabled / enabled
4	Unit tests: model, validation, projection, assembly, I/O
5	Numerical tests: PF, OPF, analysis
6	Sanitizers: AddressSanitizer, UndefinedBehaviorSanitizer
7	Documentation: LaTeX build
Nightly	Large benchmarks, ThreadSanitizer, coverage, performance regression

18.7 Dependency Boundary Rules

18.7.1 Allowed Dependencies

Module	May depend on
api	model,io,validation,power_flow,optimal_power_flow,analysis
io	model,validation,resulttypes
validation	model
projection	model,validation
assembly	canonical/projection output, model enums, utilities
power_flow	assembly,backend,modelresulttypes
optimal_power_flow	assembly,backend,power_models
power_models	AML, canonical model data
analysis	model,resulttypes,publicsolverAPIs
backend	external solver libraries
engine	compatibility/adapters only

18.7.2 Disallowed Dependencies

Module	Must not depend on
model	io, power_flow, optimal_power_flow, analysis
validation	solver internals
power_flow	io
optimal_power_flow	io
assembly	Excel / JSON I/O
backend	model-specific business logic
analysis	private Newton/IPM internals (unless intentionally internal)

These boundaries can be enforced by include-guard audits or a build-system layer-visibility policy.

18.8 Priority Roadmap

Priority 1 — Trustworthiness

1. Add independent PF and OPF audit checks (`OpfAudit`).
2. Add derivative finite-difference tests for Jacobians.
3. Add serialisation completeness tests.
4. Add projection mapping reports.
5. Add solver trace reporting for fallback method transitions.

Priority 2 — User Experience

1. Add strict/permissive import modes.
2. Add runtime capability discovery API (`get_solver_capabilities()`).
3. Add better infeasibility diagnostics with probable causes.
4. Add example workflows (see Section ??).
5. Publish JSON schema files.

Priority 3 — Maintainability

1. Enforce dependency boundaries via build-system rules.
2. Split large analysis submodules if necessary.
3. Centralise field descriptors into a machine-readable registry.
4. Keep `engine/` as adapter/façade only.

Priority 4 — Performance and Scalability

1. Benchmark assembly time separately from solve time.
2. Track sparse-matrix nonzero counts across test cases.
3. Add performance regression cases for large networks.
4. Profile three-phase and time-series simulation.
5. Cache symbolic factorisations where safe.

18.9 Example Clean Public Workflow

A mature public API should look like the following. The pattern “load, validate, configure, solve, report” captures the full user journey:

```
#include <hacdcpf/api/hacdcpf.hpp>

int main() {
    // 1. Load with error propagation
    auto system_result = hacdcpf::try_load_json("case.json");
    if (!system_result) {
        std::cerr << system_result.error().message << "\n";
        return 1;
    }
    auto system = system_result.value();

    // 2. Validate before solving
    auto report = hacdcpf::validate(
        system, hacdcpf::ValidationLevel::SolverReady);
    if (!report.ok()) {
        std::cerr << report.summary() << "\n";
        return 1;
    }

    // 3. Structured option assembly
    auto options = hacdcpf::PowerFlowOptions::from_parts(
        hacdcpf::ConvergenceOptions{},
        hacdcpf::LinearSolverOptions{},
        hacdcpf::GlobalizationOptions{},
        hacdcpf::PVPQSwitchingOptions{},
        hacdcpf::ConverterModeOptions{},
        hacdcpf::ZipLoadOptions{},
        hacdcpf::RuntimeOptions{}
    );

    // 4. Solve
    auto pf = hacdcpf::solve_power_flow(system, options);
    if (!pf.converged) {
        std::cerr << pf.diagnostics.termination_reason << "\n";
        return 2;
    }

    // 5. Save result
    hacdcpf::save_power_flow_result_json(pf, "pf_result.json");

    // 6. Post-solve analysis
    auto carbon = hacdcpf::analysis::run_carbon_analysis(system, options);
    hacdcpf::analysis::save_carbon_report_json(carbon, "carbon.json");

    return 0;
}
```

18.10 Final Recommendation Summary

The package has moved substantially in the right direction. The next most valuable work is not adding more solver features, but improving **trust, diagnostics, and maintainability**:

1. Add independent numerical audit reports for PF and OPF.
2. Add automated serialisation completeness tests.
3. Add projection mapping reports.
4. Add runtime backend capability discovery.
5. Enforce dependency boundary rules.
6. Keep analysis modules modular and subdomain-separated.
7. Improve infeasibility and non-convergence explanations.
8. Build benchmark and regression infrastructure.

The main remaining structural risks are:

- **analysis/** becoming a miscellaneous collection without subdomain separation.
- Documentation and serialisers drifting without a field-descriptor registry.
- Incomplete backend capability discovery forcing compile-time conditional code into user applications.
- Three-phase unbalanced models being over-coupled to balanced PF abstractions.
- OPF infeasibility diagnostics remaining too coarse for user debugging.

If the roadmap in Section ?? is followed, the package will be significantly easier to maintain, debug, validate, and adopt by external users.

19 Complete Data Model: Enumerations, Defaults, and Error Types

This chapter provides the complete reference for the lower layers of the `model/` subsystem: the enumeration hierarchy (nine sub-headers under `include/hacdcpf/model/enums/`), the centralised `Defaults` constant struct, the `Result<T>/Error/ErrorCode` monadic result type, and the validation subsystem.

19.1 Enumeration Hierarchy

All domain enumerations live in `include/hacdcpf/model/enums/` and are re-exported by the umbrella header `include/hacdcpf/model/enums.hpp`. Each enum is a scoped `enumclass` in namespace `hacdcpf`.

19.1.1 Bus Type Enumerations

Declared in `model/enums/bus_types.hpp`.

Enum	Enumerator	Meaning
BusType	PQ=1	Constant-power load bus
	PV=2	Voltage-controlled generator bus
	SLACK=3	Swing bus (voltage & angle reference)
	ISOLATED=4	Disconnected bus (removed from solve)
DCBusType	DC_P=1	DC active-power controlled bus
	DC_V=3	DC voltage-controlled bus

19.1.2 Generator Enumerations

Declared in `model/enums/generator_enums.hpp`.

Enum	Enumerator	Meaning
GenCostModel	Polynomial=2	$f(P_g) = c_2 P_g^2 + c_1 P_g + c_0$
	PiecewiseLinear=1	Segmented cost curve
FuelType	Coal, Gas, Oil, Nuclear	Conventional thermal
	Hydro, Wind, Solar	Renewable hydro / wind / PV
	Biomass, Geothermal	Alternative renewables
	Storage	Storage acting as generator
	Unknown=0	Unspecified

19.1.3 Converter and Loss Model Enumerations

Declared in `model/enums/converter_enums.hpp`.

Enum	Enumerator	Meaning
ConverterMode	PQ_MODE	Fixed active + reactive injection
	VDC_Q	DC voltage regulation + fixed Q
	VDC_VAC	DC voltage + AC voltage regulation
LossModelType	Linear	$P_{loss} = a_0 + a_1 I$
	CurrentBased	$P_{loss} = a_0 + a_1 I + a_2 I^2$
DCDCControlMode	Voltage=0	Regulate DC output voltage
	Power=1	Regulate transferred power
	Droop=2	Droop-controlled power sharing
ERPortType	AC=0	AC-side energy-router port
	DC=1	DC-side energy-router port
ERControlMode	PQ=0	Fixed power dispatch
	VF=1	Grid-forming voltage/frequency control
	Droop=2	Droop-based power sharing

19.1.4 Load, Storage, and Renewable Enumerations

Declared in `model/enums/load_enums.hpp`, `storage_enums.hpp`, and `renewable_enums.hpp`.

Enum	Enumerator	Meaning
LoadModel	ConstantPower=0	Standard PQ bus
	ZIP=1	Polynomial: $P = P_0(Z V ^2 + I V + R)$
	Exponential=2	$P = P_0V^\alpha$
LoadPriority	Low=1 to Critical=4	Shedding priority
StorageMode	Idle=0, Charging=1, Discharging=2	Operating mode
MobileStorageStatus	Stationary=0	Not in transit
	InTransit=1	Relocating between sites
	Deployed=2	Operational at new site
RenewableType	Wind=0, SolarPV=1, SolarCSP=2, Hydro=3	Renewable technology
SgenType	PV=0, Wind=1, CHP=2, Diesel=3, FuelCell=4	Distributed generation
PVControlMode	MPPT=0	Max power point tracking
	PQ=1	Fixed P and Q setpoints
	VQ=2	Voltage reactive support
	Curtailed=3	Output curtailment

19.1.5 Switching and Grid Enumerations

Enum	Enumerator	Meaning
SwitchType	CircuitBreaker=0	Full fault-interrupting capability
	Disconnecter=1	No-load isolation
	LoadBreakSwitch=2	Switching under load
	Fuse=3	Overcurrent protection
	Recloser=4	Auto-reclosing
	Sectionalizer=5	Sectionalization
BreakerType	CB=0, LS=1, DS=2	Circuit breaker / load switch / disconnecter
MicrogridMode	GridConnected=0	Connected to utility
	Islanded=1	Autonomous operation
	Transition=2	Mode-change transient
ChargerType	AC_L1=0	Level 1 AC (≤ 3.7 kW)
	AC_L2=1	Level 2 AC (≤ 22 kW)
	DC_Fast=2	DC fast charge (≤ 350 kW)

19.2 EnumStrings Utility

The header `include/hacdcpf/model/enum_strings.hpp` provides header-only inline bidirectional string converters for every enum type. Each type `X` has two functions:

Function	Description
<code>x_str(X) → std::string</code>	Enum value to string
<code>x_from_str(std::string) → X</code>	String to enum (default on unknown)

Converters exist for: `BusType`, `DCBusType`, `ConverterMode`, `FuelType`, `LossModelType`, `LoadModel`, `StorageMode`, `RenewableType`, `SgenType`, `PVControlMode`, `MicrogridMode`, `SwitchType`, and `ChargerType`. They are consumed by `json_io`, `excel_io`, and any future serialisers.

19.3 Defaults: System-Wide Numerical Constants

`include/hacdcpf/model/defaults.hpp` declares a single POD struct `hacdcpf::Defaults` whose static `constexpr` members serve as the single source of truth for all numerical defaults across JSON/Excel import, the MATPOWER parser, model constructors, solver assembly, and tests.

Constant	Value	Units	Description
<code>kBaseMva</code>	100.0	MVA	System base apparent power
<code>kFreqHz</code>	50.0	Hz	Nominal network frequency
<code>kBusVoltage</code>	1.0	pu	Bus voltage magnitude initial value
<code>kBusAngleDeg</code>	0.0	deg	Bus voltage angle initial value
<code>kVoltageMinPu</code>	0.9	pu	AC bus lower voltage bound
<code>kVoltageMaxPu</code>	1.1	pu	AC bus upper voltage bound
<code>kDCVoltageMinPu</code>	0.9	pu	DC bus lower voltage bound
<code>kDCVoltageMaxPu</code>	1.1	pu	DC bus upper voltage bound
<code>kTapRatio</code>	1.0	pu	Transformer tap ratio
<code>kShiftDeg</code>	0.0	deg	Phase-shifting transformer angle
<code>kRateAMva</code>	0.0	MVA	Thermal rating (0 = unlimited)
<code>kGenPmax</code>	10^6	MW	Default generator upper active limit
<code>kGenPmin</code>	0.0	MW	Default generator lower active limit
<code>kGenQmax</code>	10^6	MVAr	Default reactive upper limit
<code>kGenQmin</code>	-10^6	MVAr	Default reactive lower limit
<code>kGenVgPu</code>	1.0	pu	Generator terminal voltage setpoint
<code>kPFTol</code>	10^{-8}	pu	Newton PF convergence tolerance
<code>kPFMaxIter</code>	50	—	Newton PF maximum iterations
<code>kOPFTol</code>	10^{-8}	pu	OPF convergence tolerance
<code>kOPFMaxIter</code>	300	—	OPF maximum iterations
<code>kCostScaleFactor</code>	1.0	\$/h	Objective cost scale factor
<code>kSchemaVersion</code>	"1.0"	—	JSON schema version string
<code>kPackageVersion</code>	"0.5.0"	—	Library release version string

Usage pattern: `usingD=hacdcpf::Defaults;doubletol=D::kPFTol;`

19.4 Structured Error and Result Types

`include/hacdcpf/model/error.hpp` provides lightweight, exception-free error propagation for batch workflows. It is header-only with no dependencies beyond the C++ standard library.

19.4.1 ErrorCode Enumeration

Code	Value	Description
InvalidInput	1000	Malformed input data
ValidationFailed	1001	Pre-solve validation returned errors
MissingSlackBus	1002	No swing bus defined
DisconnectedNetwork	1003	Network has isolated subsystem
SingularAdmittance	1004	Ybus is numerically singular
NotConverged	2000	Newton solver did not converge
MaxIterationsReached	2001	Iteration limit exceeded
NumericalFailure	2002	Floating-point failure (NaN/Inf)
JacobianSingular	2003	Jacobian is numerically singular
HomotopyFailed	2004	Homotopy path tracking failed
FileNotFound	3000	File path does not exist
ParseError	3001	File format / parsing error
SchemaVersionMismatch	3002	JSON schema version incompatible
NotImplemented	9000	Feature not yet implemented
Unknown	9999	Unclassified error

19.4.2 Error Descriptor

Field	Type	Description
code	ErrorCode	Machine-readable error code
message	std::string	Human-readable description
details	vector<ValidationIssue>	Attached validation issues (optional)

Three factory methods simplify common cases:

- `Error::validation_failed(report)` — attaches a `ValidationReport`.
- `Error::not_converged(reason)` — fills code `NotConverged` with a message.
- `Error::parse_error(path,msg)` — fills code `ParseError`.

19.4.3 Result<T> Monad

`Result<T>` is a `std::variant<T,Error>` wrapper that carries either a success value or an error:

Member	Description
<code>has_value() → bool</code>	True if successful
<code>operatorbool()</code>	Alias for <code>has_value()</code>
<code>value() → T&</code>	Access success payload (UB on error)
<code>operator*(), operator->()</code>	Dereference to payload
<code>error() → Error&</code>	Access error payload
<code>value_or_throw() → T&</code>	Access value or throw <code>std::runtime_error</code>

Typical usage:

```
auto res = safe_solve_power_flow(sys, opt);
if (!res) { std::cerr << res.error().message; return; }
auto& r = res.value();
```

19.5 Validation Subsystem

`include/hacdcpf/validation/` contains two headers that implement static pre-solve network validation.

19.5.1 ValidationReport and ValidationIssue

Declared in `validation_report.hpp`:

Type / Member	Description
<code>Severity::Info</code>	Informational note; solving proceeds
<code>Severity::Warning</code>	Degraded results possible; solving proceeds
<code>Severity::Error</code>	Fatal; system cannot be solved as-is
<code>ValidationIssue::severity</code>	Diagnostic severity level
<code>ValidationIssue::component_type</code>	e.g. "ACBus", "VSCConverter"
<code>ValidationIssue::component_id</code>	External ID / name
<code>ValidationIssue::field</code>	Field name that is problematic
<code>ValidationIssue::message</code>	Human-readable description
<code>ValidationReport::is_valid()</code>	True if no Error -severity issues
<code>ValidationReport::has_errors()</code>	Alias for <code>!is_valid()</code>
<code>ValidationReport::has_warnings()</code>	True if any Warning issues
<code>ValidationReport::count(Severity)</code>	Count of issues at a given severity
<code>ValidationReport::add(s,type,id,field,msg)</code>	Append a new issue

19.5.2 Validation Checks

`validate(constHybridPowerSystem&)` in `validation/validate_system.hpp` / `src/validation/validate_system.cpp` runs 15 static checks and returns a `ValidationReport`:

1. **base_mva > 0** — system base must be positive.
2. **Duplicate AC bus IDs** — no two AC buses share an index.
3. **Duplicate DC bus IDs** — no two DC buses share an index.
4. **AC branch bus references** — `from_bus` and `to_bus` exist.
5. **DC branch bus references** — same, in the DC network.
6. **AC voltage limits** — $V_{min} > 0$, $V_{min} < V_{max}$.
7. **Generator P limits** — $P_{min} \leq P_{max}$.
8. **Generator Q limits** — $Q_{min} \leq Q_{max}$.
9. **Generator bus reference** — generator bus exists in AC network.
10. **Slack bus presence** — at least one `BusType::SLACK` bus.
11. **Transformer tap ratio** — $0.5 < \text{tap} < 2.0$.
12. **VSC converter bus references** — AC and DC bus indices exist.
13. **DC-DC converter bus references** — both DC buses exist.

14. **Non-negative AC branch resistance** — $r_{pu} \geq 0$.

15. **AC/DC base MVA consistency** — subsystem base MVA matches system base.

Any `Error`-severity result from `validate()` will prevent the solver from proceeding. The `Error::validation_failed()` factory method wraps a failed report into the structured error type for API return.

20 Solver Infrastructure: Assembly, Projection, Backend, and Utilities

This chapter documents the horizontal infrastructure layers that sit between the high-level `HybridPowerSystem` model and the numerical solver kernels: the **assembly** layer (flat solver-ready data), the **projection** layer (canonical model expansion and bus merging), the **backend** abstraction interfaces (linear, NLP, and QP solvers), and the **thread pool** utility.

20.1 Assembly Layer

The assembly layer (`include/hacdcpf/assembly/`) converts a `HybridPowerSystem` into flat, pre-computed data structures ready for direct consumption by numerical solvers.

20.1.1 SolverData

`assembly/solver_data.hpp` — namespace `hacdcpf::powerflow`.

`SolverData` is the canonical flat representation of a projected network. It owns all component vectors and the precomputed sparse matrices used by every PF/OPF kernel.

Component vectors

Field	Description
<code>ac_buses</code>	AC bus vector (post-projection)
<code>ac_branches</code>	AC branch vector (incl. expanded transformer equivalents)
<code>dc_buses</code>	DC bus vector
<code>dc_branches</code>	DC branch vector
<code>converters</code>	VSC AC/DC converters
<code>dcdc_converters</code>	DC-DC converters
<code>energy_routers</code>	Multi-port energy routers
<code>generators</code>	Synchronous and asynchronous generators
<code>loads</code>	AC constant-power and ZIP loads
<code>flexible_loads</code>	Demand-response flexible loads
<code>static_generators</code>	Distributed generation (inverter-coupled)
<code>renewable_gens</code>	Wind and solar renewable generation
<code>pv_systems</code>	PV array systems (MPPT model)
<code>storage_units</code>	AC battery / flywheel storage
<code>dc_loads</code>	DC bus loads
<code>dc_storage</code>	DC bus storage
<code>dc_static_generators</code>	DC-connected distributed generation
<code>dc_pv_arrays</code>	DC PV arrays
<code>shunts</code>	Fixed and switchable shunts

Field	Description
charging_stations	EV charging stations
chargers	Individual EV chargers
external_grids	External grid equivalents
switches	Switching devices (CB, DS, LBS, etc.)
vpps	Virtual power plants
microgrids	Microgrid aggregates
mobile_storage	Mobile storage units

Numerical parameters and flags

Field	Default	Description
base_mva	100.0	System base MVA
loss_model	Linear	VSC converter loss model type
zip_pw[3]	{1,0,0}	ZIP P weights (Z,I,P fractions)
zip_qw[3]	{1,0,0}	ZIP Q weights
has_component_loads	false	True after ZIP load aggregation
pd_pu, qd_pu	—	Aggregated bus P/Q load (pu)
bus_zip_*	—	Per-bus ZIP weight vectors (Z/I/P, P and Q)
enable_coupled_jacobian	false	Coupled AC/DC Jacobian
enable_augmented_equations	false	VDC/VAC augmented rows
enable_semi_smooth_newton	false	Fischer-Burmeister PV/PQ
ncp_mu	0.0	Smooth-NCP smoothing parameter
pg, qg	—	Aggregated bus generator P/Q (pu)
ybus	—	Complex admittance matrix $\mathbf{Y}_{bus}$ (sparse)
gdc	—	DC conductance matrix $\mathbf{G}_{dc}$ (sparse)
build_id	0	Monotone counter incremented on each rebuild
bus_merge_map	nullopt	Optional zero-impedance bus merge map

Factory and utility functions

Function	Description
make_solver_data(sys)	Build from HybridPowerSystem (copy)
make_solver_data(sys&&)	Build from rvalue (move-efficient)
make_solver_data_projected(projected&&)	Build from already-projected system
aggregate_generation(data)	Sum generator injections onto bus vectors
aggregate_load_demand(data)	Sum load demands onto bus vectors
rebuild_matrices(data)	Recompute $\mathbf{Y}_{bus}$ and $\mathbf{G}_{dc}$

20.1.2 Index Maps

assembly/index_map.hpp maps between external (1-based user-facing) and internal (0-based solver) bus and branch indices.

Type	Description
BusIndexMap::ext_to_pos	unordered_map<int,int>: external $\rightarrow$ internal
BusIndexMap::pos_to_ext	vector<int>: internal $\rightarrow$ external
BranchIndexMap	Same pattern for branches
SystemIndexMap	Combined AC/DC bus and branch maps + optional merge map pointer

20.1.3 Admittance (Y-bus) Builder

assembly/ybus_builder.hpp provides three matrix builders:

Functions

Function	Returns
build_admittance_matrix(data)	Full complex $\mathbf{Y}_{bus}$ for AC Newton PF
build_dc_conductance(data)	Real $\mathbf{G}_{dc}$ for DC PF
build_susceptance_matrix(data,flags)	Real $\mathbf{B}'$ or $\mathbf{B}''$ for FDPF

BpBuildFlags (used with build_susceptance_matrix):

Flag	Default	Effect
zero_resistance	false	Set series $R = 0$ (builds B' for XB-FDPF)
zero_charging	false	Ignore line charging b_{pu}
unit_tap	false	Treat tap magnitude as 1
zero_phase_shift	false	Zero all transformer phase shifts
add_bus_shunts	false	Add bus shunt susceptance $-B_s$ (builds B'')
min_x_pu	0.0	Skip branches where $ x_{pu} <$ this threshold

Two preset builders are provided:

- make_bp_flags(min_x=1e-6) — builds $\mathbf{B}'$ for the FDXB variant.
- make_bpp_flags() — builds $\mathbf{B}''$ for the FDBX variant.

20.2 Projection Layer

The projection layer (include/hacdcpf/projection/) converts a *rich* HybridPowerSystem (with transformers, switches, complex converters) into a *canonical* form ready for solver assembly.

20.2.1 Canonical Network Types

Declared in projection/canonical_network.hpp:

Type	Description
BranchOriginType	Transformer2W, Transformer3W, Switch
BranchExpandEntry	Records which branch was synthesised from which rich element
BranchExpandMap	Collection of BranchExpandEntry records

Type	Description
<code>BusMergeMap::ext_to_int</code>	External bus index $\rightarrow$ merged internal position
<code>BusMergeMap::int_to_ext</code>	Internal position $\rightarrow$ representative external index
<code>BusMergeMap::groups</code>	Per-representative list of merged external buses
<code>BusMergeMap::dead_bus_indices</code>	External indices of dead-island buses stripped
<code>BusMergeMap::branch_orig_to_proj</code>	Original branch pos $\rightarrow$ projected pos (-1 = dead)
<code>BusMergeMap::has_merges()</code>	True if any zero-impedance merging occurred
<code>BusMergeMap::has_dead_buses()</code>	True if any dead islands were stripped
<code>BusMergeMap::identity(n)</code>	Constructs no-op (identity) mapping for n buses

20.2.2 Projection Functions

Declared in `projection/project_to_canonical.hpp`:

Count queries (read-only)

Function	Returns
<code>n_ac_buses(sys), n_dc_buses(sys), n_buses(sys)</code>	Bus counts
<code>n_ac_branches(sys), n_dc_branches(sys), n_branches(sys)</code>	Branch counts
<code>n_generators(sys), n_loads(sys), n_shunts(sys)</code>	Component counts
<code>n_storage(sys), n_renewable_gens(sys), n_converters(sys)</code>	Component counts

Aggregation queries

Function	Description
<code>total_gen_capacity_mw(sys)</code>	Sum of all generator P_{max}
<code>total_gen_pmin_mw(sys)</code>	Sum of all generator P_{min}
<code>total_load_p_mw(sys) / q_mvar</code>	Total AC active / reactive load
<code>total_renewable_capacity_mw(sys)</code>	Total renewable nameplate capacity
<code>total_storage_capacity_mwh(sys)</code>	Total storage energy capacity
<code>total_storage_power_mw(sys)</code>	Total storage peak power
<code>total_system_inertia_mws(sys)</code>	Total rotational inertia
<code>total_emission_tco2_h(sys)</code>	Total CO ₂ emission rate

Topology helpers

- `ac_adjacency_list(sys)` — adjacency list of AC buses.
- `generators_per_bus(sys)` — generator count per bus.
- `print_summary(sys, os) / summary_string(sys)` — formatted system summary.

Projection pipeline `project_to_canonical_models(sys)` performs:

1. Expand `Transformer2W` / `Transformer3W` into equivalent π -model `ACBranch` entries.
2. Expand closed `Switch` devices into zero-impedance branches.
3. Populate `BranchExpandMap` for result un-projection.

Two overloads accept either `const` reference (copy) or `rvalue` (move-efficient).

Zero-impedance bus merging `merge_zero_impedance_buses(sys)` merges buses connected by branches with $|Z| < \text{kBusMergeZThreshold} = 10^{-4}$ pu into equivalence classes, choosing one representative per class. The resulting `BusMergeMap` allows `unproject_bus_vector(merged, map)` to expand solver voltage solutions back to the original bus indexing, returning 0.0 for dead-island buses.

`strip_dead_islands(sys)` removes buses with no generation path (neither direct generators nor grid connections).

20.3 Backend Solver Abstractions

The `include/hacdcpf/backend/` headers expose thin adapter namespaces and abstract interfaces for the three solver categories used by the numerical kernels.

20.3.1 Linear Solver Backend

`backend/linear_solver.hpp` re-exports into `hacdcpf::backend` and `hacdcpf::powerflow` (for backward compatibility):

- `SparseLinearSolver` — abstract base class for sparse LU-type solvers.
- `EigenSparseLUSolver` — Eigen SparseLU concrete implementation.
- `make_default_sparse_solver()` — factory returning an `EigenSparseLUSolver`.

These are defined in `engine/kernel/linear_algebra/linear_solver.hpp` and exposed here for layer-agnostic consumption.

20.3.2 NLP Solver Backend

`backend/nlp_solver.hpp` — namespace `hacdcpf::backend`.

20.3.3 QP Solver Backend

`backend/qp_solver.hpp` — namespace `hacdcpf::backend`.

The `IQPSolver` abstract class solves:

$$\min_x \frac{1}{2}x^\top Hx + c^\top x \quad \text{s.t.} \quad Ax \leq b, \quad l \leq x \leq u. \quad (141)$$

Type / Field	Default	Description
QPStatus		
<code>Optimal</code>	—	Optimal solution found
<code>Infeasible, Unbounded</code>	—	Problem degenerate
<code>MaxIterations, NumericalFailure</code>	—	Solver limits
QPResult		
<code>primal</code>	—	Solution x^*
<code>dual</code>	—	Constraint multipliers
<code>objective</code>	0.0	Optimal objective
<code>iterations</code>	0	Simplex / interior-point iterations

Type / Field	Default	Description
QPOptions		
<code>feasibility_tol</code>	10^{-8}	Primal feasibility tolerance
<code>optimality_tol</code>	10^{-8}	Dual optimality tolerance
<code>max_iterations</code>	10000	Iteration cap

20.4 Thread Pool

`include/hacdcpf/util/thread_pool.hpp` provides a lightweight fixed-size worker pool for parallelising the AC Jacobian evaluation kernel. It exposes three core operations:

Member	Description
<code>ThreadPool(num_threads)</code>	Construct; 0 uses <code>hardware_concurrency()</code>
<code>submit(callable)</code>	Enqueue a task; returns <code>std::future</code>
<code>parallel_for(n, fn)</code>	Execute <code>fn(i)</code> for $i \in [0, n)$ across workers
<code>ThreadPool::global()</code>	Process-wide singleton (created on first call)

The thread count is controlled by `PowerFlowOptions::ac_eval_threads`; setting it to 1 (the default) disables parallelism and uses the calling thread. The pool is persistent across solve calls (no spin-up overhead per solve).

20.5 Detail Utilities

`include/hacdcpf/detail/` contains internal implementation helpers not part of the public API:

Header	Contents
<code>logging.hpp</code>	<code>hacdcpf::detail::log(level, msg)</code> debug logger
<code>string_utils.hpp</code>	String trimming, splitting, and formatting helpers
<code>core_compat.hpp</code>	C++17/20 portability shims (<code>std::span</code> polyfill, etc.)
<code>internal_helpers.hpp</code>	Miscellaneous internal utilities (range checks, etc.)

These headers are consumed only by `src/` translation units. External users should not include them directly; the public API encapsulates all necessary behaviour.

21 Power-Flow Options, Results, and NCP Complementarity

This chapter provides the complete reference tables for `PowerFlowOptions` and its seven sub-structs, all power-flow result types, and the Fischer-Burmeister NCP functions used by the semi-smooth Newton solver.

21.1 PowerFlowOptions Sub-Structs

`PowerFlowOptions` (`include/hacdcpf/power_flow/power_flow_options.hpp`) is a flat layout struct for backward compatibility; it can also be assembled from seven orthogonal sub-structs via `PowerFlowOptions::from_parts()`.

21.1.1 ConvergenceOptions

Field	Default	Units	Description
max_iter	50	—	Newton max iterations (Defaults::kPFMaxIter)
tol	10^{-8}	pu	Newton convergence tolerance
fdpf_max_iter	1000	—	FDPF inner iteration limit

21.1.2 LinearSolverOptions

Field	Default	Units	Description
max_line_search_steps	10	—	Backtracking step budget
max_regularization_steps	8	—	Tikhonov regularisation attempts
regularization_lambda0	10^{-6}	—	Initial regularisation strength
regularization_growth	10.0	—	Regularisation growth factor
max_delta_va_rad	1.50	rad	Clamp on angle step $ \Delta\theta $
max_delta_vm_pu	0.50	pu	Clamp on voltage step $ \Delta V $
max_delta_vdc_pu	0.50	pu	Clamp on DC voltage step $ \Delta V_{dc} $
enable_coupled_jacobian	false	—	Coupled AC/DC Newton block
enable_augmented_equations	false	—	Augmented VDC/VAC control rows
enable_semi_smooth_newton	false	—	Fischer-Burmeister PV/PQ

21.1.3 GlobalizationOptions

Field	Default	Units	Description
strategy	LineSearch	—	LineSearch TrustRegion PseudoTransient
trust_region_radius0	1.0	—	Initial trust-region radius
trust_region_max	10.0	—	Maximum trust-region radius
ptc_delta0	1.0	—	Initial PTC pseudo-timestep
ptc_growth	2.0	—	PTC timestep growth factor

21.1.4 PVPQSwitchingOptions

Field	Default	Units	Description
pv_q_hysteresis_pu	0.01	pu	Q-limit hysteresis band for PV→PQ switch
pv_recover_vm_tol_pu	0.01	pu	Voltage recovery tolerance for PQ→PV
enable_pv_pq_conversion	true	—	Allow dynamic PV/PQ bus switching
enable_auto_swing_selection	true	—	Automatically select slack bus if missing

21.1.5 ConverterModeOptions

Field	Default	Units	Description
converter_vdc_switch_high_pu	0.03	pu	VSC mode switch high threshold
converter_vdc_switch_low_pu	0.01	pu	VSC mode switch low threshold
mode_hysteresis_iters	2	—	Min iterations between mode changes
enable_converter_mode_switching	true	—	Dynamic VSC mode changes
loss_model	Linear	—	VSC loss model type

21.1.6 ZipLoadOptions

Field	Default	Units	Description
pw[3]	{1, 0, 0}	—	P model weights: {Z fraction, I fraction, P fraction}
qw[3]	{1, 0, 0}	—	Q model weights: {Z fraction, I fraction, P fraction}

The ZIP load model computes:

$$P_i = P_i^0 (z_P |V_i|^2 + i_P |V_i| + p_P), \quad (142)$$

$$Q_i = Q_i^0 (z_Q |V_i|^2 + i_Q |V_i| + p_Q). \quad (143)$$

Default weights {1, 0, 0} recover the constant-power (PQ) model.

21.1.7 RuntimeOptions

Field	Default	Units	Description
ac_eval_threads	1	—	Parallel threads for Jacobian evaluation
enable_solver_profiling	false	—	Collect per-call timing counters
enable_iteration_log	false	—	Store per-iteration log entries
verbose	false	—	Print convergence progress to stdout

21.2 Power-Flow Result Hierarchy

All result types are declared in `include/hacdcpf/power_flow/power_flow_result.hpp`.

21.2.1 ResidualBreakdown

Reports per-equation group ℓ_∞ norms at a single iteration:

Field	Description
ac_p_norm	$\ \Delta P_{AC}\ _\infty$
ac_q_norm	$\ \Delta Q_{AC}\ _\infty$
dc_p_norm	$\ \Delta P_{DC}\ _\infty$
converter_p_norm	$\ \Delta P_{conv}\ _\infty$
converter_q_norm	$\ \Delta Q_{conv}\ _\infty$
control_eq_norm	$\ \Delta f_{ctrl}\ _\infty$ (augmented equations)
total_norm()	max of the above six norms

21.2.2 IterationLogEntry

Field	Default	Description
iteration	0	Newton iteration index
residual_norm	0.0	ℓ_∞ residual at this step
step_norm	0.0	ℓ_∞ Newton step size
damping_factor	1.0	Line-search damping α
jacobian_refactorized	false	True if Jacobian was refactorised
lm_fallback_used	false	True if LM step replaced Newton step
breakdown	—	Per-group ResidualBreakdown

21.2.3 SolverDiagnostics

High-level convergence summary stored in every `PowerFlowResult`:

Field	Default	Description
converged	false	Final convergence flag
termination_reason	—	Human-readable reason string
iterations	0	Total Newton iterations
initial_mismatch_norm	0.0	Residual norm at iteration 0
final_mismatch_norm	0.0	Residual norm at convergence
max_p_mismatch_pu	0.0	Worst active power mismatch
max_q_mismatch_pu	0.0	Worst reactive power mismatch
max_i_violation_pu	0.0	Worst current limit violation
condition_estimate	0.0	Jacobian ℓ_1 condition proxy
final_breakdown	—	Per-group residuals at final step
iteration_log	{}	Per-step log (populated if enabled)
warnings	{}	Textual warnings from the solver

21.2.4 SolverProfiling

Detailed performance counters (populated when `enable_solver_profiling=true`):

Field	Default	Description
linear_solver_backend	""	Name of sparse solver used
ac_eval_threads	1	Thread count for Jacobian eval
jacobian_pattern_rebuilds	0	Sparsity pattern rebuilds
jacobian_analyze_calls	0	Symbolic factorisation calls
factorization_calls	0	Numeric factorisation calls
linear_solve_calls	0	Back-substitution calls
regularization_attempts	0	Tikhonov regularisation invocations
line_search_evaluations	0	Total line-search residual evaluations
rejected_steps	0	Steps discarded by line search
p_v_to_p_q_switches	0	PV→PQ bus type transitions
p_q_to_p_v_switches	0	PQ→PV bus type recoveries
converter_mode_switches	0	VSC control mode transitions

Field	Default	Description
residual_by_iter	{}	Residual norm per iteration
eval_jacobian_ms_total	0.0	Total Jacobian wall time (ms)
linear_solve_ms_total	0.0	Total linear solve wall time (ms)
line_search_ms_total	0.0	Total line-search wall time (ms)
raw_residual_norm	0.0	Unscaled residual norm
scaled_residual_norm	0.0	Scaled residual norm (convergence criterion)
condition_estimate	0.0	Proxy Jacobian condition number
regularization_count	0	Successful regularisations applied

21.2.5 Component Flow Result Types

Struct	Fields	Description
BranchFlow	index, from_bus, to_bus	AC branch identifiers
	pf_mw, qf_mvar	From-end apparent power flow
	pt_mw, qt_mvar	To-end apparent power flow
VSCTransfer	index, bus_ac, bus_dc	Converter identifiers
	p_ac_mw, q_ac_mvar	AC terminal power
	p_dc_mw, loss_mw	DC terminal power and loss
DCDCTransfer	index, bus_in, bus_out	DC-DC converter identifiers
	p_in_mw, p_out_mw, loss_mw	Terminal powers and loss
Trafo3WFlow	hv_bus, mv_bus, lv_bus	Three-winding transformer buses
	p/qatHV, MV, LV	Terminal power flows
	loss_mw, loading_pct, rate_mva	Loss and loading metrics
ERPortTransfer	router_index, port_index, bus	Energy-router port
	is_ac, p_mw, q_mvar, v_pu	Port type and power

21.2.6 Top-Level Result Types

Type	Key Fields	Use Case
PowerFlowResult	vm, va, vdc (bus voltages)	Primary AC/DC PF output
	converged, iterations, residual	Convergence summary
	profiling, diagnostics	Performance and diagnostics
	branch_flows, vsc_transfers	Component-level flows
	dcdc_transfers, trafo3w_flows	Transformer and DC-DC flows
	er_port_transfers	Energy-router port flows
DCPowerFlowResult	vdc, converged, iterations, residual	Linearised DC-only PF
AdaptiveSolveResult	vm, va, vdc, islands, branch_flows	Island-decomposed adaptive solve
IslandedSolveResult	vm, va, vdc, islands	Pure island detection result
DistributedSlackResult	vm, va, vdc	Distributed slack solve
	distributed_slack_p	Per-bus slack participation ΔP
	hit_limits	Buses that hit reactive limits

21.3 Fischer-Burmeister NCP Functions

`include/hacdcpf/power_flow/ncp_functions.hpp` provides the complementarity functions used by the semi-smooth Newton PV/PQ handling.

21.3.1 Standard Fischer-Burmeister Function

The smoothed Fischer-Burmeister (FB) NCP function is:

$$\text{FB}(a, b) = \sqrt{a^2 + b^2 + \varepsilon} - a - b, \quad \varepsilon = 10^{-14}. \quad (144)$$

$\text{FB}(a, b) = 0 \iff a \geq 0, b \geq 0, ab = 0$.

Its partial derivatives are:

$$\frac{\partial \text{FB}}{\partial a} = \frac{a}{\sqrt{a^2 + b^2 + \varepsilon}} - 1, \quad \frac{\partial \text{FB}}{\partial b} = \frac{b}{\sqrt{a^2 + b^2 + \varepsilon}} - 1. \quad (145)$$

21.3.2 PV/PQ Complementarity Equation

For a generator bus with reactive limits $[Q_{\min}, Q_{\max}]$ and voltage setpoint $\bar{V}$:

$$F_{NCP} = \text{FB}(Q_{\max} - Q_g, V - \bar{V}) + \text{FB}(Q_g - Q_{\min}, \bar{V} - V). \quad (146)$$

$F_{NCP} = 0$ enforces three complementary conditions simultaneously:

- $Q_{\min} < Q_g < Q_{\max}$: voltage regulation $V = \bar{V}$ (PV).
- $Q_g = Q_{\max}$: $V \leq \bar{V}$ (reactive limit, switch to PQ).
- $Q_g = Q_{\min}$: $V \geq \bar{V}$ (lower limit, switch to PQ).

Jacobian w.r.t. (Q_g, V) :

$$\frac{\partial F_{NCP}}{\partial Q_g} = -\partial_a \text{FB}_1 + \partial_a \text{FB}_2, \quad \frac{\partial F_{NCP}}{\partial V} = \partial_b \text{FB}_1 - \partial_b \text{FB}_2. \quad (147)$$

21.3.3 Smooth (μ -Perturbed) Fischer-Burmeister

Phase 2 of `RobustNonlinearOptions` uses a μ -perturbed smooth FB:

$$\varphi_\mu(a, b) = \sqrt{a^2 + b^2 + 2\mu} - a - b, \quad \mu > 0. \quad (148)$$

As $\mu \rightarrow 0$, this recovers the standard near-zero FB. μ is annealed from `ncp_mu0` to `ncp_mu_min` as the residual decreases, using a two-phase schedule:

$$\mu_{k+1} = \begin{cases} \text{ncp_mu_factor_coarse} \cdot \mu_k & \text{if } \|F\| > \theta \cdot \|F_0\|, \\ \text{ncp_mu_factor} \cdot \mu_k & \text{otherwise (fine phase).} \end{cases} \quad (149)$$

where $\theta = \text{ncp_mu_phase_transition}$ (default 0.1).

The smooth PV/PQ NCP residual and its Jacobian follow the same structure as the standard version but with `smooth_fb` replacing `fischer_burmeister`.

Activity hysteresis (`enable_activity_hysteresis=true`) prevents PV/PQ chattering by requiring `min_active_set_hold_iters = 3` iterations to elapse between consecutive active-set changes.

22 Analysis Suite: Carbon, Distribution, Reliability, and Time Series

This chapter documents the `include/hacdcpf/analysis/` module, which provides post-solve analytical capabilities: carbon emissions tracing, distribution network power flow, three-phase unbalanced power flow, PV array power modelling, reliability and resilience assessments, and time-series simulation.

22.1 Carbon Emissions Analysis

Declared in `analysis/carbon_analysis.hpp`, implemented in `src/analysis/carbon_analysis.cpp`.

`run_carbon_analysis(sys, opt)` performs power-flow-based emissions attribution via two independent methods:

1. **Proportional tracing** — Kirchhoff-based upstream tracing that allocates each generator's output to loads and losses in proportion to power flow direction.
2. **Nodal matrix method** — Builds and solves a linear system $\mathbf{A} \mathbf{c} = \mathbf{q}$ where c_i is the carbon intensity at bus i (tCO₂/MWh).

22.1.1 Options and Result Types

Field (CarbonAnalysisOptions)	Default	Description
<code>enable_tracing</code>	true	Run proportional tracing method
<code>enable_matrix</code>	true	Run nodal matrix method
<code>base_mva</code>	100.0	System base MVA

`CarbonAnalysisResult` aggregates per-component results:

Field	Description
<code>tracing_verified</code>	Tracing mass-balance check passed
<code>matrix_solved</code>	Matrix method converged
<code>matrix_residual</code>	Residual of the linear system
<code>load_carbon</code>	Per-AC-load: demand, carbon intensity, total emissions, supply mix
<code>dc_load_carbon</code>	Same for DC loads
<code>branch_carbon</code>	Per-AC-branch: loss, carbon intensity, generator attribution
<code>dc_branch_carbon</code>	Same for DC branches
<code>bus_carbon</code>	Per-AC-bus carbon intensity (tCO ₂ /MWh)
<code>dc_bus_carbon</code>	Same for DC buses
<code>vsc_carbon</code>	Per-VSC: input/output power, losses, carbon attribution
<code>dcdc_carbon</code>	Per-DC-DC converter: same
<code>storage_carbon</code>	Per-storage: SoC, stored energy, emissions
<code>energy_router_carbon</code>	Per-router: port-level power and emissions
<code>generator_loss_allocation</code>	Per-generator share of network losses
<code>tracing_summary</code>	EmissionsSummary : generation, load, loss, balance error
<code>matrix_summary</code>	Same from matrix method

22.1.2 EmissionsSummary

Field	Description
total_generation_emissions_tco2	Weighted sum of generator emission rates
total_load_emissions_tco2	Emissions attributed to all loads
total_loss_emissions_tco2	Emissions attributed to network losses
balance_error_tco2	$ E_{gen} - E_{load} - E_{loss} $
balance_error_pct	Relative balance error (%)

22.2 Distribution Network Power Flow

Declared in `analysis/distribution_power_flow.hpp`, implemented in `src/analysis/distribution_power_flow.cpp`.

Two solver variants are provided for radial distribution networks:

1. **Backward-Forward Sweep (BFS)** — classical two-pass algorithm for single-source radial feeders. Fast and numerically robust for weakly meshed topologies.
2. **Three-phase Newton-Raphson** — full $3n \times 3n$ abc-domain admittance formulation for meshed and unbalanced networks (`solve_three_phase_nr(sys, opt)`).

22.2.1 Distribution PF Options and Results

Field (DistributionPFOptions)	Default	Units	Description
max_iterations	100	—	Maximum BFS/NR iterations
convergence_tolerance	10^{-6}	pu	Residual convergence tolerance
backward_forward_sweep	true	—	Use BFS (false = use NR)

Field (DistributionPFResult)	Description
converged, iterations, residual	Convergence indicators
buses	DistributionBusResult list: V_m , V_a , P , Q per bus
status	Human-readable solver status string

22.3 Three-Phase Unbalanced Power Flow

Declared in `power_flow/three_phase.hpp`. The primary solver (`solve_three_phase(sys, opt)`) delegates to `analysis::solve_three_phase_nr()` and reports:

Field (ThreePhaseFlowResult)	Description
bus_results	Per-bus three-phase voltages (ThreePhaseBusResult)
converged, iterations, residual	Convergence indicators
total_p_loss_mw	Total three-phase active power loss
total_q_loss_mvar	Total three-phase reactive power loss
max_vuf_percent	Worst-case Voltage Unbalance Factor (%)

ThreePhaseFlowOptions mirrors DistributionPFOptions with an additional `include_shunts` flag (default true).

22.4 PV Array Power Curve Model

Declared in `power_flow/pv_power_curve.hpp`. Implements the empirical five-parameter PV module model:

$$I(V) = I_{sc} \left(1 - \left(\frac{V}{V_{oc}} \right)^a \right)^b \left(1 - c \left(\frac{V}{V_{mpp}} \right)^2 \right), \quad (150)$$

with fixed constants $a = 10.0$, $b = 0.5476$, $c = 0.02381$ (Julia DistributionPowerFlow parity). `compute_pv_power_mw(pv)` evaluates the complete PV system output:

$$P_{array} = I(V_{mpp}) \cdot V_{mpp} \cdot N_{series} \cdot N_{parallel} \cdot \frac{G}{1000} \cdot \eta_{inv}, \quad (151)$$

where G is irradiance (W/m^2) and η_{inv} is inverter efficiency. Falls back to `pv.p_mw` when array parameters are absent.

22.5 Reliability Assessment

Declared in `analysis/reliability_assessment.hpp`. Performs Monte Carlo reliability simulation over a configurable time horizon.

Field (Options)	Default	Units	Description
<code>num_simulations</code>	10000	—	Monte Carlo sample count
<code>time_horizon_years</code>	1.0	years	Simulation time horizon
<code>random_seed</code>	42	—	RNG seed for reproducibility

Metric (Result)	Acronym	Description
<code>lolp</code>	LOLP	Loss of Load Probability
<code>eens_mwh</code>	EENS	Expected Energy Not Supplied (MWh/year)
<code>saidi_hr</code>	SAIDI	System Average Interruption Duration Index (hr/cust/year)
<code>saifi</code>	SAIFI	System Average Interruption Frequency Index (interr/cust/year)

`ReliabilityComponent` describes each component's failure model: `failure_rate_per_year` and `repair_time_hr`.

22.6 Resilience Assessment

Declared in `analysis/resilience_assessment.hpp`, implemented in `src/analysis/resilience_assessment.cpp`.

`run_distribution_resilience_assessment(sys,opt)` evaluates load curtailment under fault scenarios with optional microgrid islanding and storage dispatch:

Field (Options)	Default	Units	Description
<code>fault_duration_hr</code>	1.0	hr	Fault scenario duration
<code>curtailment_penalty</code>	1000.0	—	Load shedding penalty weight
<code>enable_islanding</code>	true	—	Allow microgrid island operation
<code>enable_storage_dispatch</code>	true	—	Dispatch storage during outage
<code>max_restoration_hr</code>	8.0	hr	Maximum restoration time
<code>verbose</code>	false	—	Print diagnostic output

Field (DistributionResilienceResult)	Description
total_ens_mwh	Total Energy Not Served (MWh)
max_curtailment_mw	Peak curtailment power (MW)
restoration_time_hr	Time to restore full supply (hr)
bus_curtailments	Per-bus: curtailed load, duration, ENS
completed,status	Success flag and status message

22.7 Time-Series Power Flow

Declared in `analysis/time_series_pf.hpp`. Provides the data types for multi-period simulation.

22.7.1 Time-Series Data Model

The data types live in the global `hacdcpf` namespace for backward compatibility with JSON serialisation:

Type	Fields	Description
TimeSeriesProfile	id,name,values	Named time profile vector
TimeSeriesData	num_steps=24	Collection of named profiles
	step_duration_hr=1.0	Step width in hours
	profiles	Vector of TimeSeriesProfile

22.7.2 Time-Series PF Results

In namespace `hacdcpf::analysis`:

Field (TimeSeriesPFOptions)	Description
num_threads=1	Parallel threads for multi-step solve
abort_on_failure=false	Stop on first diverged step if true

Field (TimeSeriesPFSnapshot)	Description
step,time_hr	Step index and absolute time
converged	Per-step convergence flag
vm,va,vdc	Bus voltage magnitudes and angles at this step

Field (TimeSeriesPFResult)	Description
all_converged	True if every step converged
num_steps,num_failed	Step counts
snapshots	Ordered vector of TimeSeriesPFSnapshot

Multi-step parallelism is coordinated through the thread pool (§??): each step's `HybridPowerSystem` is projected and assembled independently; threads claim steps from a work queue to avoid shared mutable state.

23 Test Suite: Design, Coverage, and Results

This chapter documents the eight new test suites added to the project during the most recent development cycles. All 308 CTest entries pass (2 skipped due to a parser limitation) in the `build_rel` configuration, giving a total of **306/306 active tests passing** on macOS with Apple Silicon (Accelerate, HiGHS, Gurobi, Ipopt, SCIP, KLU, UMFPACK, PaPILO all active). The two skipped tests (`case533mt_hi` and `case533mt_lo`) contain arithmetic expressions ($135/\sqrt{3}$) in their data files that the current MATPOWER parser does not evaluate; they are correctly identified and skipped at runtime rather than failing.

23.1 Overview of the Eight New Suites

Table ?? summarises the eight new executables and their CTest registration ranges.

Table 150: New test suites registered in `tests/CMakeLists.txt`.

Executable	Focus area	CTest IDs	Count
<code>test_hacdcdss</code>	Model, power models, power flow, DC OPF smoke tests	48–63	16
<code>test_io_json</code>	JSON I/O, MATPOWER parsing, round-trip fidelity	64–97	34
<code>test_rich_components</code>	24-component-type JSON round-trips and builders	98–122	25
<code>test_matpower_cases</code>	Power-flow convergence on 87 MATPOWER cases	123–209	87
<code>test_validation</code>	Validation API, levels, regression guardrails	210–239	30
<code>test_solver_capabilities</code>	Compile-time solver flags, OPF audit helper	240–255	16
<code>test_advanced_pf</code>	Island detection, distributed slack, PV/PQ control	256–281	26
<code>test_three_phase_pf</code>	Three-phase data model, projection, meshed, JSON	282–307	26
Total (new)		48–307	260

23.2 `test_hacdcdss`: Model, Power Models, Power Flow, DC OPF

`tests/test_hacdcdss.cpp` provides smoke-level coverage for the four major logical modules exposed through the `hacdcpf` API. The file was completely rewritten from the original version (which referenced the now-defunct `hacdcdss::` library) using the current `hacdcpf` namespace and type system.

23.2.1 Module: Model (tests #48–#51)

- **Construct and validate 2-bus AC network:** a slack bus plus a 50MW PQ load bus with a single π -model branch and one generator are assembled programmatically; `val::validate(sys)` returns `ok()==true`.
- **Out-of-range branch bus:** setting `to_bus=99` on a 2-bus network triggers an **Error**-level issue.
- **Missing slack bus:** removing the slack type and `is_slack` flag causes validation to fail.
- **Hybrid AC/DC network:** a 2-bus DC network with a `VSCConverter` is appended; all collections are verified to have the expected cardinality.

23.2.2 Module: Power Models (tests #52–#54)

Three tests verify Y-bus correctness through observable physical effects:

- **Flat-start, zero-load network:** residual is below 10^{-6} pu after one iteration.

- **Y-bus symmetry:** swapping `from_bus/to_bus` on every branch in a symmetric π -model network produces identical per-bus voltage magnitudes, confirming that the admittance matrix is symmetric.
- **P/Q consistency:** after convergence all bus voltage magnitudes are positive and all angles are finite.

23.2.3 Module: Power Flow (tests #55–#60)

- **Result sizing:** `r.vm` and `r.va` have the same length as `sys.ac.buses`.
- **Lightly loaded 2-bus:** 10 MW load; converged with slack voltage held at 1.0 pu and load-bus voltage in $[0.9, 1.1]$ pu.
- **Physical voltage monotonicity:** higher load produces a lower load-bus voltage.
- **IEEE-14 AC/DC builder:** the pre-built case converges with residual $< 10^{-4}$.
- **Island detection:** a single-bus islanded system completes without exception.
- **Distributed slack:** two generators on the slack bus yield a converged result.

23.2.4 Module: DC OPF (tests #61–#63)

- **3-bus economic dispatch:** a 3-bus network with a cheap (\$20/MWh) and an expensive (\$40/MWh) generator and 30 MW load; the cheaper generator dispatches at least as much as the expensive one, and total dispatch equals 30 MW within 1 MW.
- **Single-generator coverage:** on the lightly loaded 2-bus network the single generator dispatches within its declared $[P_{\min}, P_{\max}]$ bounds.
- **case9 feasibility:** the case-9 MATPOWER network is parsed, solved, and every generator's dispatch is within its MW limits.

All 16 tests pass.

23.3 test_io_json: JSON I/O and MATPOWER Parsing

`tests/test_io_json.cpp` validates the `hacdcpf::io` module through 34 independent test cases covering four areas.

23.3.1 JSON Round-trip Fidelity (tests #64–#67)

Each test constructs a `HybridPowerSystem`, serialises it with `to_json()`, deserialises with `from_json()`, and asserts that bus count, branch count, and generator count are preserved exactly. Compact-output mode (`indent=0`) is verified to produce a non-empty string with no leading whitespace.

23.3.2 Safe Monadic API (tests #68–#75)

Eight tests exercise the `Result<T>`-returning wrappers:

- **try_from_json:** valid input returns a success `Result`; malformed JSON and empty strings return `Error`.
- **try_load_json:** a non-existent file returns an error variant whose code is `FileNotFound`; a valid file path returns a success variant.

- **ImportMode::Permissive**: permissive mode still parses a well-formed file without throwing.
- File round-trip via `save_json/load_json` for the IEEE-24 builder case.

23.3.3 OPF/PF Result Serialisation (tests #76–#77)

An `OPFResult` with a known objective value is serialised and the `converged` flag and floating-point objective are verified after round-trip. A minimal `PowerFlowResult` is verified to produce a non-empty JSON object.

23.3.4 MATPOWER Parser (tests #78–#97)

Nineteen tests parse representative MATPOWER `.m` files from `data/`:

- **Topology checks** (case5, case9, case14, case30, case57, case118, case300, case33bw, case39, case_ieee30, case24_ieee_rts): bus counts, branch endpoints, generator-limit fields.
- **Cross-mode round-trip** (case14): `parse` $\rightarrow$ `to_json` $\rightarrow$ `from_json` $\rightarrow$ power-flow convergence.
- **Heavy cases** (case1354pegase, case2383wp) parse without throwing.
- **Piecewise economics file** (case3_case4_formal_soc_piecewise.json): confirmed to be valid non-empty JSON (economics schema, not a `HybridPowerSystem` schema).

All 34 tests pass. The single SOC-piecewise test was relaxed from a bus-count assertion to a JSON-parse check once it was confirmed that the file stores piecewise cost curves rather than network topology.

23.4 test_rich_components: 24-Type Component Coverage

`tests/test_rich_components.cpp` exercises every non-trivial component type defined in `hacdcpf/model/`.

23.4.1 Component Round-trip Tests (tests #98–#118)

For each of the 21 individual component types listed in Table ??, the test:

1. Constructs a minimal `HybridPowerSystem` and appends an instance of the component with representative field values.
2. Serialises with `to_json` and deserialises with `from_json`.
3. Asserts that the collection size and at least two key fields are preserved within floating-point tolerance ($|e| < 10^{-9}$).

Note: The `s_max_mva` field of `DCBranch` is present in the struct but is not round-tripped through the JSON serialiser; the test therefore checks `rate_a_mva` (which is serialised) instead.

23.4.2 System Builder Tests (tests #119–#122)

Four larger builders are exercised end-to-end:

- **IEEE-14 AC/DC builder**: JSON round-trip preserves all component collections (buses, branches, generators, converters, DC side).
- **Case33 microgrid builder**: system converges after a full `to_json/from_json` round-trip.

Table 151: Component types covered by `test_rich_components`.

Component	Key fields verified	Location
Transformer2W	vn_hv_kv, sn_mva	sys.ac
Transformer3W	sn_mva	sys.ac
StaticGenerator	p_mw, q_mvar	sys.ac
RenewableGen	p_mw, qmax_mvar	sys.ac
PVSystem	p_mw, pmax_mw	sys.ac
Storage	pmax_mw, soc_init	sys.ac
FlexibleLoad	p_mw, flex_up_mw	sys.ac
AsymmetricLoad	pa_mw, pb_mw, pc_mw	sys.ac
Shunt	gs_mw, bs_mvar	sys.ac
ChargingStation	p_total_kw	sys.ac
AsynchronousMotor	sn_mva, cos_phi	sys.ac
MobileStorage	p_mw, soc_init	sys
VirtualPowerPlant	p_output_mw, pmax_mw	sys.ac
Microgrid	internal_buses, p_exchange_mw	sys.ac
VSCConverter	p_ac_mw, q_ac_mvar	sys
DCDCConverter	p_set_mw	sys.dc
EnergyRouter	ports[0].p_set_mw	sys
DCLoad	p_mw	sys.dc
PVArrayDC	p_set_mw	sys.dc
StaticGeneratorDC	p_set_mw, pmax_mw	sys.dc
DCBranch	r_pu, rate_a_mva	sys.dc

- **Dist33 microgrid DER builder:** DER collections (storage, renewables, PV systems, flexible loads) are non-empty after construction.
- **Comprehensive hybrid builder:** VSC converters and DC side are populated after construction.

All 25 tests pass.

23.5 test_matpower_cases: MATPOWER Convergence Coverage

`tests/test_matpower_cases.cpp` exercises power-flow convergence across all three tiers of the MATPOWER library bundled in `data/`. Following an expansion in the most recent cycle, the suite now covers **87 cases** (up from 52), with 2 skipped due to a parser limitation.

23.5.1 Tier-1: Core Cases (tests #123–#135)

Nine canonical cases from 9 to 300 buses are solved with the default Newton solver and checked for convergence (`r.converged==true`) and finite residual. Additional physical sanity checks include:

- **case5:** total generation approximately covers total load (within 5% tolerance).
- **case9 slack-bus voltage:** the solved voltage at the slack bus is within the regulatory range $[0.95, 1.10]$ pu.
- **case14 slack angle:** the slack bus angle is zero within 10^{-6} rad.

- **case14 generation–load balance:** total P_{gen} covers total P_{load} within 5 %.
- **case30 Newton vs FDPF:** both solvers converge and their bus-voltage vectors agree within 10^{-4} pu (per-element L^∞ norm).

All 13 Tier-1 tests pass.

23.5.2 Tier-2: Medium Cases (tests #136–#204)

55 cases from 4 to 2000 buses are solved individually; each is checked for convergence. A batch test (#205) verifies that the convergence rate across all 54 non-skipped cases is at least 85 %. In practice the observed rate is **100 %** (54/54 active cases; 2 skipped for arithmetic-expression parser limitation).

The original 25-case Tier-2 suite has been expanded with 30 additional cases covering small distribution networks, the IEEE RTS-GMLC benchmark, and additional European/synthetic transmission systems.

23.5.3 Tier-3: Large Cases (tests #136–#204, large/very_large tag)

Fourteen large transmission systems (3012 to 82000 buses) are solved; each is asserted to converge. The suite has been extended with five additional very-large cases: `case6495rte`, `case6515rte`, `case_ACTIVSg25k`, `case_ACTIVSg70k`, and `case_SyntheticUSA` (the Synthetic US 82000-bus model).

All 14 Tier-3 tests pass.

23.5.4 Safe API and DC OPF (tests #206–#209)

- **safe_solve_power_flow** (case14, case118): the `Result<PowerFlowResult>`-returning wrapper returns a success variant for both cases.
- **DC OPF case9:** the objective value is finite and the solve returns without exception.
- **DC OPF case14:** the generator dispatch in the solution is within the declared MW limits for every generator.

23.6 test_validation: Validation API Coverage

`tests/test_validation.cpp` covers the `hacdcpf::validation` module through 30 test cases.

23.6.1 ValidationReport API (tests #210–#213)

- `ok()` is an alias for `is_valid()`.
- `summary()` formats issue counts in a human-readable string for a system with known errors and warnings.
- `summary()` returns an empty string for a clean system.
- `count(severity)` returns the correct count for each `IssueSeverity` level.

23.6.2 Positive and Negative Validation (tests #214–#223)

Ten tests confirm that the validator correctly accepts or rejects:

- **Positive:** 2-bus system, IEEE-14 AC/DC builder, case9 parsed from MATPOWER, case118 parsed from MATPOWER.
- **Negative (structural errors):** missing slack bus, duplicate bus IDs, dangling branch endpoint, inverted voltage limits, inverted generator MW limits, zero `base_mva`.

23.6.3 ValidationLevel Behaviour (tests #224–#230)

23.6.4 Validate-then-Solve Pattern (tests #231–#234)

Two integration tests:

- **case14 happy path:** `val::validate(sys)` returns a clean report, then `solve_power_flow` converges.
- **Faulty system:** a system with a missing slack bus fails validation before any solver is called, confirming that validation acts as a correctness gate.

Two further tests confirm that VSC converters referencing non-existent AC or DC buses produce `Error`-level issues.

23.6.5 Regression Guardrails (tests #235–#239)

Five cases (case5, case30, case57, IEEE-14 builder, IEEE-118 builder) are asserted to always produce a clean validation report. These tests act as regression guardrails against inadvertent breakage of canonical inputs.

All 30 validation tests pass.

23.7 test_solver_capabilities: Compile-time Flags and Audit

`tests/test_solver_capabilities.cpp` covers the `SolverCapabilities` struct and its companion functions.

23.7.1 SolverCapabilities Struct (tests #240–#246)

- **Default construction:** a value-initialised struct has `power_flow_available==true`, all flags in a known initial state.
- **get_solver_capabilities():** returns the struct by value and is `noexcept`; repeated calls return the same flags.
- **Always-on features:** `power_flow_available`, `dc_power_flow`, and `linearised_dc_pf` are `true` unconditionally.
- **MIP dependency:** `integer_variables` requires at least one MIP backend; without any MIP backend the flag is `false`.
- **Linear-algebra independence:** sparse-linear-algebra flags (`klu_available`, `umfpack_available`) are independent of MIP solver flags.

23.7.2 Compile-time Solver Flags (tests #247–#252)

Table ?? shows the expected runtime values on the test machine (all backends active).

23.7.3 `verify_opf_result` Audit (tests #253–#255)

- **Converged result passes audit:** a clean `OPFResult` built from `case9` returns `feasible()==true`.
- **Non-converged result:** setting `converged=false` causes `infeasibility_hints` to be non-empty.
- **Feasibility alias:** `audit.feasible()` returns `true` for a clean result with no hints.

All 16 solver-capability tests pass.

23.8 `test_advanced_pf`: Island Detection, Distributed Slack, PV/PQ Control

`tests/test_advanced_pf.cpp` (CTest #256–#281) adds 26 tests and 119 assertions covering three power-flow control features that were not exercised by any existing suite.

23.8.1 Island Detection and Islanded Solving (tests #256–#266)

Five tests verify the `detect_islands()` function on the 9-bus and 14/118-bus MATPOWER cases, including forced outage scenarios that split the network into two or three islands. A further six tests cover the `solve_islanded_power_flow()` and adaptive solver (`solve_power_flow_adaptive()`) paths: single-island equivalence, live DG-only islands, multi-island convergence, and dead-bus voltage assignment.

23.8.2 Distributed Slack Buses (tests #267–#276)

Six tests check the `build_distributed_slack_factors()` helpers for capacity-based, uniform, and droop participation. Four integration tests confirm that `case9`, `case14`, and `case30` MATPOWER cases converge with distributed slack active.

23.8.3 PV/PQ Conversion and Integration (tests #277–#281)

Three tests confirm that the `enable_pv_pq_conversion` option produces consistent results on `case118` and `case2383wp`, and that the try-without-then-retry adaptive pattern works. Two integration tests combine island detection with distributed slack on the same system.

The fast path (`enable_pv_pq_conversion=false`) was introduced as a Tier-3 optimisation: it reduces the Synthetic USA 82 000-bus solve time from ≈ 2.24 s to ≈ 0.73 s. All 26 tests pass.

23.9 `test_three_phase_pf`: Three-Phase Power Flow

`tests/test_three_phase_pf.cpp` (CTest #282–#307) adds 26 tests and 94 assertions covering the three-phase data model and its projection-based power-flow path.

23.9.1 Data Model (tests #282–#286)

Five tests construct and query every three-phase component type (`ThreePhaseACBus`, `ThreePhaseACLine`, `ThreePhaseLoad`, `ThreePhaseGenerator`) and verify field assignment, struct member counts, and `ThreePhaseACSystem` assembly.

23.9.2 Projection-Based Solve (tests #287–#292)

The library implements three-phase power flow via a projection step inside `solve_power_flow()`: when a `HybridPowerSystem` contains a `three_phase_ac` sub-system but `ac.buses` is empty, the three-phase network is automatically converted to a balanced single-phase equivalent (sum of per-phase injections) before the Newton solve.

Three projection tests confirm that the conversion runs, that per-bus total load is preserved, and that the projected single-phase equivalent converges. Three balanced-radial tests verify that slack-bus voltage is exactly 1.0 pu, that all non-slack voltages are in the range [0.8, 1.1] pu (a heavily loaded 10 MVA feeder at 39 % loading can drop below 0.9 pu), that phase angles decrease monotonically down the feeder, and that Newton converges in fewer than 15 iterations.

23.9.3 Unbalanced, Meshed, and Multi-Source Topologies (tests #293–#299)

Three unbalanced-load tests confirm that asymmetric per-phase loading, zero-phase loading, and load redistribution (same total, different split) all converge. Two meshed-topology tests construct a 3-bus ring (1–2–3–1) and verify convergence and improved voltage profile relative to the radial equivalent. Two multi-source tests add a distributed generator (DG) at a load bus and confirm reduced slack injection and convergence when every load bus carries a DG.

23.9.4 Transformer Support (test #300)

One test constructs a `ThreePhaseTransformer` between buses 2 and 3 (11 kV $\rightarrow$ 0.4 kV, $S_n = 0.5$ MVA, $v_k = 4\%$) and verifies that the projected system converges.

23.9.5 Distribution PF Stub and JSON Round-Trip (tests #301–#307)

Two tests confirm that `analysis::solve_distribution_power_flow()` returns `status=="not_implemented"` and that the `DistributionPFOptions` struct compiles and runs.

Five JSON round-trip tests serialise a `HybridPowerSystem` containing a `three_phase_ac` sub-system via `hacdcpf::io::to_json()` / `from_json()` and verify that bus counts, voltage setpoints, load values, line parameters, and power-flow results are preserved exactly.

All 26 tests pass in ≈ 0.01 s.

23.10 Quantitative Results and Timing

This section records the per-suite assertion counts, wall-clock execution times, and representative accuracy metrics measured on the reference build (macOS 14, Apple M4, `build_rel`, all backends active). All times are *real* wall-clock seconds from the CTest output.

23.10.1 Per-Suite Assertion and Timing Summary

Table ?? lists the number of assertions (from the Catch2 compact reporter) and the accumulated CTest time for each suite.

The total CTest wall-clock time is **11.70 s** for all 308 entries, of which 5.00 s is the performance sweep (`test_powerflow_performance`) and 1.16 s is the ACOPF vs DCOPF comprehensive comparison. Excluding the performance suite, all functional tests complete in under 6.8 s.

The `test_matpower_cases` suite time improved significantly from ≈ 5.0 s to ≈ 2.5 s following the PV/PQ conversion optimisation introduced in Section ??: the fast path (`enable_pv_pq_conversion==false`) now processes the 82 000-bus Synthetic USA model in ≈ 0.73 s instead of ≈ 2.24 s.

23.10.2 Tier-3 Scaling: Bus Count vs. Solve Time

Table ?? shows how Newton power-flow solve time scales with system size for Tier-3 cases. All times are CTest wall-clock seconds on the reference machine; they include process startup, MATPOWER file parsing, and the actual Newton solve.

The solver exhibits sub-linear scaling for the RTE/PEGASE families: the 6 500-bus cases take the same time as 8 000-bus cases due to favourable network structure. The Synthetic USA 82 000-bus model is the most computationally demanding, taking 0.73 s, compared to 2.24 s before the PV/PQ conversion fast path was introduced (see Section ??). All Tier-3 cases remain well within the 5 s CTest timeout.

23.10.3 Key Accuracy Metrics

The tests enforce the following quantitative accuracy thresholds; observed values from representative cases are recorded in Table ??.

The DC OPF solver (NativeLCQP) converges to a relative tolerance of $\approx 10^{-6}$ on the dual gap; for the case-14 instance the reported objective is \$7 642.59/h and convergence is reached in under 0.001 s.

23.10.4 Performance Sweep (test #256)

`test_powerflow_performance` runs a single Catch2 test case that internally sweeps 75 MATPOWER cases across three solver configurations (Tier-1 through Tier-3). The three asserted properties are:

- Tier-1 convergence rate $\geq 90\%$ (observed: 100 %).
- Tier-2 extended convergence rate $\geq 85\%$ (observed: 100 %).
- Tier-3 large-system count ≥ 11 cases converging (observed: 11/11).

Total wall-clock time for the sweep is **5.00 s** on the reference machine.

23.11 Summary and Pass Rates

The MATPOWER Tier-2 batch test (#205) enforces a minimum convergence rate of 85 %; the observed rate is 100 % (54/54 active medium-sized cases). The Tier-3 suite covers 14 large transmission systems up to 82 000 buses (the Synthetic USA national model), all of which converge on the first try with the default Newton solver.

Table 152: Tier-2 MATPOWER cases solved in `test_matpower_cases` (original 25 + 30 new).

Case file	Buses	Network type
<i>Original cases</i>		
case33bw	33	Distribution (radial)
case33mg	33	Microgrid
case69	69	Distribution (radial)
case85	85	Distribution
case89pegase	89	PEGASE transmission
case118zh	118	IEEE 118-bus variant
case141	141	Transmission
case145	145	Transmission
case_ieee30	30	IEEE 30-bus benchmark
case1197	1197	European transmission
case1354pegase	1354	PEGASE
case1888rte	1888	French RTE network
case1951rte	1951	French RTE network
case2383wp	2383	Polish winter-peak
case2736sp	2736	Polish summer-peak
case2737sop	2737	Polish summer off-peak
case2746wp	2746	Polish winter-peak
case2746wop	2746	Polish winter off-peak
case2848rte	2848	French RTE network
case2868rte	2868	French RTE network
case2869pegase	2869	PEGASE
case_RTS_GMLC	73	IEEE RTS-GMLC
case_ACTIVSg200	200	Synthetic US grid
case_ACTIVSg500	500	Synthetic US grid
case_ACTIVSg2000	2000	Synthetic US grid
<i>New cases (30 additional)</i>		
case4gs	4	Distribution
case4_dist	4	Distribution
case6ww	6	Transmission
case9Q	9	Modified IEEE 9-bus
case9target	9	Target dispatch
case10ba	10	Brazilian distribution
case12da	12	Danish distribution
case15da	15	Danish distribution
case15nbr	15	Distribution (NB/radial)
case16am	16	AM distribution
case16ci	16	CI distribution
case17me	17	ME distribution
case18	18	Transmission
case18nbr	18	Distribution
case22	22	Transmission
case28da	28	Danish distribution
case30Q	30	Modified IEEE 30-bus
case30pwl	30	Piecewise-linear cost
case34sa	34	South African distribution
case38si	38	Sicilian distribution

Table 153: Tier-3 MATPOWER cases solved in `test_matpower_cases` with wall-clock solve times (macOS, Apple M4, `build_rel`).

Case file	Buses	Network type	Solve time
<i>Original cases</i>			
case3012wp	3 012	Polish winter-peak	0.05 s
case3120sp	3 120	Polish summer-peak	0.04 s
case3375wp	3 375	Polish winter-peak	0.04 s
case6468rte	6 468	French RTE	0.06 s
case6470rte	6 470	French RTE	0.09 s
case8387pegase	8 387	PEGASE	0.09 s
case9241pegase	9 241	PEGASE	0.18 s
case13659pegase	13 659	PEGASE	0.14 s
case_ACTIVSg10k	10 000	Synthetic US grid	0.08 s
<i>New cases (5 additional)</i>			
case6495rte	6 495	French RTE	0.09 s
case6515rte	6 515	French RTE	0.09 s
case_ACTIVSg25k	25 000	Synthetic US grid	0.18 s
case_ACTIVSg70k	70 000	Synthetic US grid	0.54 s
case_SyntheticUSA	82 000	Synthetic US national grid	2.24 s

Table 154: Expected behaviour of each `ValidationLevel`.

Level	Contract verified by tests
Basic	Valid system has no issues. Electrical errors are reported; pure style/warning issues are suppressed.
Electrical	Missing slack bus is caught. Issues are a subset of those reported at <code>SolverReady</code> level.
SolverReady	Result is identical to calling <code>val::validate(sys)</code> with no level argument.
Strict	Warnings are promoted to errors. A valid system with no warnings remains valid at this level.

Table 155: Solver capability flags on the reference build (macOS, all backends active).

Flag	Compile define	Observed value
ac_opf_available	HACDCPF_HAVE_IPOPT or HiGHS	true
dc_opf_available	HACDCPF_HAVE_HIGHS_LIB	true
power_flow_available	(always)	true
highs_available	HACDCPF_HAVE_HIGHS_LIB	true
ipopt_available	HACDCPF_HAVE_IPOPT	true
klu_available	HACDCPF_HAVE_KLU	true

Table 156: Assertions and wall-clock time per test suite (reference build).

Suite	Tests	Assertions	CTest time (s)	Notes
test_hacdcpf	20	50	≈ 0.20	Core API
test_all_components_pf	4	163	≈ 0.04	29-type PF
test_enhanced_solver	2	28	≈ 0.13	Semi-smooth / homotopy
test_acopf_dcopf_crossval	4	28	≈ 1.18	ACOPF vs DCOPF (heavy)
test_scaling_regression	7	16	≈ 0.07	Scaling + conditioning
test_unified_pf_upgrade	10	39	≈ 0.10	PF options refactor
test_hacdcoss	16	58	≈ 0.16	Model/PF/OPF smoke
test_io_json	34	277	≈ 0.36	JSON I/O + MATPOWER
test_rich_components	25	85	≈ 0.25	21 component types
test_matpower_cases	87	2 142	≈ 2.50	85 pass, 2 skip
test_validation	30	50	≈ 0.30	Validation API
test_solver_capabilities	16	58	≈ 0.16	Solver flags & audit
test_advanced_pf	26	119	≈ 0.03	Islands, distr. slack, PV/PQ
test_three_phase_pf	26	94	≈ 0.01	Three-phase projection & JSON
test_powerflow_performance	1	3	5.00	75-case perf sweep
Grand total	308	$\approx 3\,060$	11.70	

Table 157: Power-flow solve time vs. bus count for large MATPOWER cases (Tier-3). Times include process startup and file parse overhead (≈ 0.01 s).

Case	Buses	Wall time (s)	Converged
case3012wp	3 012	0.05	✓
case3120sp	3 120	0.04	✓
case3375wp	3 375	0.04	✓
case6468rte	6 468	0.06	✓
case6470rte	6 470	0.09	✓
case6495rte	6 495	0.09	✓
case6515rte	6 515	0.09	✓
case8387pegase	8 387	0.09	✓
case_ACTIVSg10k	10 000	0.08	✓
case9241pegase	9 241	0.18	✓
case13659pegase	13 659	0.14	✓
case_ACTIVSg25k	25 000	0.18	✓
case_ACTIVSg70k	70 000	0.54	✓
case_SyntheticUSA	82 000	0.73	✓

Table 158: Accuracy thresholds enforced by the test suite.

Metric	Threshold	Representative observed value
Newton residual (convergence)	$< 10^{-4}$ pu	$\ll 10^{-6}$ pu for small/medium cases
Flat-start residual (zero load)	$< 10^{-6}$ pu	≈ 0 (machine precision)
Slack-bus voltage (case9)	$\in [0.95, 1.10]$ pu	≈ 1.0 pu
Slack-bus angle	$ \theta < 10^{-6}$ rad	0 rad (by convention)
Generation-load balance (case14)	within 5 %	< 1 %
Newton vs FDPF agreement (case30)	$< 10^{-4}$ pu (L^∞)	$\approx 10^{-6}$ pu
Generator dispatch within limits	strict $\leq$	verified for case9, case14, case118
DC OPF objective (case14)	feasible, finite	dispatch respects $[P_{\min}, P_{\max}]$
Y-bus symmetry	identical per-bus V_m	exact match (passive π -model)

Table 159: Test pass summary across all six new suites.

Suite	IDs	Count	Passed	Rate
test_hacdcdss	48–63	16	16	100 %
test_io_json	64–97	34	34	100 %
test_rich_components	98–122	25	25	100 %
test_matpower_cases	123–209	87	85 (*)	100 % (active)
test_validation	210–239	30	30	100 %
test_solver_capabilities	240–255	16	16	100 %
test_advanced_pf	256–281	26	26	100 %
test_three_phase_pf	282–307	26	26	100 %
Pre-existing suites	1–47, 308	48	48	100 %
Grand total	1–308	308	306 (*)	100 % (active)

* 2 tests skipped: parser does not evaluate $135/\sqrt{3}$ arithmetic.